

Analysis of Continuous Data project

Thomas Sertijn, Ilja Van Bever, Lieselot Van de Putte

Research question

For this research the effect of socio-economic disadvantage to violent crime rates was investigated. More specifically we want to explore the association between poverty and violent crime rates in the USA. Becker (1968) stated that the decision to commit crime is a rational choice where people weigh the benefits and costs of committing crime against each other. It could be argued that the incentive to commit crime is higher for people who have a lower income, as they have more to gain and less to lose by committing crimes. Following this, we would then also expect that communities with higher poverty rates will also be associated with higher crime rates.

Design of the dataset

For this research, a community-level dataset related to data gathered for the prediction of serious crime rates in the US is used. This dataset combines 1990 U.S. Census socio-economic data, 1990 law enforcement data from the Law Enforcement Management and Admin Stats (LEMAS) survey, and 1995 FBI crime data, thereby creating two cohorts. For the FBI crime data, it is mentioned that states with a lower amount of visitors have a lower per capita crime rate and vice versa. The LEMAS survey covers all communities with police departments of at least 100 officers and a random sample of smaller departments. If communities were absent from either the crime or census datasets (e.g., those with very small departments), then they were removed. All demographic data is from 1990, but per-capita crime rates use 1995 population counts. Finally, rape counts, a component of violent crime, are missing in some states due to inconsistent reporting, which resulted in missing total violent crime values for those states.

Methods

For the purpose of our research question we selected a subset of variables present in the dataset. Some economic variables: *PctPopUnderPov*, *perCapInc*, and *PctEmploy*, which give some information about economic status of the community, and a sociological subset, containing education levels in the communities, combined with *NummImmig*, *RacePctBlack*, *AgePct12t29*, all variables giving some info about the social composition and demographic structure of the communities.

1. To gain an initial understanding of the association between poverty rates (*PctPopUnderPov*) and violent crime rates (*ViolentCrimesPerPop*), a univariate regression is performed.
2. To analyze if the relationship is confounded by other socio-economic variables, whether there are relevant interaction effects at play and to see if the inclusion of these other socio-economic variables improves the performance of our model substantially, a multivariate regression is performed.

Before building the models, some descriptive analysis of the dataset and variables is performed. Next, the dataset is randomly split into a training dataset (80% of the data) and a holdout set (20% of the data).

For the multivariate model, an All-variable procedure is employed to test every combination of possible variables as a model. The best model is chosen based on the Bayesian Information Criterion (BIC). Next, the assumptions of this best model are checked and partial regression plots are generated to check if each of the added variables are in the correct functional form. Afterwards, the most appropriate interaction term with the main predictor *PctPopUnderPov* is selected based on the same BIC procedure. Finally, multicollinearity is assessed using the Variance Inflation Factor (VIF) to obtain a more interpretable model. After determining the final model, some model diagnostics are calculated to determine outliers and influential points. Finally the model is validated by the holdout dataset.

Data preparation

The variable *NumImmig* is converted to a percentage of the total population, since the outcome variable *ViolentCrimesPerPop* (total number of violent crimes per 100K population) is expressed relative to the population size. We call this converted variable *PctImmig*. It's important to mention that this is not an exact transformation, because all demographic data is from 1990, but per-capita crime rates use 1995 population counts. By examining the missing data, there were 221 (of the 2215) observations identified as NA values for the outcome variable *ViolentCrimesPerPop*. No imputations were used in order to have a more transparent model building procedure and to avoid overestimating the quality of the model fit. Therefore these observations were not taken into account for the model building. It can be noted that the variables *countyCode* and *communityCode* are also frequently unknown.

Descriptive analysis

Univariate descriptive analysis is performed, for both the missing values and the non-missing values. Neither summary statistics nor visualisations of the predictor distributions (like boxplots and histograms) indicate that the missing data have characteristics that differ substantially from the non-missing data. To understand how the variables in the dataset relate to one another, correlation coefficients were calculated and scatter plots were examined to visualize the relationships between the variables, the outcome variable (*ViolentCrimesPerPop*) and the main predictor (*PctPopUnderPov*). It is notable that the variable *racepctblack* shows the strongest correlation with the outcome variable ($r = 0.63$), even surpassing *PctPopUnderPov*, the main predictor selected for this study. Given some strong correlations between predictor variables (for example between *PctNotHSGrad* and *PctLess9thGrade* ($r = 0.93$), between *perCapInc* and *PctBSorMore* ($r = 0.77$), and between *PctNotHSGrad* and *PctBSorMore* ($r = -0.75$)) and the linear relationships some variables have with *PctPopUnderPov*, it is recommended to assess multicollinearity during the model-building stage. The scatter plots show that most variables display a roughly linear relationship with *ViolentCrimesPerPop*, although that trend is often distorted in the extreme regions of the x-axis. *agePct12t29* has a very low correlation coefficient with *ViolentCrimesPerPop* (0.11). The variable *perCapInc* shows a higher correlation (-0.32), but there is clearly no linear trend present. Two extreme values for the *ViolentCrimesPerPop* variable appear. Chestercity has the highest number of violent crimes (4877) per 100K population. Spencercity, on the other hand, reports zero violent crimes. Given that Spencercity is a small community (11,066 inhabitants), it is difficult to assess the accuracy of the reported value. In the model diagnostics section, we will examine whether these two observations are outliers with respect to our linear model. It was also investigated whether or not small communities have a higher probability to have more extreme values of the response and predictor variables. Scatter plots show that communities with a very small population indeed show a very large spread for all variables.

Model Building

Univariate linear regression

The simple univariate regression equation we estimate with the training set is given as follows:

$$ViolentCrimesPerPop_i = 138.26 + 38.87 \cdot PctPopUnderPov_i + \epsilon_i$$

Here a coefficient for the slope of 38.87 could be found, which represents the expected increase in violent crimes per 100K population if poverty rate increases by one percentage point. The model has a R^2 of 0.2701, which is rather small and indicates that poverty by itself is insufficient to explain the variation in violent crime rates. For the simple linear model the assumptions of linearity are violated. Larger outcome values tend to be underestimated, the variance for larger outcome values is larger and the QQ-plot shows violation of the normality assumption. Regarding the population size, it can be noted that the residuals for smaller populations are rather negative, while the residuals for larger populations are rather positive. This means that the model tends to underestimate the total number of crimes for large populations, while the total number of crimes for small populations are rather overestimated.

Multivariate linear model

The first all-variable procedure resulted in a model using the following predictor variables: *racepctblack*, *PctPopUnderPov*, *PctLess9thGrade*, *PctNotHSGrad_i* and *PctImmig*. However for this model, the assumptions of normality and equal error variances of the error terms are again violated. Applying the log transformation offered a substantial improvement of our model showing approximate constant variances and residuals reasonably close to normal. Using the log transform Y variable to select a new model resulted in a similar model as before but with the *PctBSorMore* variable instead of the *PctNotHSGrad*. In this model, it could be observed from the partial regression plots that *PctRaceBlack* was not in the correct functional form. For this reason, a logit transformation was attempted which resulted in a more linear relation on the partial regression plots. Fitting the model with this logit transformed *racepctblack* seemed to also stabilize the residual vs fitted plot. This was followed by selecting a interaction term for the model, which showed that the best interaction given the BIC values would be between the main predictor *PctPopUnderPov* and *PctLess9thGrade*. However, this interaction introduced a high VIF value for both predictors and the interaction term possibly due to the presence of multiple education variables. Opting us to only take one education variable going forward. This resulted in a similar model with the *PctLess9thGrade* changed for *agePct12t29*. This model showed to have as best interaction term *PctPopUnderPov:agePct12t29*. However, This interaction resulted in a High VIF value for both variables and their interaction term. For that reason, the second best interaction term *PctPopUnderPov:PctBSorMore* was chosen in the final model. This resulted in a final model:

$$\begin{aligned} ViolentCrimesPerPop_i = & 7.358 + 0.026 \cdot PctPopUnderPov_i - 0.026 \cdot PctBSorMore_i - 0.026 \cdot agePct12t29_i \\ & + 0.030 \cdot PctImmig_i + 0.273 \cdot race_black_logit_i + 0.0008 \cdot PctPopUnderPov_i \cdot PctBSorMore_i + \epsilon_i \end{aligned}$$

Model diagnostics

After deciding on the model and the form of the variables, the effect of the outliers was determined. This was done based on the Deleted Studentized residuals, the Cook's distance, the leverage, DFFITS, and DFBETAS. These values were plotted, and values crossing a certain threshold were visualized and seen as influential. If certain datapoints were seen in multiple plots, then these were most likely more influential and possibly problematic outliers. Using these plots, many influential outliers could be discovered. In order to exclude any data points that were influential due to observation errors, the variable values of these points were extracted and manually checked to see whether these are observations errors, rare cases, or valid but extreme cases. By looking at all 318 communities in the dataset that were flagged as problematic, meaning they crossed the threshold of one of the diagnostic tests, no evidence could be found of errors or anomalies that would necessitate removal. Most of these communities were small and exhibited extreme demographic or crime-related characteristics. For example, two communities, Martinsvillecity and Vidorcity, both flagged by all 5 diagnostic tests, were reported to not have any people of black color in their community. As these are both small communities, we have no reason to suspect this observation to be wrongly annotated. Another example is Spencercity, which was reported to have a *ViolentCrimesPerPop* of 0, while this is not common in our dataset, we lack evidence to say that this observation is erroneous. Other observations showed either a high or low value in one or more variables compared to the majority of communities, reflecting genuine heterogeneity rather than data quality issues. To further assess and account for the impact of these points, we fit a robust regression using Bisquare weights. When using a robust model, the coefficients revealed nearly identical to those from the OLS, indicating that the influential points are not the result of gross errors. The small differences in coefficients show that these are already quite stable and thus that maybe the outliers and influential points did not distort the OLS model significantly.

Interpretation of the final model

Interpretation of *PctPopUnderPov* The effect of *PctPopUnderPov* varies with *PctBSorMore* through the interaction terms. This means that the effect of *PctPopUnderPov* is not simply given by its coefficient, but instead we can look at the estimated effect of *PctPopUnderPov* on the expected value of the logarithm of (violent crime rate per population of 100k + 1) as $0.02584 + 8 \times 10^{-4} \cdot PctBSorMore$. When the interacting variable is held at its average and all variables are held constant, the conditional effect of *PctPopUnderPov* on the expected value of the logarithm of (violent crime rate per population of 100k + 1) is: 0.04424. The 95%

confidence interval of this effect is [0.03776, 0.05161]. The interaction effect is reinforcing: with increasing percentages of people with higher education, the effect of *PctPopUnderPov* on the logarithm of (violent crime rate per population of 100k + 1) gets larger.

Interpretation of *PctBSorMore* The interpretation for *PctBSorMore* is similar to the one above because of their interaction. The conditional effect on of *PctBSorMore* on the logarithm of violent crime rate is given by: $-0.02596 + 8 \times 10^{-4} * \text{PctPopUnderPov}$. We can estimate the conditional effect of *PctBSorMore* on the expected value of the logarithm of (violent crime rate per population of 100k + 1) per unit increase of *PctBSorMore*, when keeping poverty at its mean and all other variables constant: $-0.02596 + 0.00922$ (95% CI: [-0.02068, -0.01239]). Contrary to above, we now see an interference effect: if a higher percentage of the population has a Bachelors degree or higher, the logarithm of the (violent crime rate per population of 100k + 1) is expected to decrease and this effect is less pronounced for higher levels of *PctPopUnderPov* (given the conditions above).

Interpretation of *race_black_logit* *race_black_logit* has no interaction terms so we can interpret the main effect on its own. A one-unit increase in the log-odds of percentage African American is associated with a 0.27348 increase in the expected value of the logarithm of (violent crime rate per population of 100k + 1) when keeping all other variables constant; where a one-unit increase in the log-odds has to be interpreted as a multiplication of the odds of being African American by e (≈ 2.72). The 95% confidence interval is [0.25066, 0.2963].

Interpretation of *PctImmig* The coefficient 0.03008 represents the expected increase in the logarithm of (violent crime rate per population of 100k + 1) for an increase of one percentage point in *PctImmig*, keeping all other variables constant (95% CI: 0.02565, 0.0345). An increase in percentage of immigrants is thus associated with an increase in the mean of log (violent crime rate per population of 100k + 1).

Interpretation of *agePct12t29* For the percentage of people between the age of 12 and 29, we find a negative association with the logarithm of (violent crime rate per population of 100k + 1). This is expressed by a coefficient of -0.0257 with 95% confidence interval [-0.03484, -0.01653]. An increasing percentage of *agePct12t29* is associated with a decrease in the mean logarithm of (violent crime rate per population of 100k + 1). This negative association is not in line with our prior assumption that violent crimes would increase for this age group.

Model validation

Refitting the model on the test data

When the final model is fitted to the test set or the full dataset, the estimated coefficients do not differ much from those obtained when fitting the model to the training set. The more data the model is fitted on, the more significant the p-values become.

Prediction of the test data

For the multivariate model 97.2431078 % of the observed values fall within the prediction interval. This means that the prediction intervals are a bit too strict. For the univariate model 91.7293233 % of the observed values fall within the prediction interval. This means that the prediction intervals are a bit too lenient. The variance of the residuals is proportional to the magnitude of the outcome variable. The univariate model incorrectly assumes homoscedasticity in the residuals, causing prediction intervals to be too wide for low outcome values and too narrow for high outcome values. As a result, the lower bound of the prediction interval is often negative for low values. In contrast, the multivariate model (which models the log-transformed outcome variable) produces prediction intervals whose width is proportional to the magnitude of the outcome variable, which aligns better with reality. Therefore, the prediction intervals from the multivariate model are more useful than those from the univariate model. When point estimates are compared to observed values, both the univariate and the multivariate model tends to underestimate large outcome values and tends to overestimate

small outcome values. This statement holds for both the prediction of the training data and the prediction of the test data. When the mean squared prediction error (for test data) is compared with the mean squared error (for training data), it can be noticed that the predictive power of the model is better for the test data than for the training data. This indicates that there is no overfitting. The same trend is visible when the values for R^2 are compared. The R^2 of the multivariate model is higher than the R^2 of the univariate model. However, these values are difficult to compare, because the outcome variable differs between the two models (*ViolentCrimesPerPop* vs. $\log(\text{ViolentCrimesPerPop})$) and back-transforming the predicted values of $\log(\text{ViolentCrimesPerPop})$ does not yield the expected value for *ViolentCrimesPerPop* (except under strict assumptions regarding the residuals).

Statistical discussion

The missing data of *ViolentCrimesPerPop* may interfere with generalizability as some communities are excluded because of the missing rape data. To account for this missingness in further research, crime definitions should be harmonized across states when data is collected, so it is verified that differences in crime rates do not stem from differences in reporting practices. In this research, we see that the characteristics from those non-missing data communities are not much different than for the missing ones. We thus assume that this missingness does not have an important effect on our found results. Results should still be interpreted with caution, as these communities might differ in ways not captured by our measured variables. Moreover, the LEMAS survey includes all communities with police departments of more than 100 officers, but only a random sample of communities with smaller departments. As a result, the representation of small communities in the dataset is less precise. Ideally, all these small communities would be included as well in the dataset. Smaller communities have greater variance in both predictor and outcome variables than larger communities. This can partly be explained by the smaller denominator: in a small community, small changes in the numerator may produce substantial changes in the rate compared to small changes in big communities. Another caveat is that the FBI's data is from 1995, while the Census' populations data containing demographic variables are from 1990. If the demographic variables of certain communities changed drastically during this period, the predictor variables would not represent these communities well in 1995 (when the actual crimes happened). Since 1990-1995, the social and economic environment have changed. Thus, results should be carefully interpreted in its temporal context as the mechanisms driving these relationships today are different in comparison to when the data collection happened. To check if these relationships still hold nowadays, more recent data should be used. Then, it is mentioned in the description of the original dataset that many relevant factors are not included. Specifically, it is mentioned that per capita crime rates are calculated using resident populations, so communities with large numbers of visitors are expected to have higher crime rates, even if the actual risk is not different across communities. Further research would ideally account for this by having data on the amount of visitors per state and correcting for this. This 'problem', together with the fact that we didn't include relevant covariates such as income inequality, poverty, ... implies that our results should be interpreted as correlations and not as causations. At last, as the data is measured at the community level and not at the individual level, we cannot determine any individual-level effects of crime and our relationships only hold at the community level. For this purpose, we recommend future data collection to gather this individual-level data in addition to community data. This way, heterogenous effects can also be more accurately determined.

Conclusion

In this report, we investigate how violent crime rates and socio-economic disadvantage, and more specifically the poverty rate, are associated in U.S. communities. For this purpose we used a merged dataset with detailed data on both crimes and demographic variables from 1990 and 1995. The univariate linear regression showed a significant positive relationship between the percentage poverty and the number of violent crimes per 100k population. For the multivariate model, that modelled the logarithm of *ViolentCrimesPerPop*, a subset of four additional (socio-economic) predictor variables was selected: *PctPopUnderPov*, *race_black_logit* and *PctImmig* showed a significant positive association and *PctBSorMore* and *agePct12t29* showed a significant negative association with the logarithm of *ViolentCrimesPerPop*. The selected interaction term, *PctPopUnderPov:PctBSorMore* is significant and positive suggesting that a higher percentage of inhabitants

with a bachelor degree or more implies a stronger association between poverty and the number of crimes per 100k population. This possibly reflects a greater inequality within such communities, and this inequality could result in a higher crime rate (Kelly (2000), Fajnzylber et al. (2002) and Kang (2016)). To see if this is the case, a variable measuring inequality should ideally be included in further research. The multivariate model predicts the logarithm of the number of crimes per 100k population with an R^2 of 0.5328 on the training dataset and an R^2 of 0.5841 on the test dataset. The modelling of the logarithm is very useful to give good prediction intervals, because in this way the heteroskedasticity of *ViolentCrimesPerPop* is considered, but makes the interpretation of the model less convenient and makes it difficult to deliver good point estimates for *ViolentCrimesPerPop*. Several limitations should be highlighted though. The dataset contains missing data, is not very recent and uses outdated demographic predictors. Keeping this context in mind, our results show that the number of violent crimes is associated with socio-economic disadvantage. It's important to mention that all results are interpreted as correlational and not as causal. The theoretical framework, which is essential for establishing causal relationships, is not further developed in this statistical study. Moreover, the number of predictors included in the analysis is very limited, which means confounding could play a significant role. Given our goal to inform policies to lower violent crime, our model would not be that useful. The main reason is that the relationships identified by the model are not causal. Our model does suggest that poverty is associated with higher violent crime rates, this could be seen as useful. However, many of the variables emerging from the model-building process are demographic in nature, such as community-level race distributions or age profiles. These could not be resolved in an ethical way so can not help in trying to lower the violent crime rate. One variable we would have liked to add was the population size as we can see that the residuals are clearly different for these larger populations so adding it would likely improve the model.

References

- Becker GS (1968) Crime and Punishment: An Economic Approach. *Journal of Political Economy* 76: 169–217
- Fajnzylber, P., Lederman, D., & Loayza, N. (2002). Inequality and violent crime. *The journal of Law and Economics*, 45(1), 1-39.
- Kang, S. (2016). Inequality and crime revisited: effects of local inequality and economic segregation on crime. *Journal of Population Economics*, 29(2), 593-626.
- Kelly, M. (2000). Inequality and crime. *Review of economics and Statistics*, 82(4), 530-539.

References dataset

- U. S. Department of Commerce, Bureau of the Census, Census Of Population And Housing 1990 United States: Summary Tape File 1a & 3a (Computer Files),
- U.S. Department Of Commerce, Bureau Of The Census Producer, Washington, DC and Inter-university Consortium for Political and Social Research Ann Arbor, Michigan. (1992)
- U.S. Department of Justice, Bureau of Justice Statistics, Law Enforcement Management And Administrative Statistics (Computer File) U.S. Department Of Commerce, Bureau Of The Census Producer, Washington, DC and Inter-university Consortium for Political and Social Research Ann Arbor, Michigan. (1992)
- U.S. Department of Justice, Federal Bureau of Investigation, Crime in the United States (Computer File) (1995)

Additional references

- Redmond, M. A. and A. Baveja: A Data-Driven Software Tool for Enabling Cooperative Information Sharing Among Police Departments. *European Journal of Operational Research* 141 (2002) 660-678.
- Rousseeuw, P. J., & van Zomeren, B. C. (1990). Unmasking Multivariate Outliers and Leverage Points. *Journal of the American Statistical Association*, 85(411), 633–639. <https://doi.org/10.1080/01621459.1990.10474920>

Source - <https://stackoverflow.com/a>

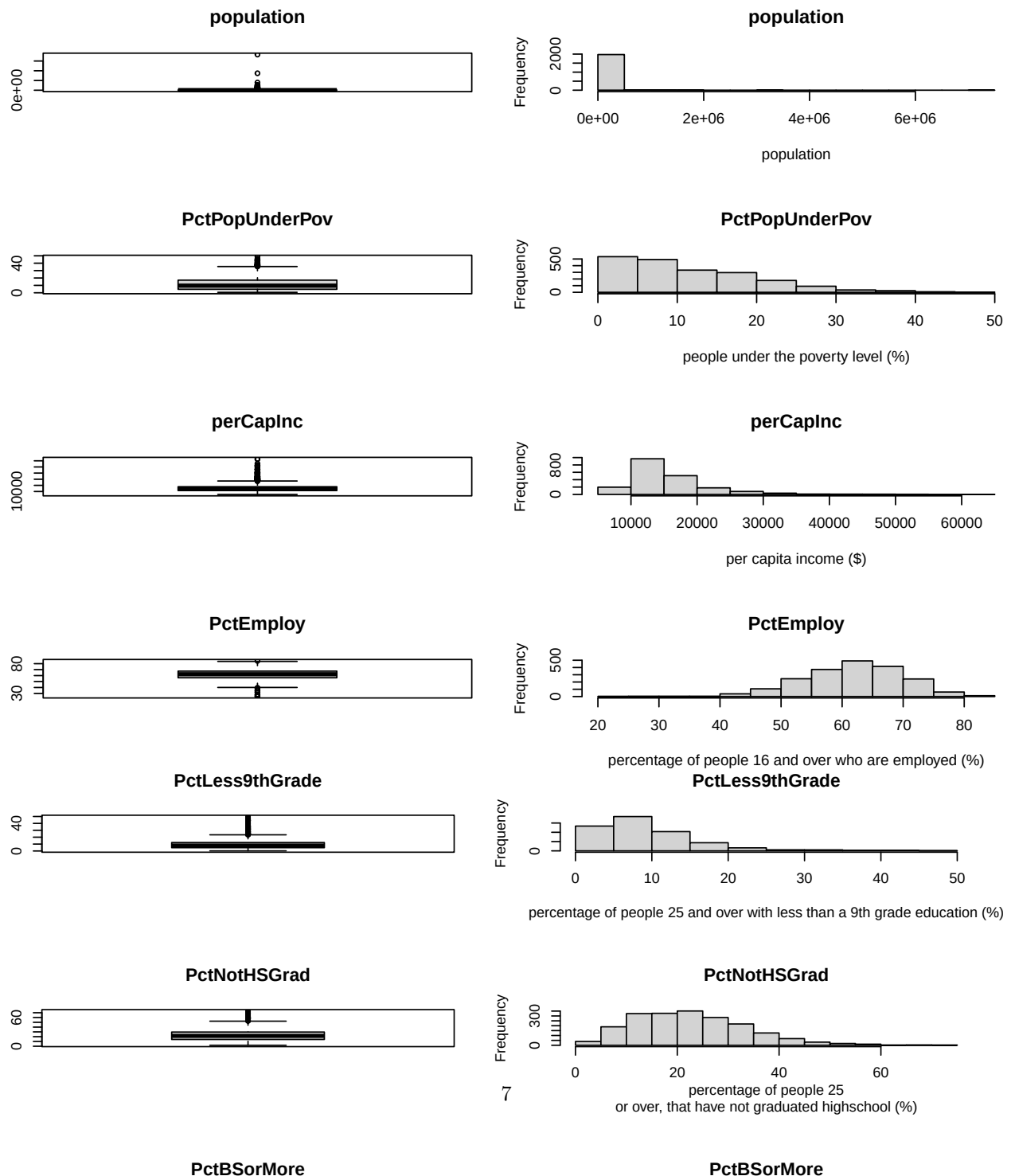
Posted by DonJ, modified by community. See post 'Timeline' for change history

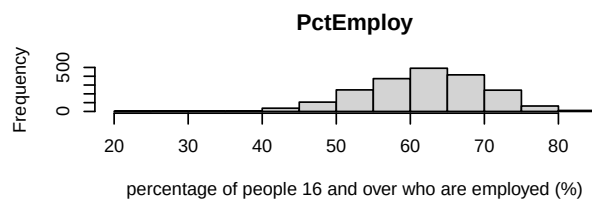
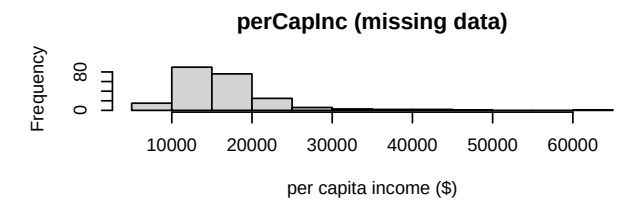
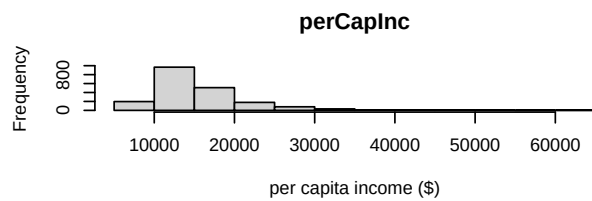
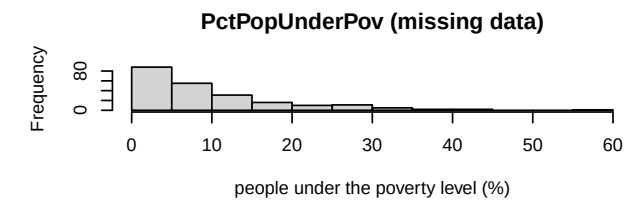
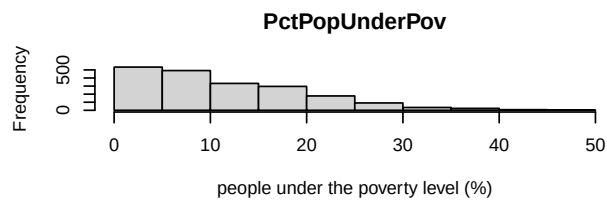
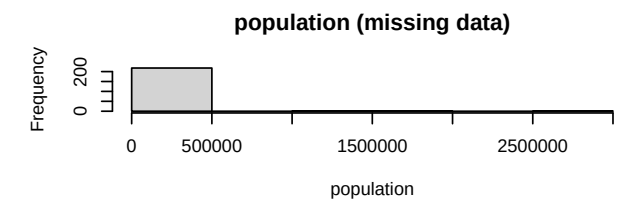
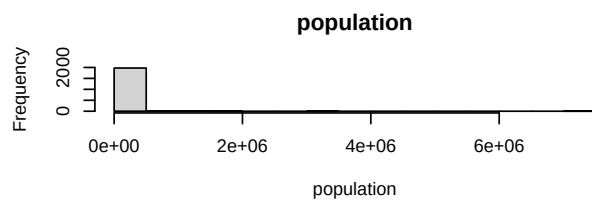
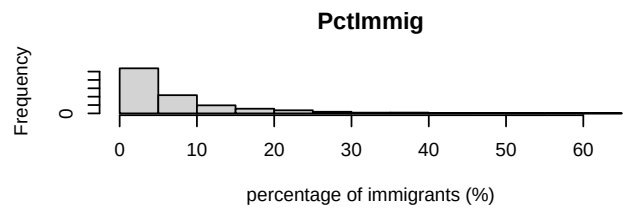
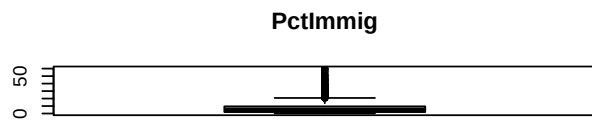
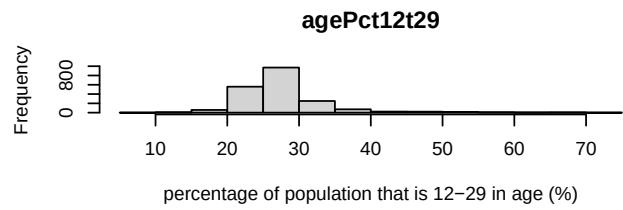
Retrieved 2025-12-03, License - CC BY-SA 3.0

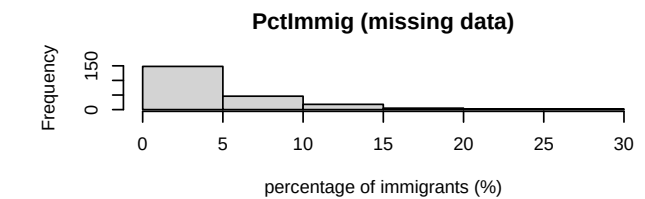
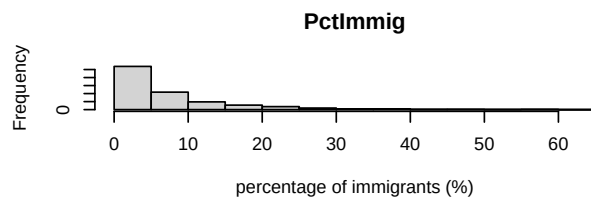
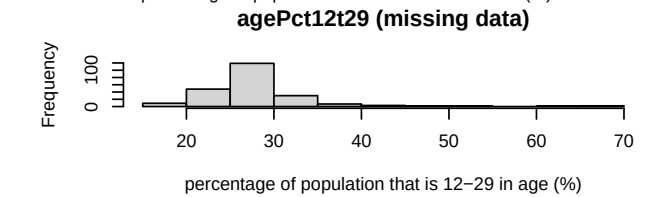
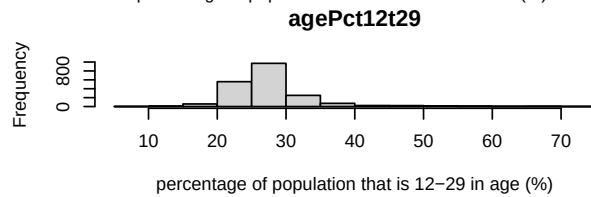
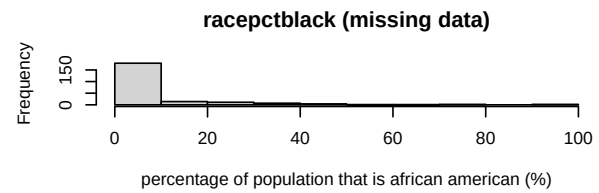
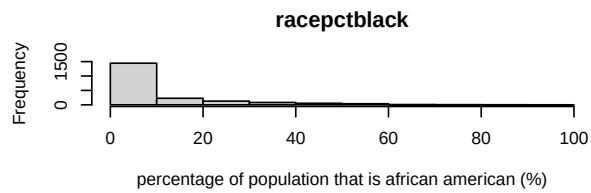
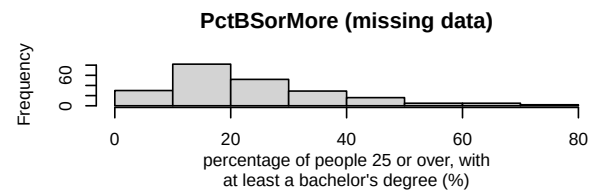
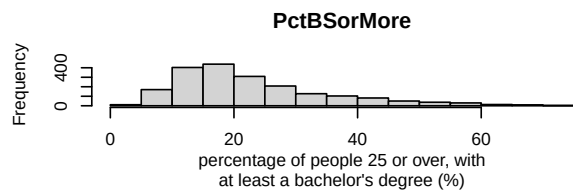
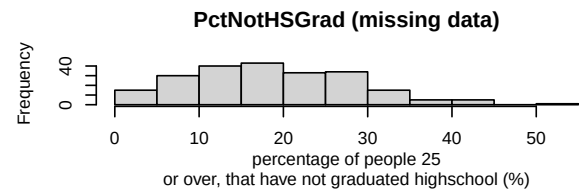
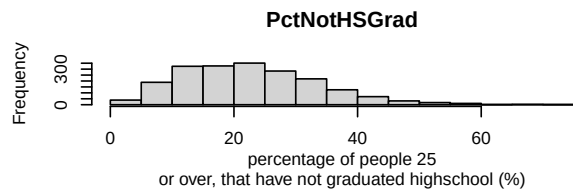
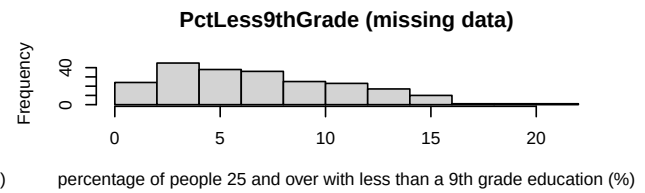
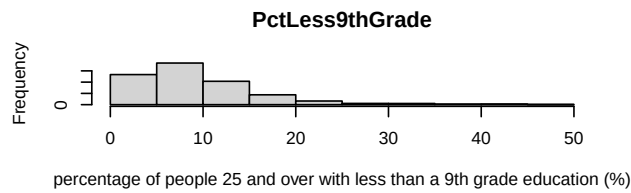
Appendix

Figures

Descriptives

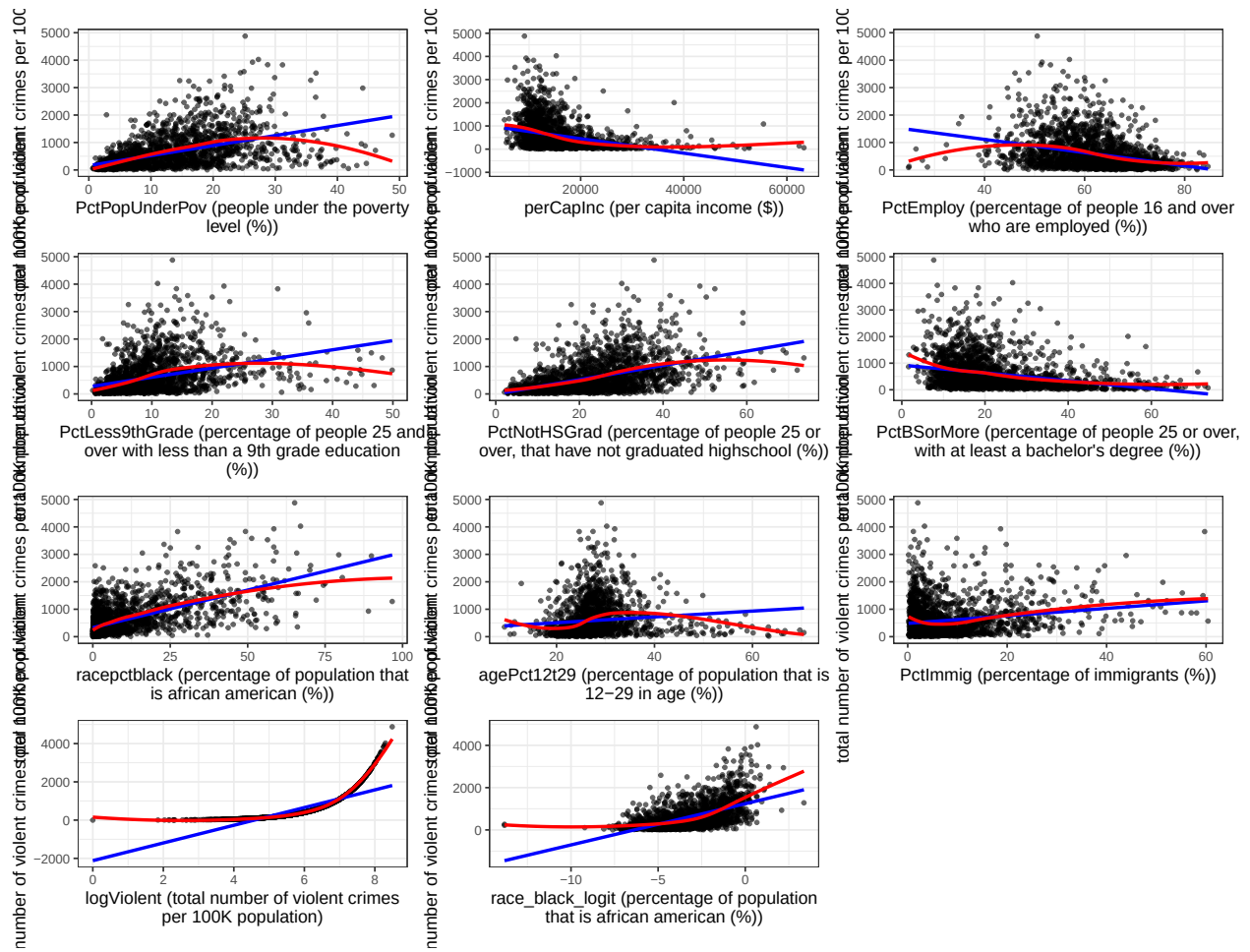


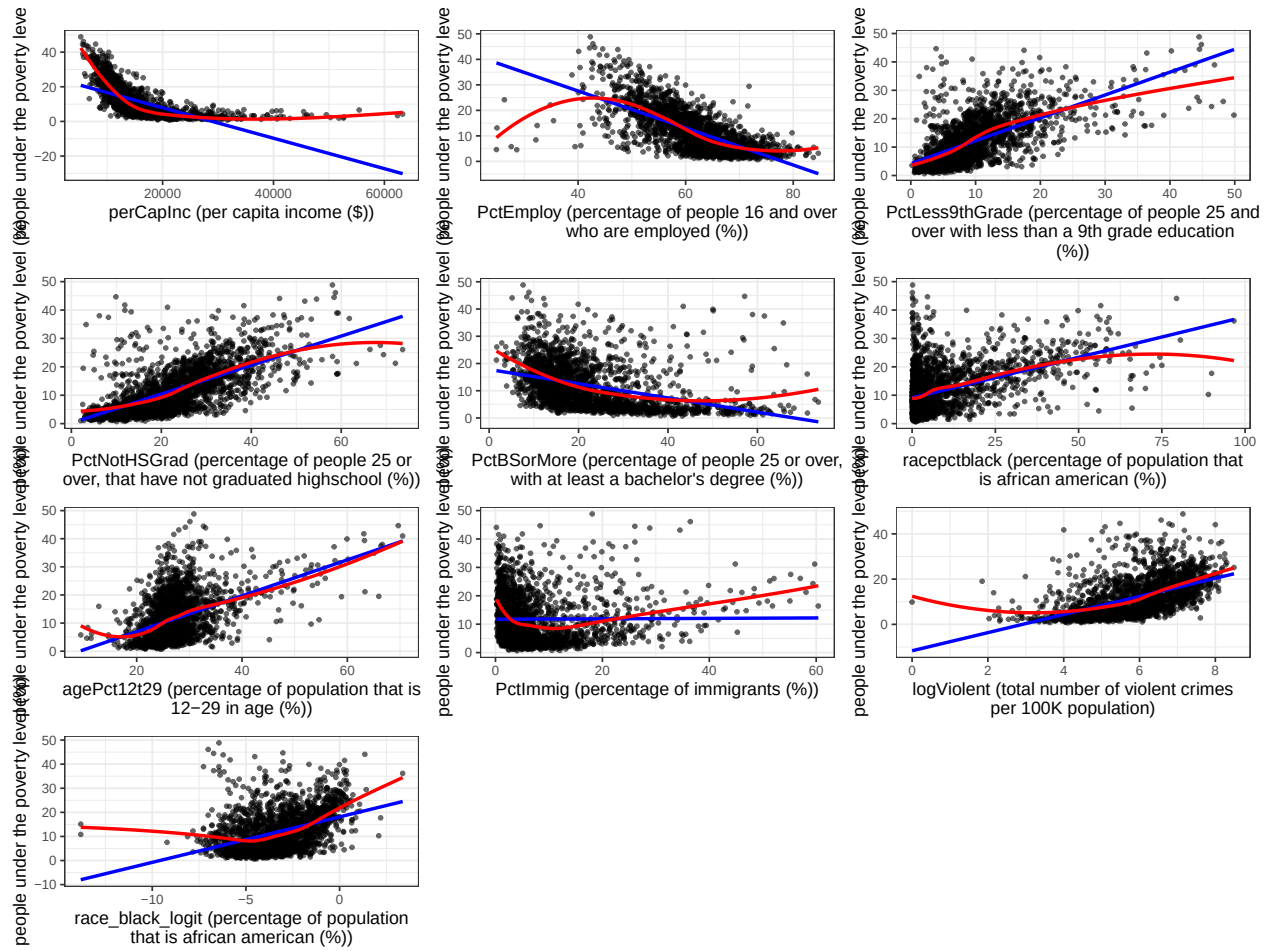




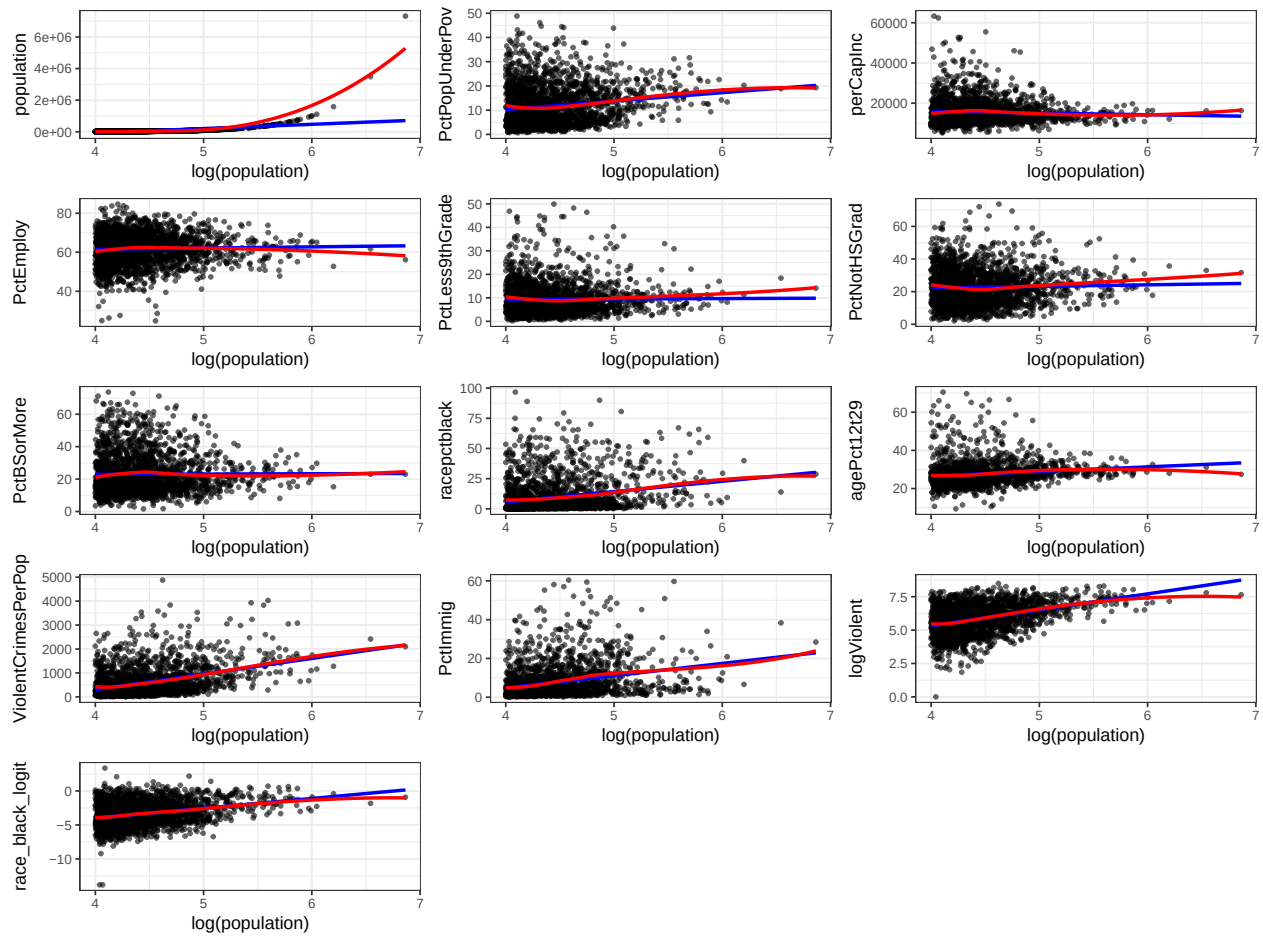
correlation plot:

scatter plots:



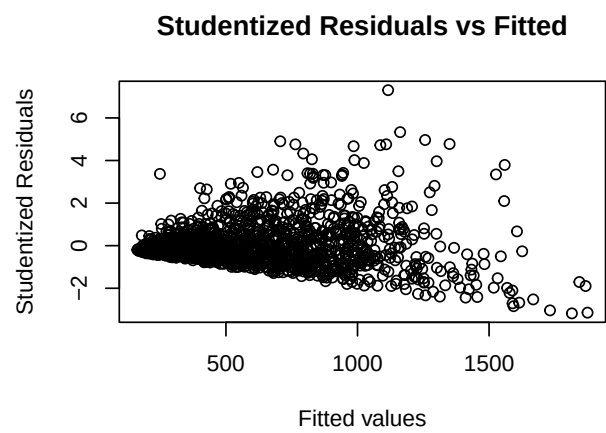
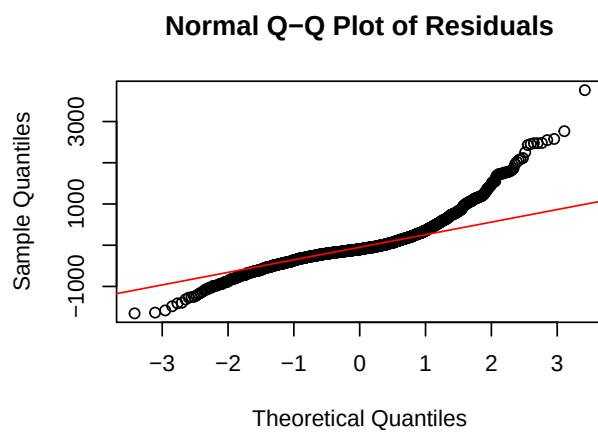
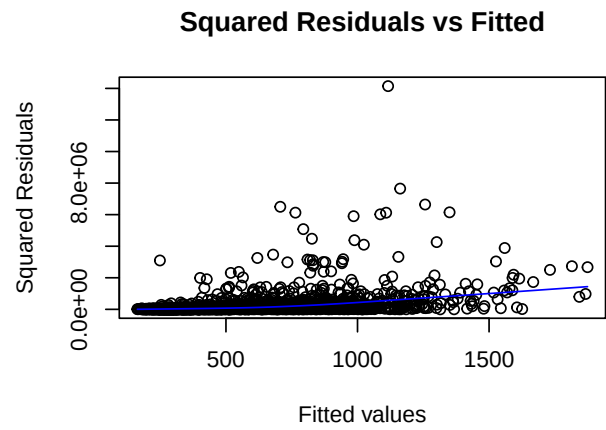
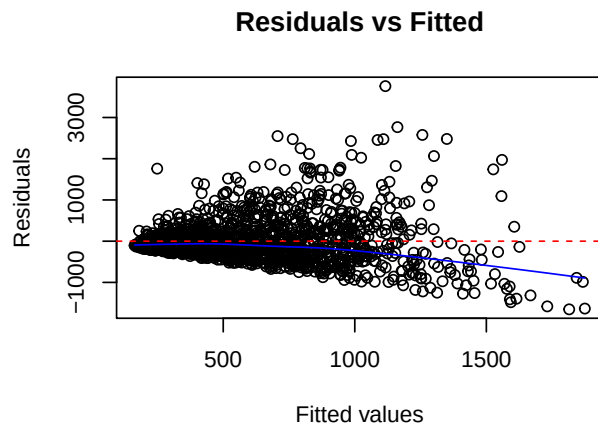


population scatter plots:

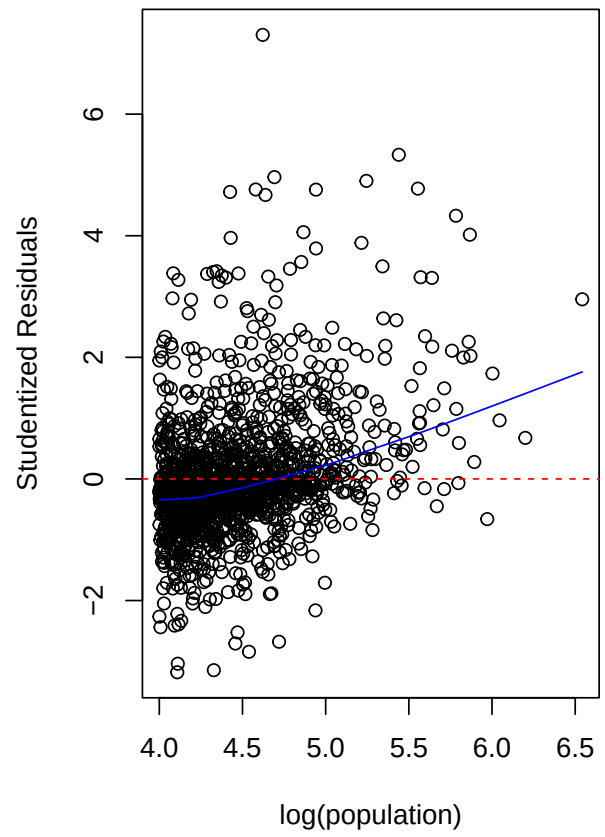
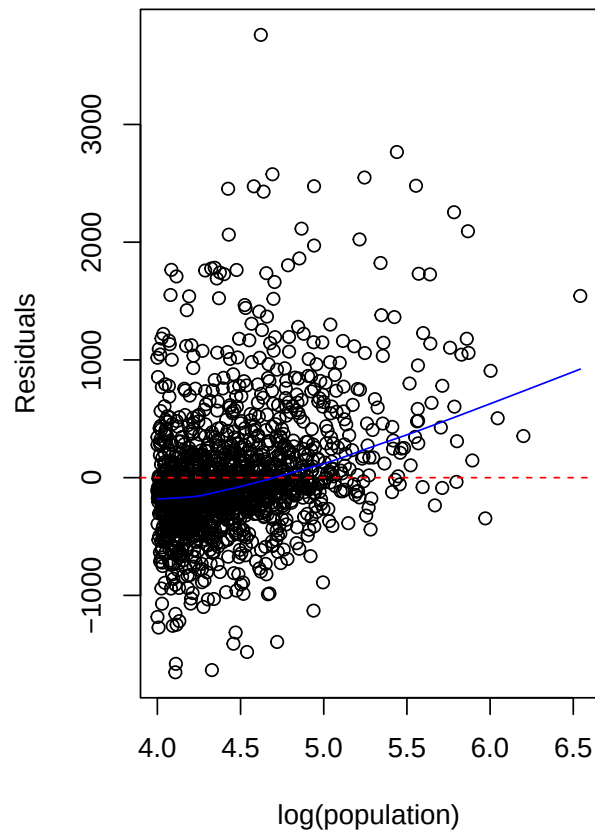


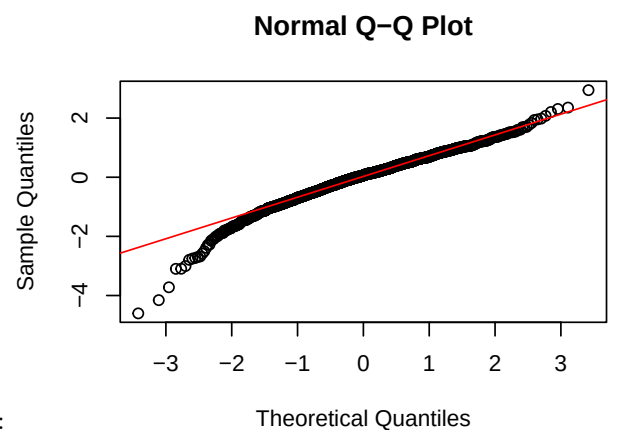
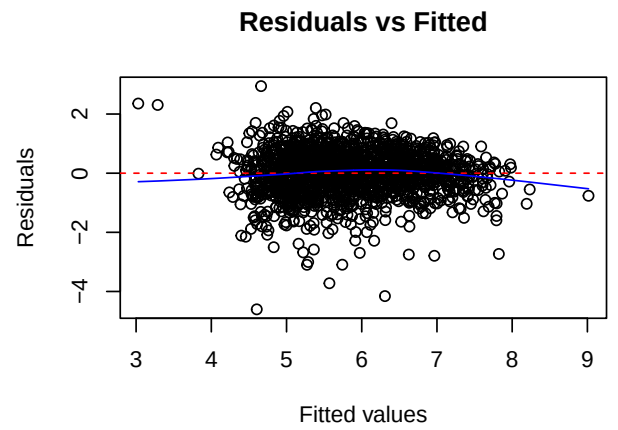
Model building

Univariate model Simple model assumption plots:

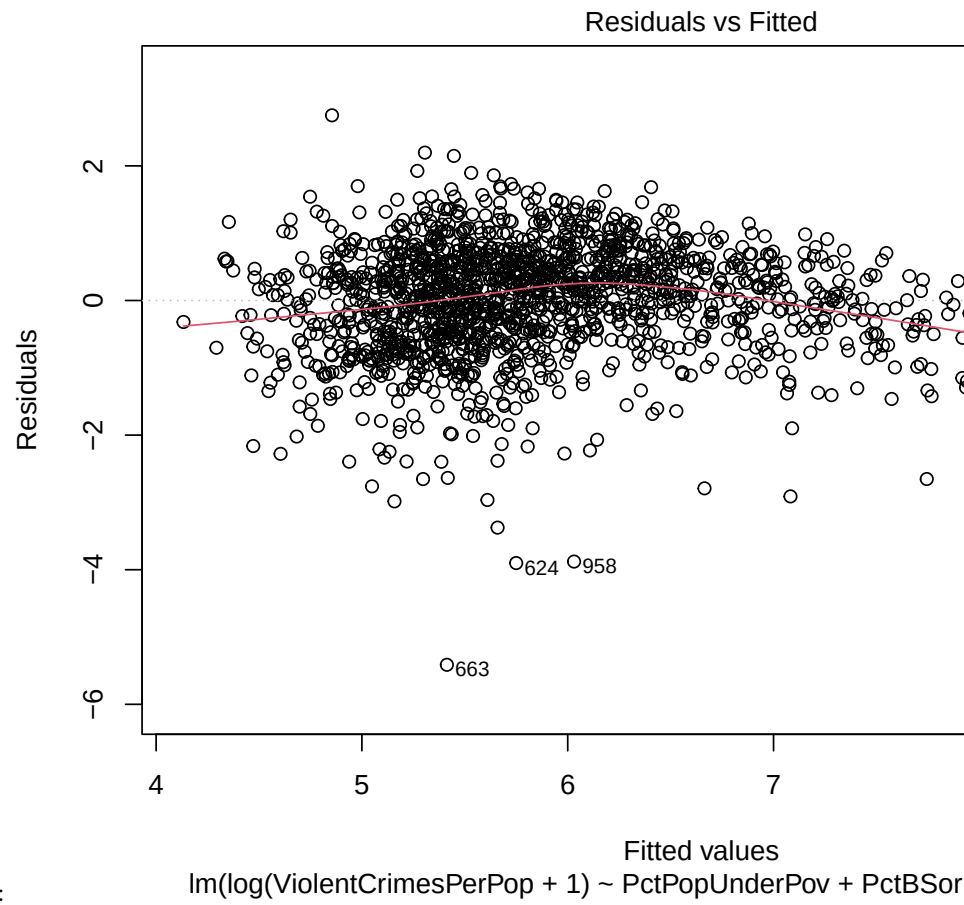


Residuals based on population size:

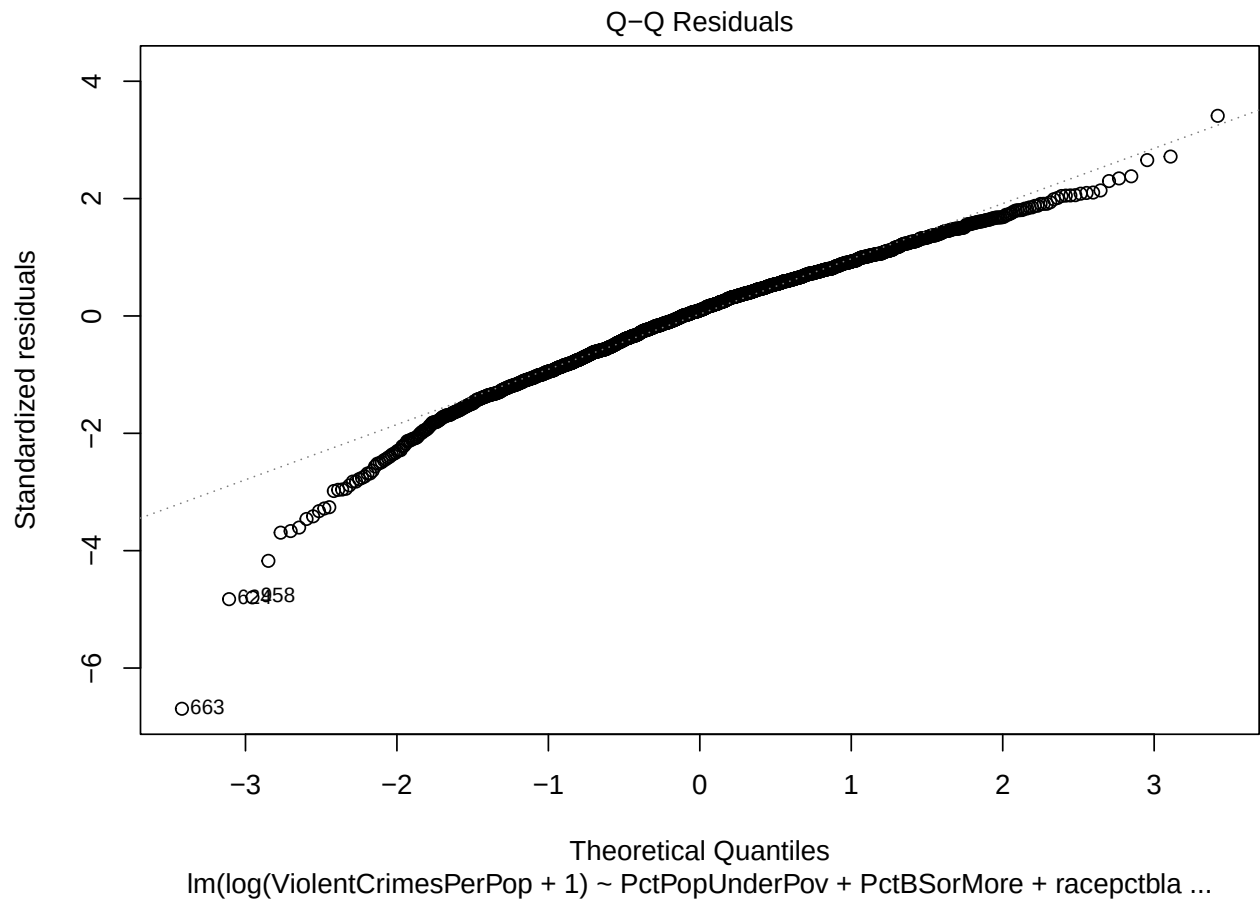


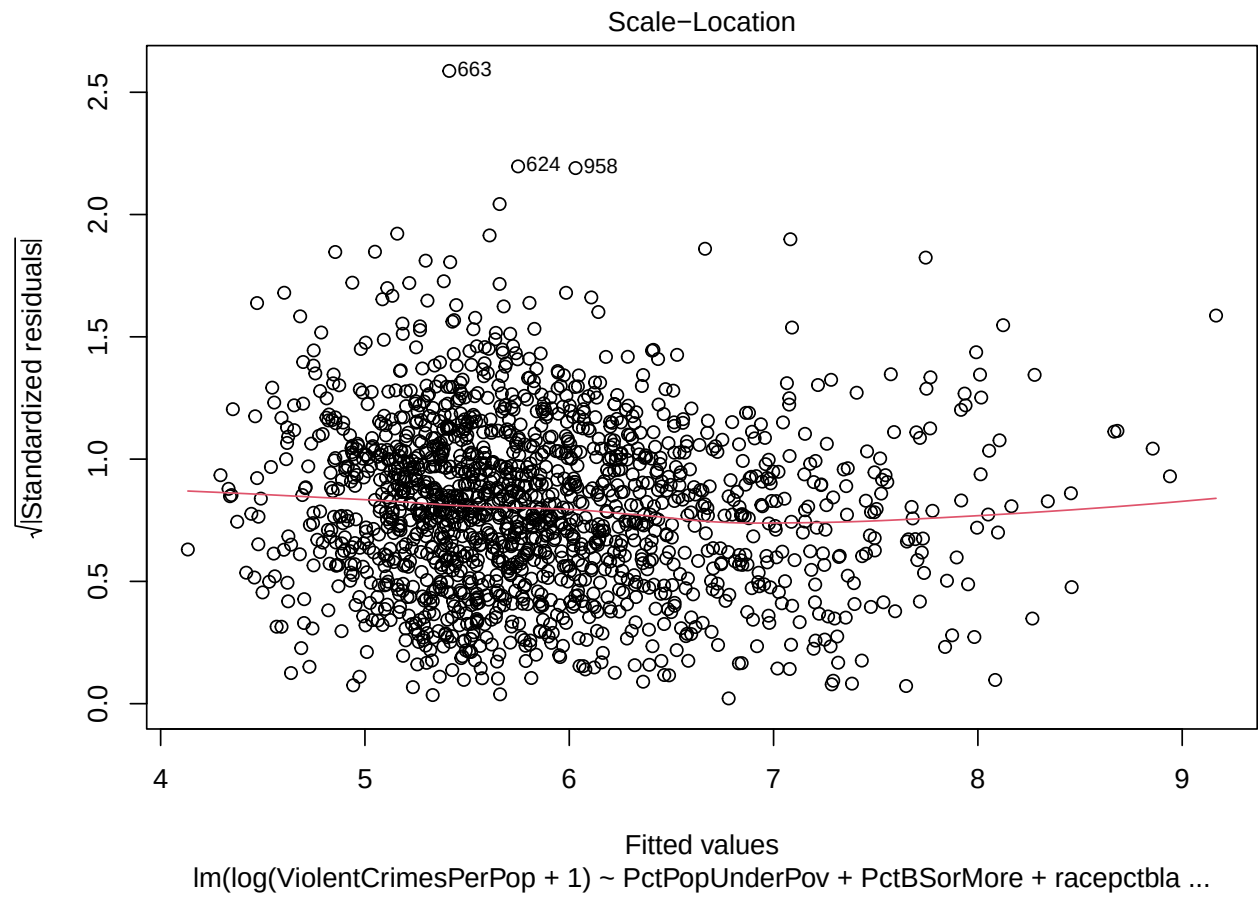


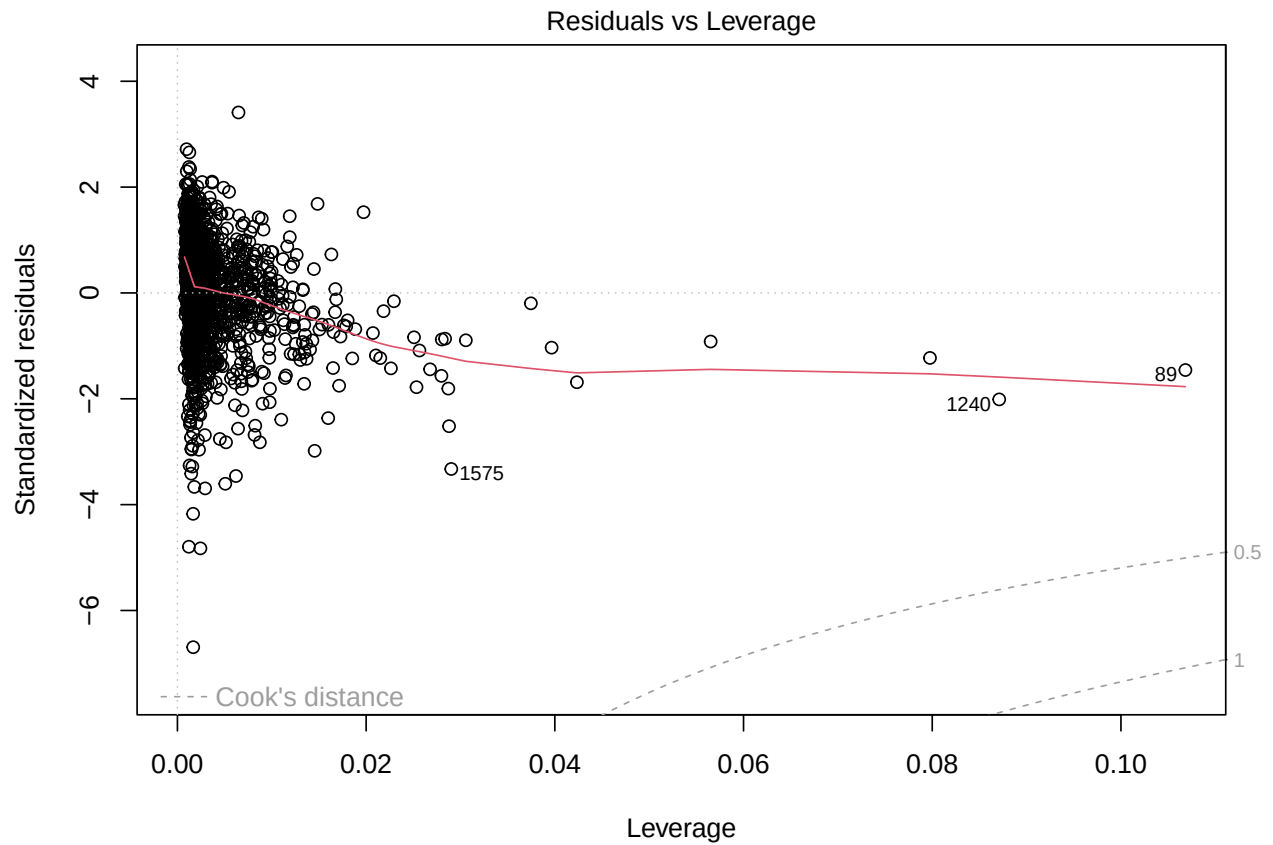
multivariate model Assumption checks first multivariate model:



Assumption checks log transformation test:

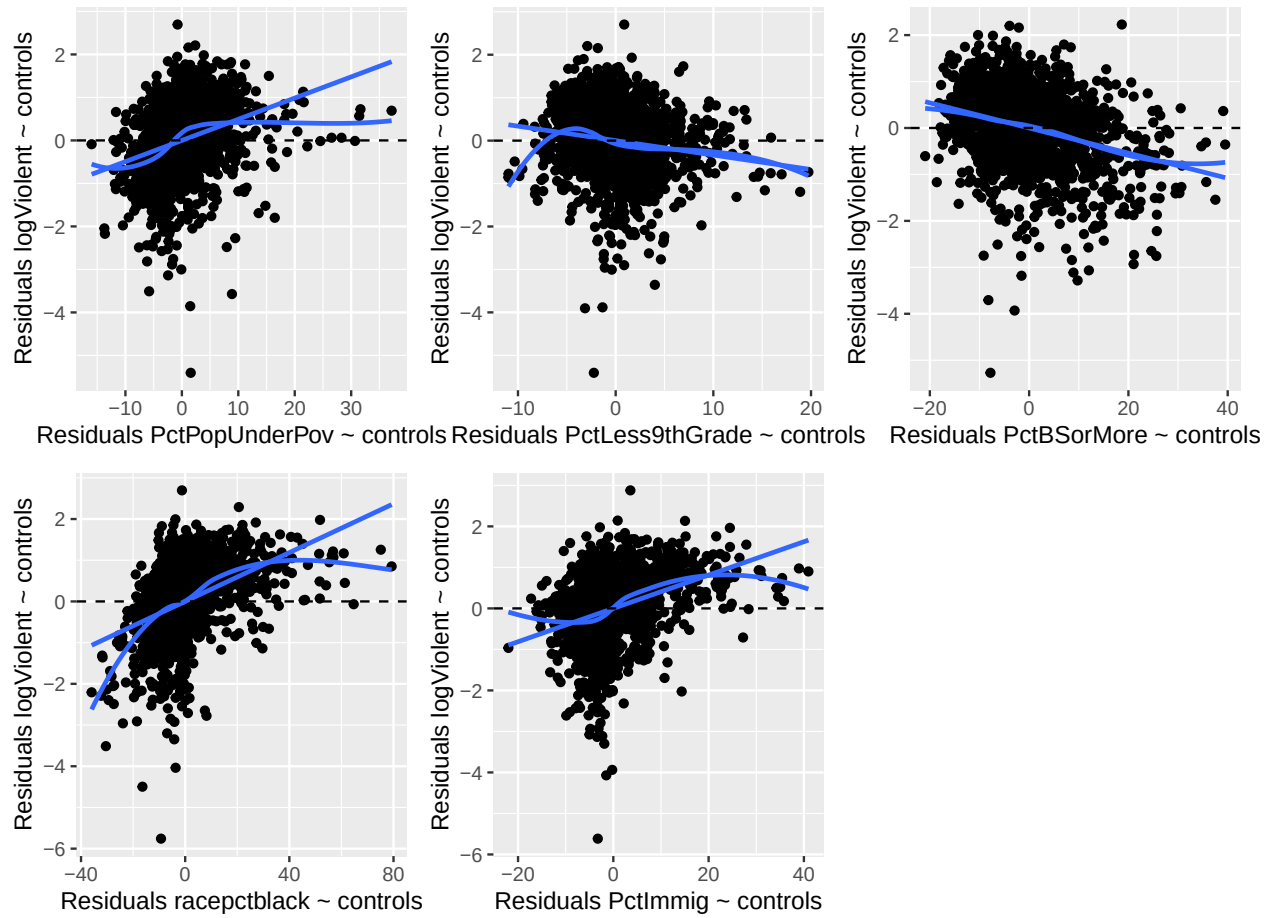




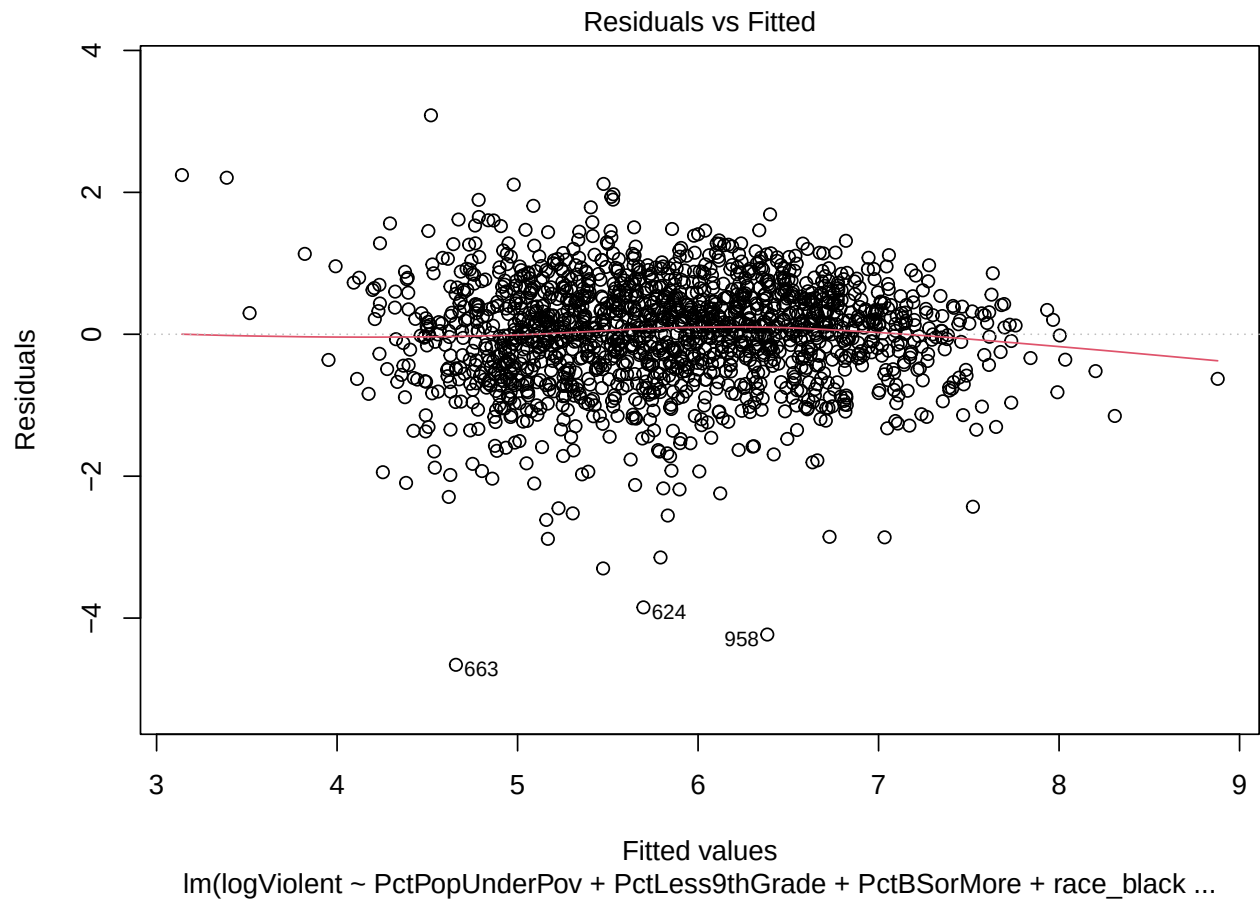


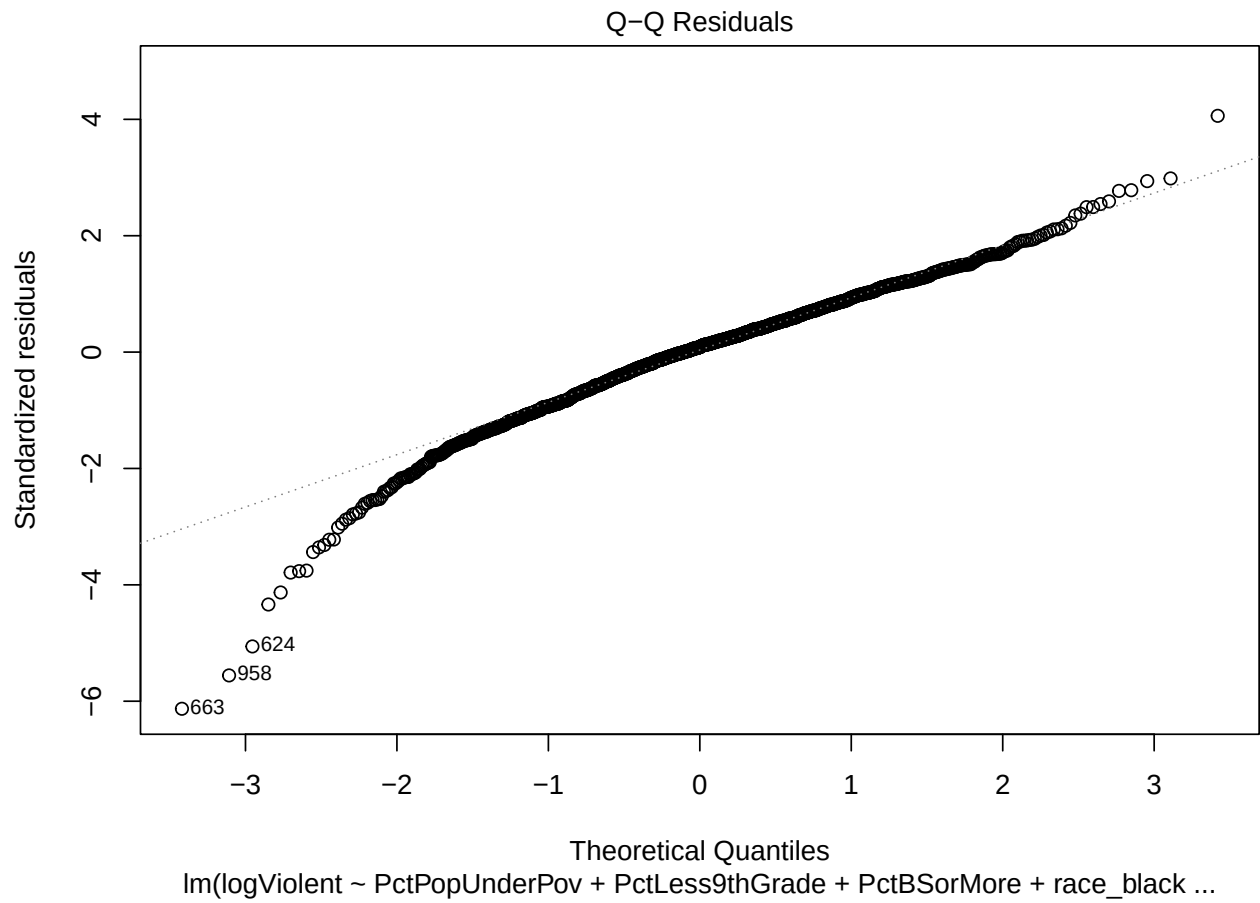
$\text{lm}(\log(\text{ViolentCrimesPerPop} + 1) \sim \text{PctPopUnderPov} + \text{PctBSorMore} + \text{racepctbla} \dots)$

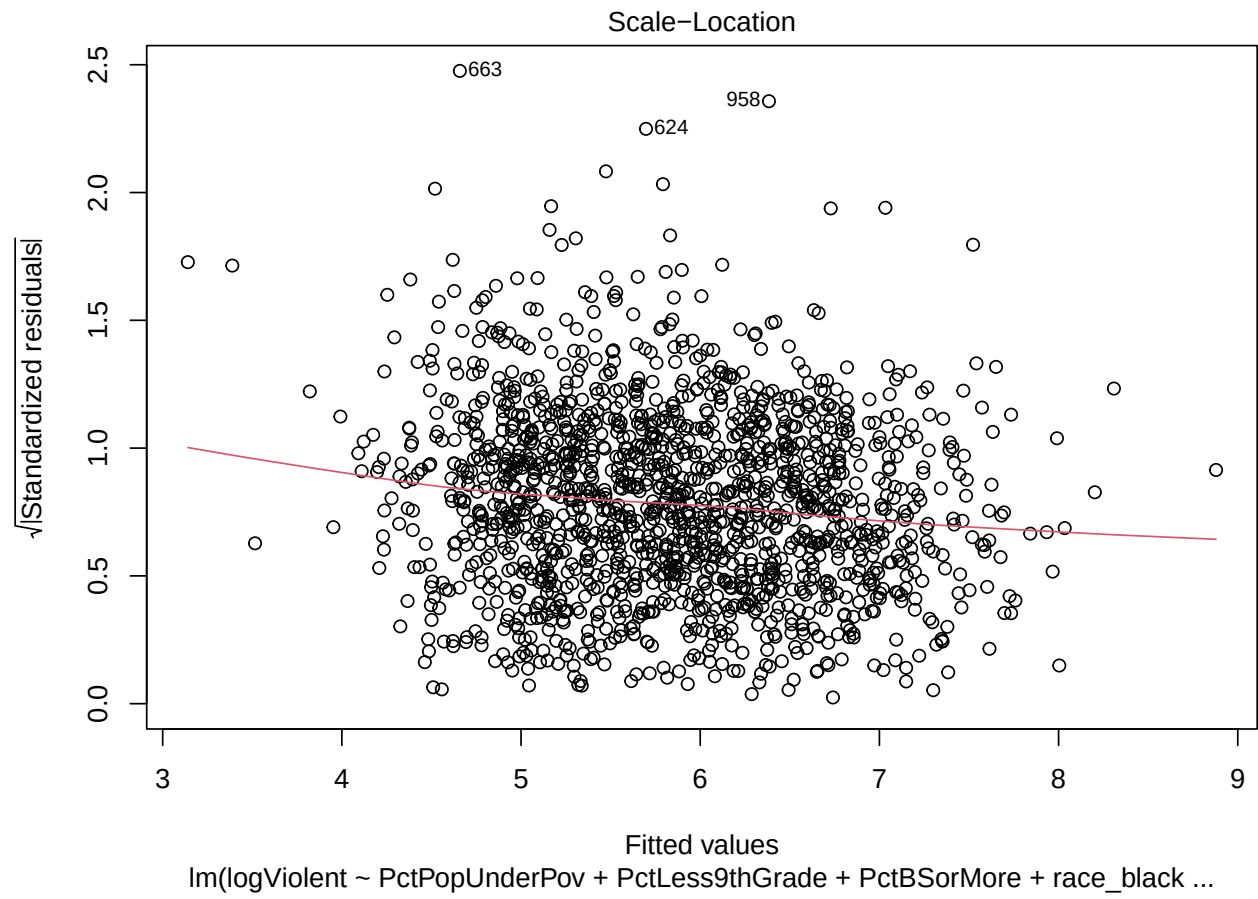
Partial regression plots first log model:

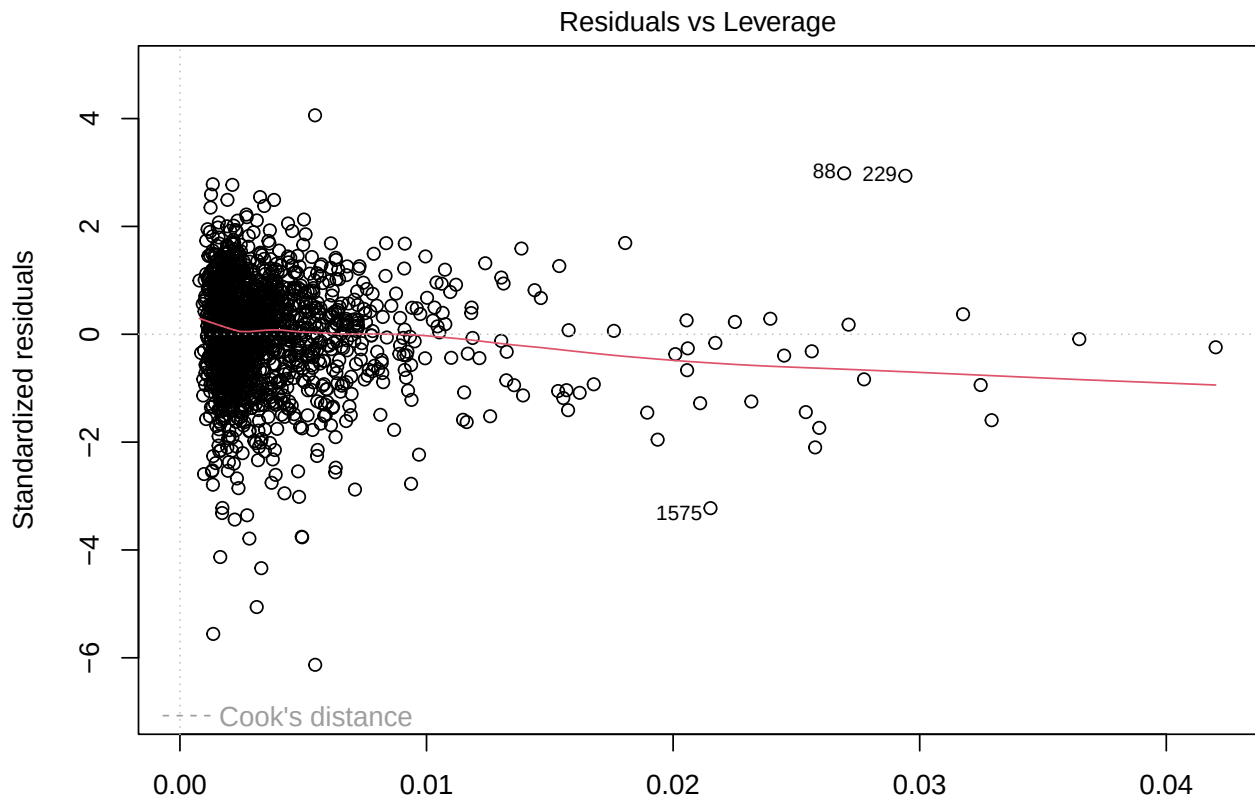


Logit transformed *racepctblack* model assumptions check:









lm(logViolent ~ PctPopUnderPov + PctLess9thGrade + PctBSorMore + race_black ...)

Partial regression logits:

VIF check model with *less9thgrade*

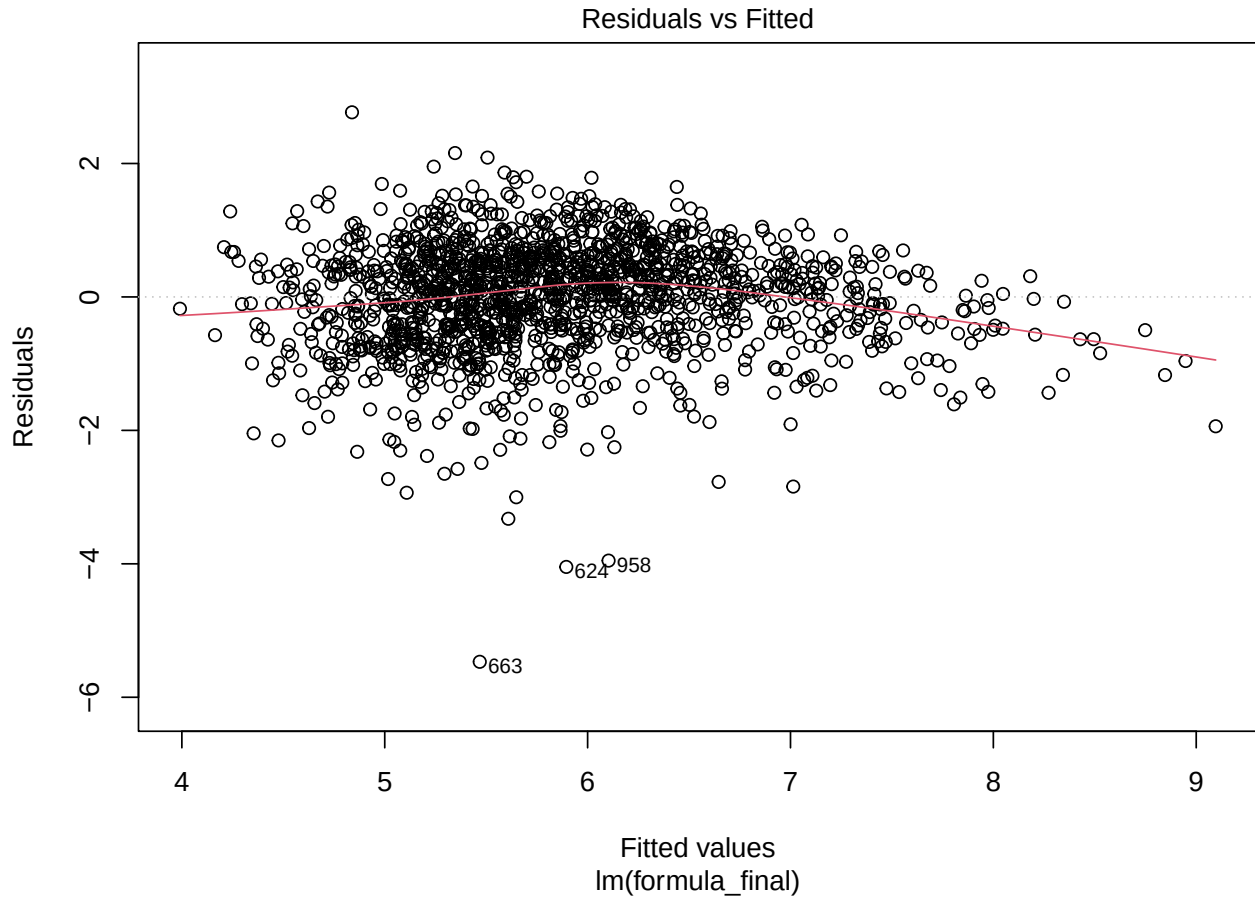
```
##          PctPopUnderPov          PctLess9thGrade
##          3.867985          7.970943
##          PctBSorMore          race_black_logit
##          1.982546          1.271535
##          PctImmig PctPopUnderPov:PctLess9thGrade
##          1.353844          9.503437
```

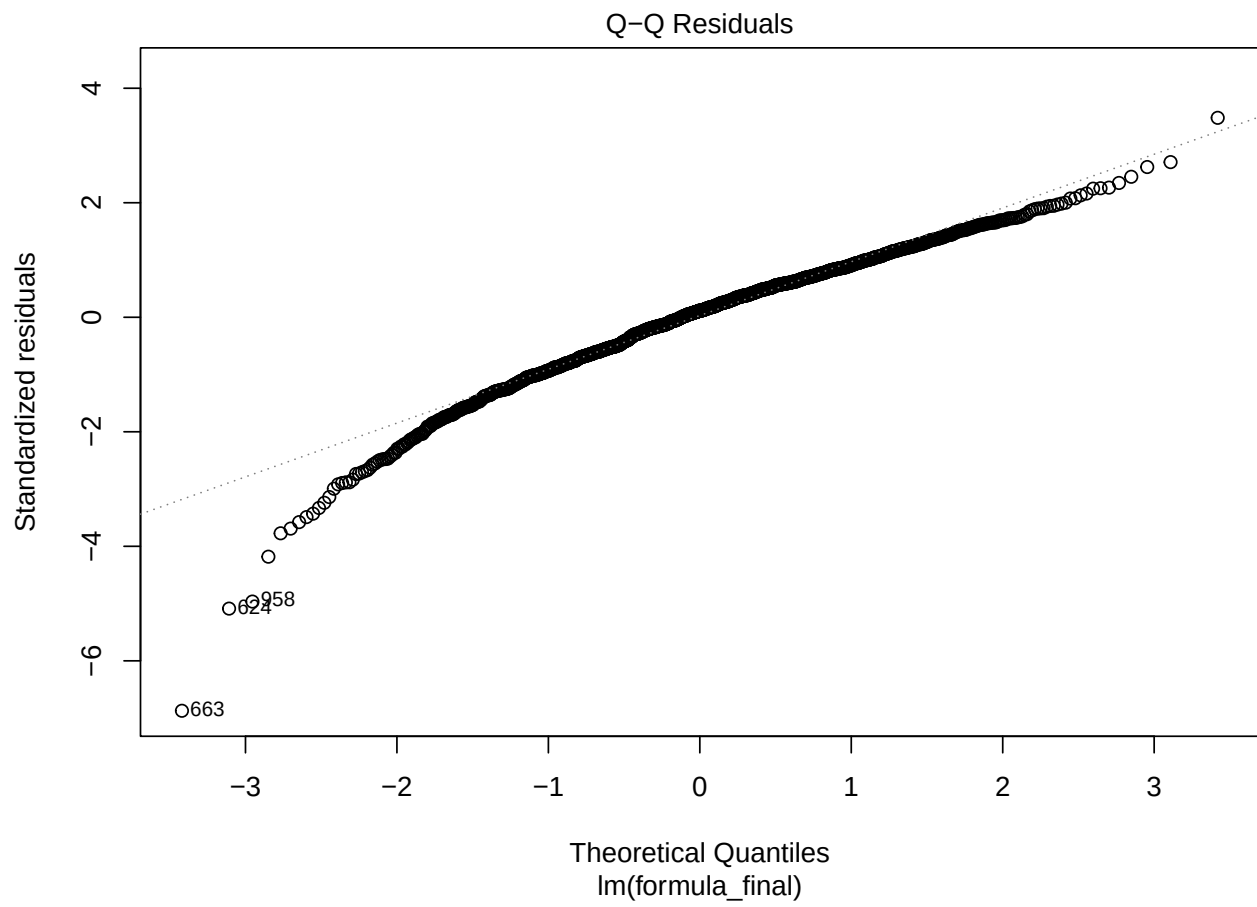
BIC: 3708.523

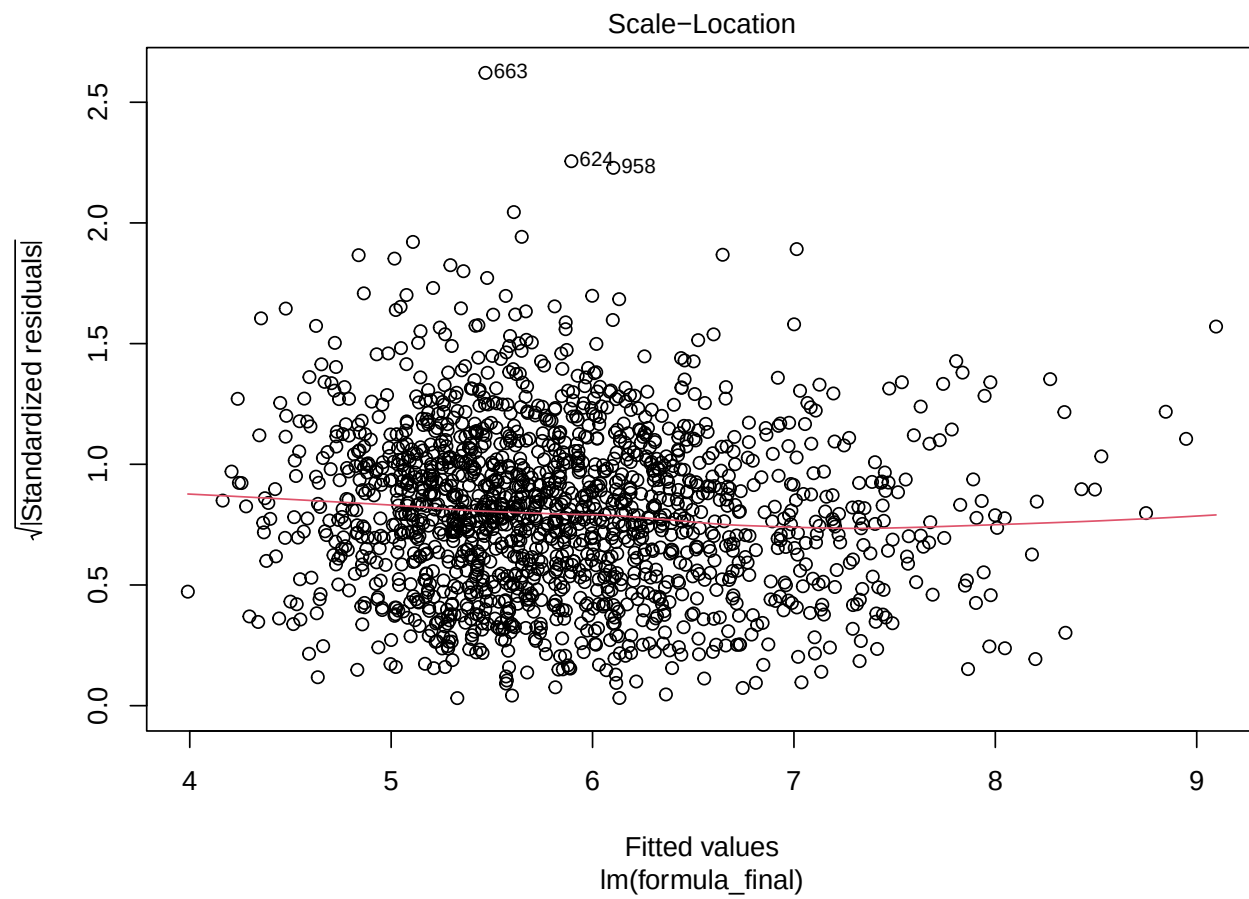
Assumptions check one education model:

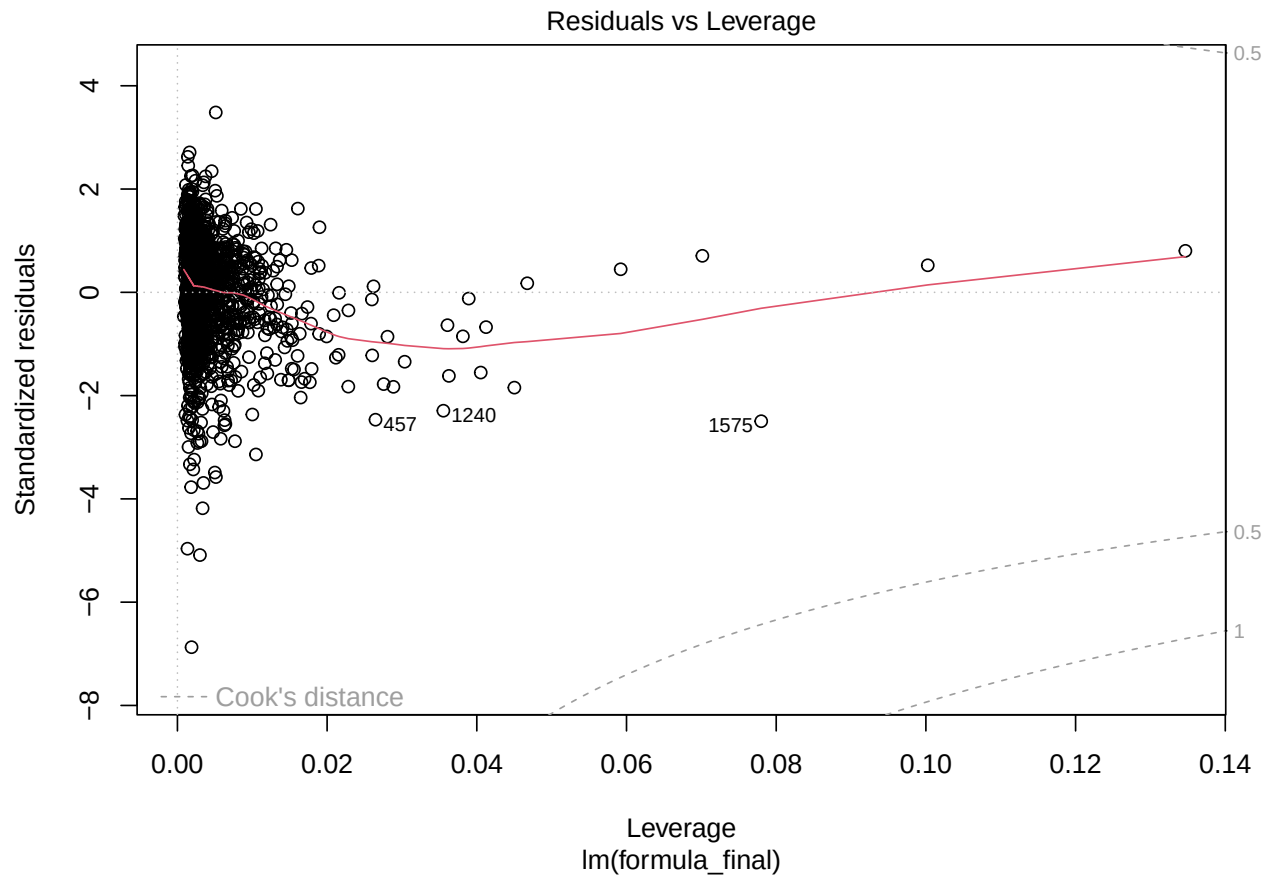
```
##
## Call:
## lm(formula = formula_final, data = train_data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -5.4683 -0.4785  0.0913  0.5278  2.7672
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    5.3665907   0.1036739   51.764 < 2e-16 ***
## PctPopUnderPov    0.0604980   0.0047273   12.798 < 2e-16 ***
## PctLess9thGrade  -0.0082566   0.0086285   -0.957 0.338765
```

```
## PctBSorMore          -0.0241650  0.0022540 -10.721  < 2e-16 ***
## racepctblack          0.0292809  0.0016312  17.950  < 2e-16 ***
## PctImmig              0.0401939  0.0026846  14.972  < 2e-16 ***
## PctPopUnderPov:PctLess9thGrade -0.0011863  0.0003203  -3.704  0.000219 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.7965 on 1588 degrees of freedom
## Multiple R-squared:  0.4899, Adjusted R-squared:  0.4879
## F-statistic: 254.1 on 6 and 1588 DF,  p-value: < 2.2e-16
```







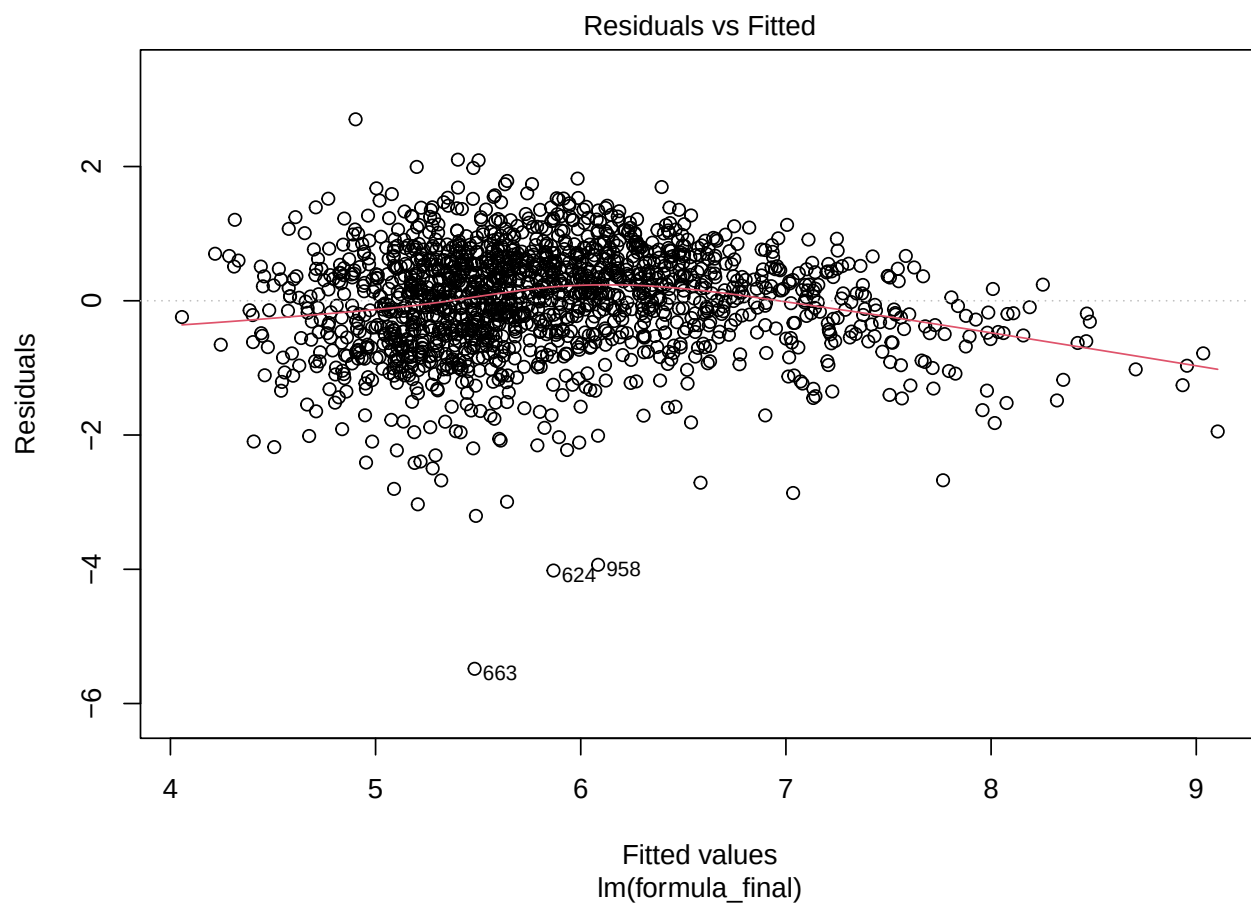


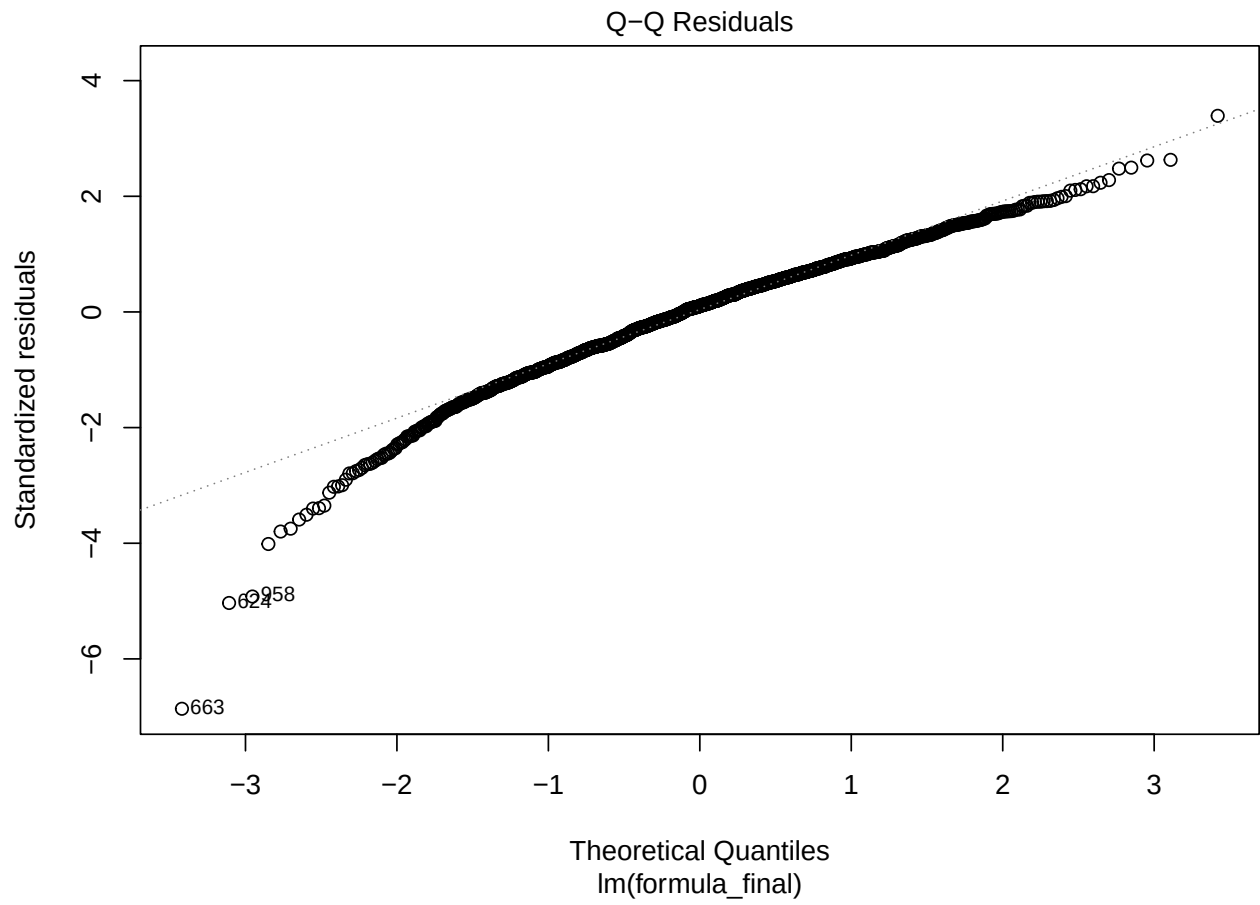
Vifs first one education model:

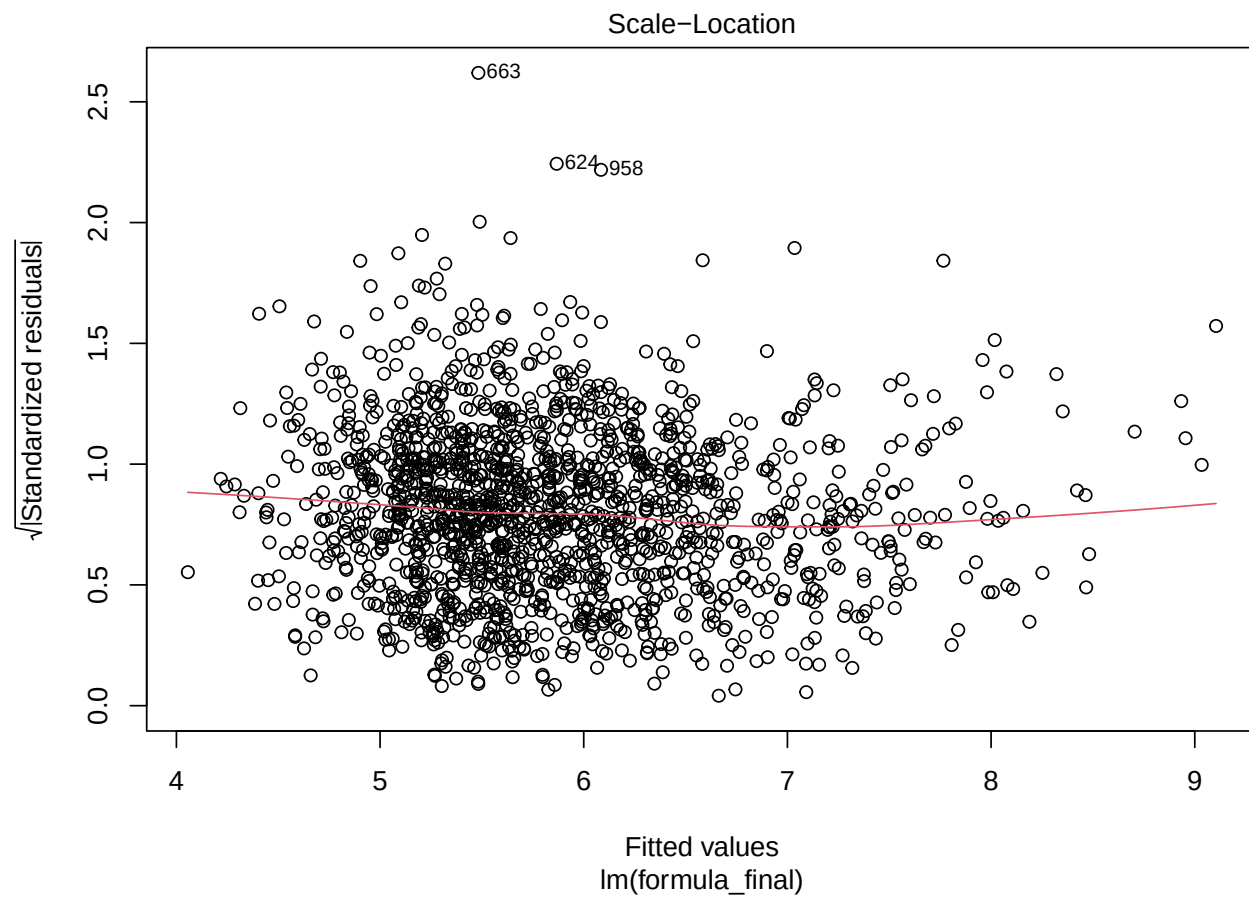
```
##          PctPopUnderPov          PctLess9thGrade
##          3.772261          7.956073
##          PctBSorMore          racepctblack
##          1.979648          1.335870
##          PctImmig PctPopUnderPov:PctLess9thGrade
##          1.355459          9.369846

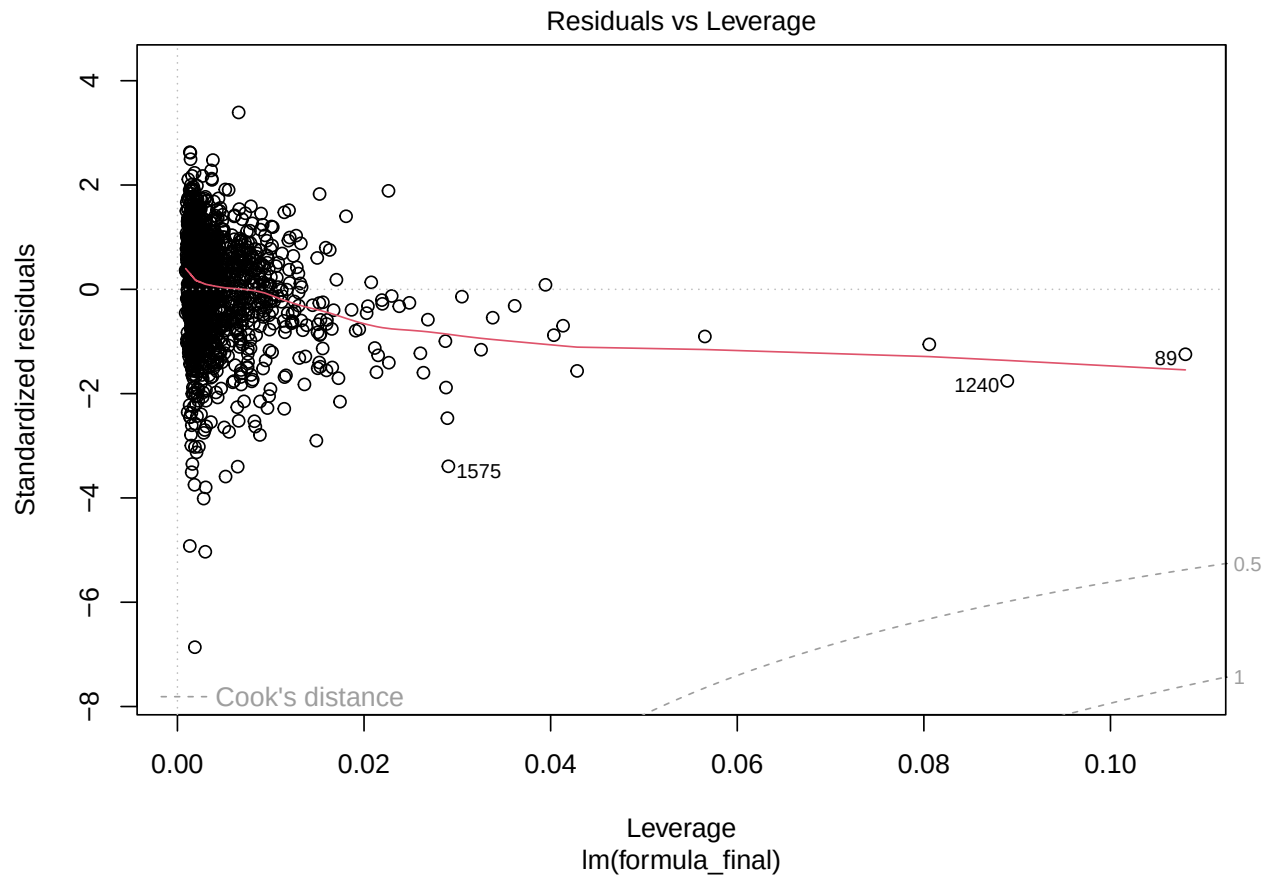
## BIC: 3852.422
```

Assumptions one education model with *PctPopPov:PctBSorMore* interaction:







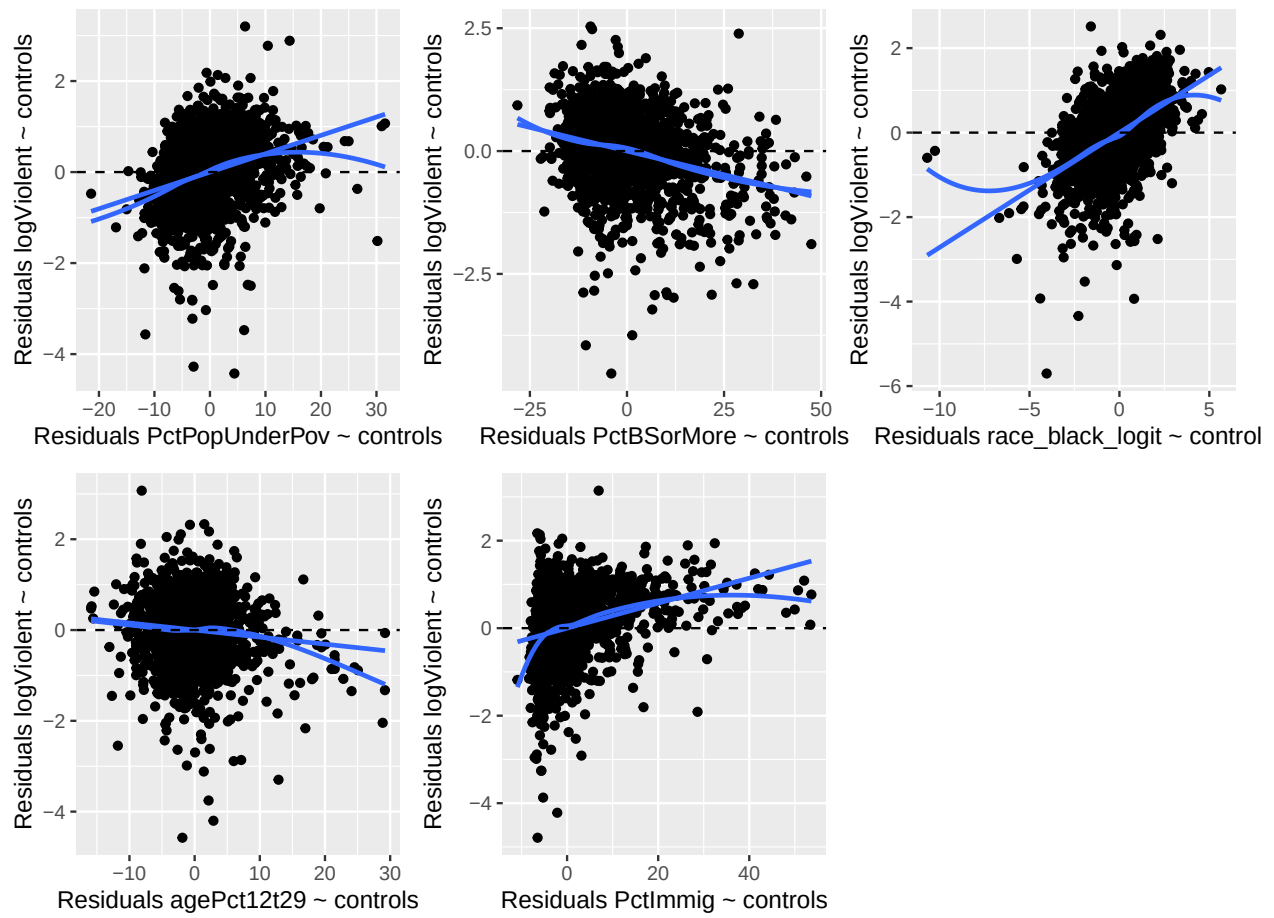


Vifs for *PctPopPov:PctBSorMore* interaction model:

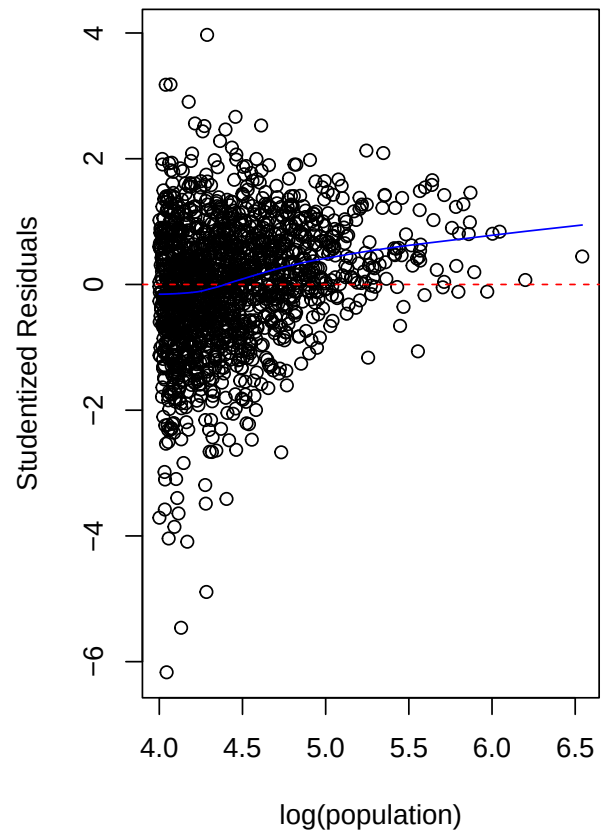
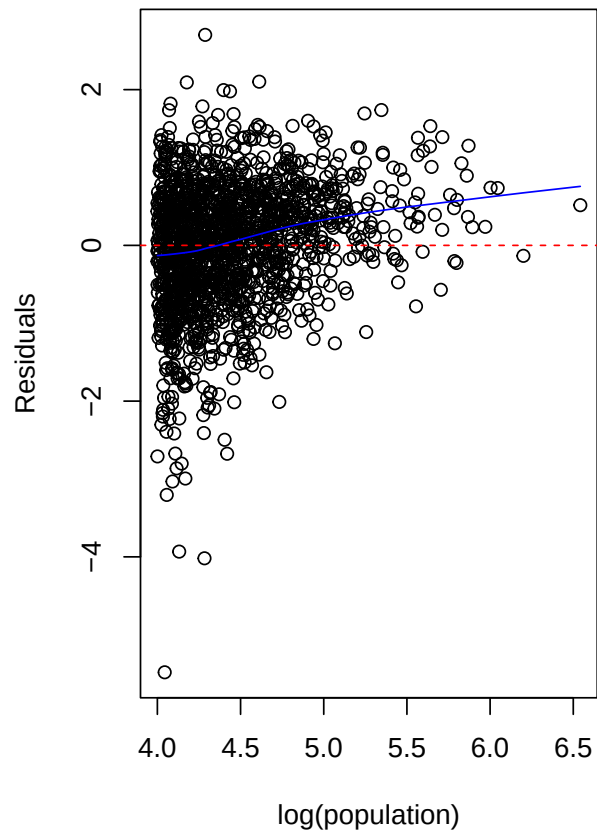
##	PctPopUnderPov	PctLess9thGrade
##	7.349363	3.413646
##	PctBSorMore	racepctblack
##	2.569881	1.358620
##	PctImmig	PctPopUnderPov:PctBSorMore
##	1.363749	4.305053

BIC: 3865.286

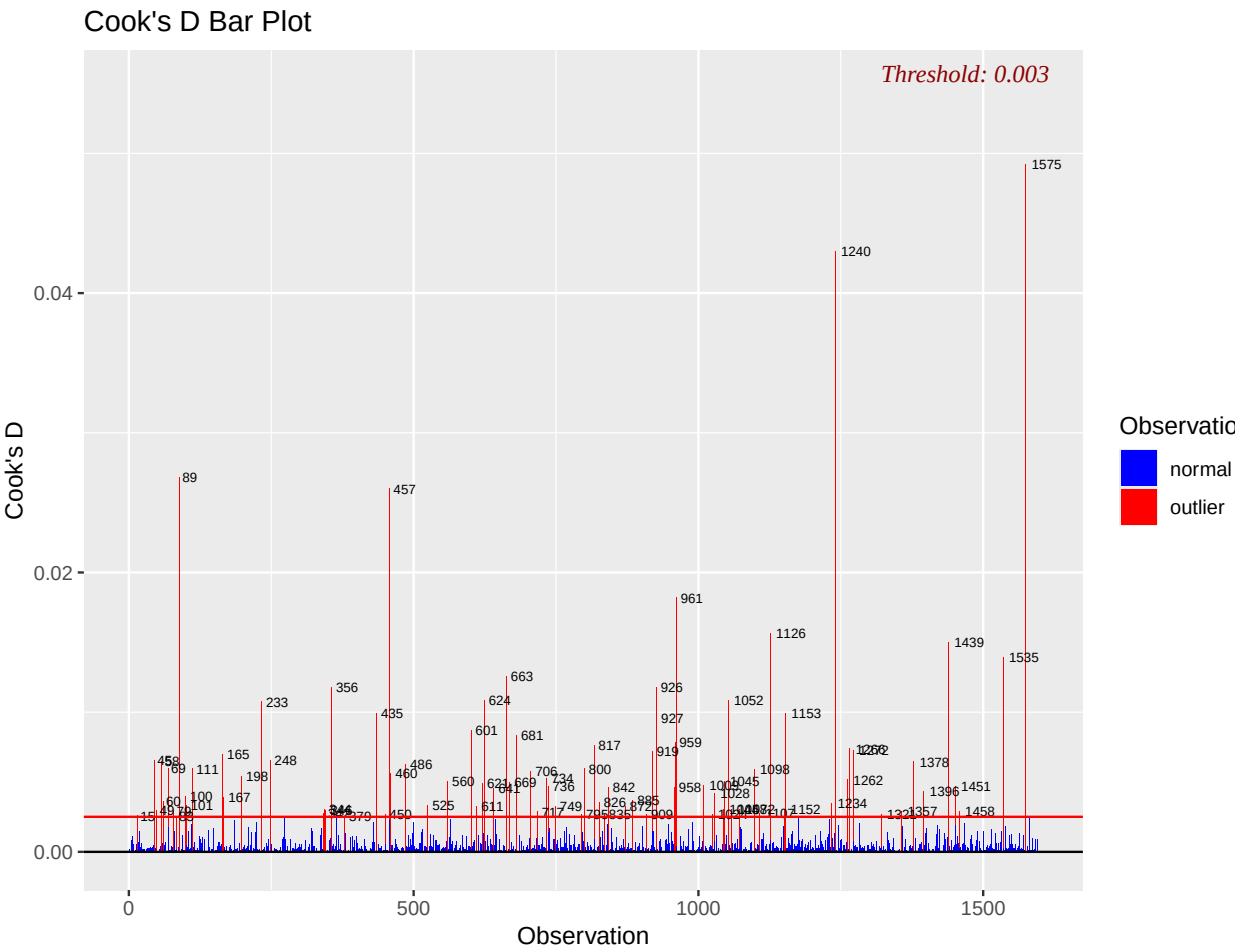
Partials for *PctPopPov:PctBSorMore* interaction model:



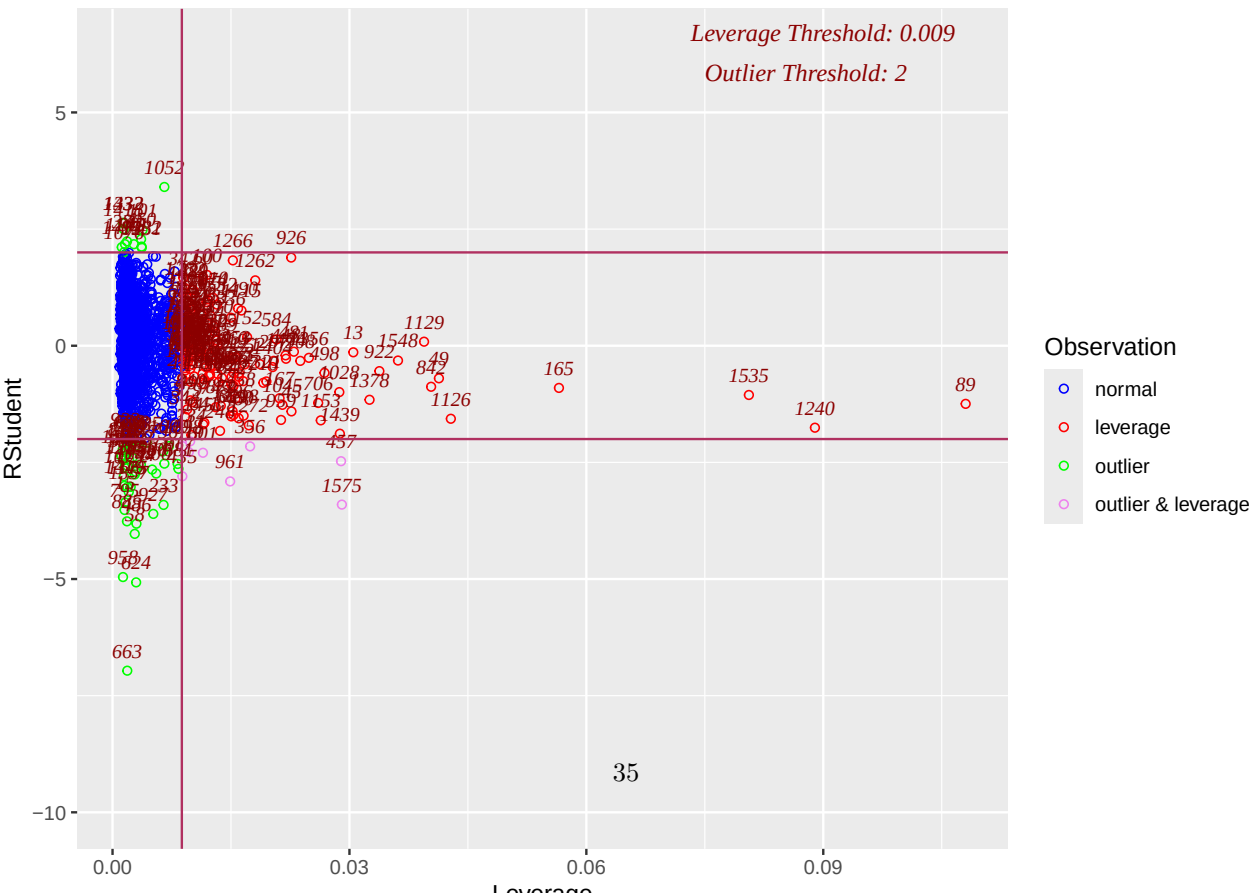
Plot of outliers vs population:



Diagnostics:

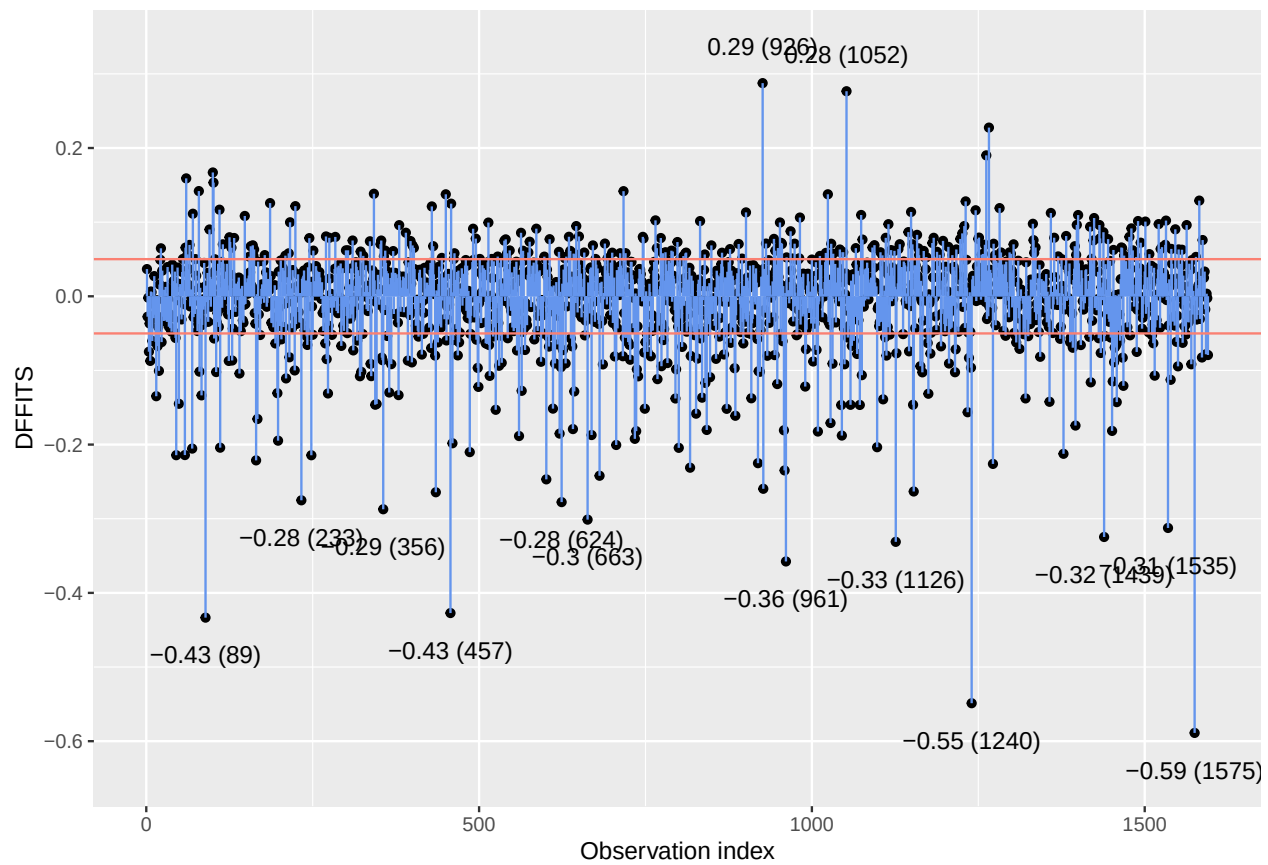


Diagnostics plots:
Outlier and Leverage Diagnostics for logViolent



DFFITS

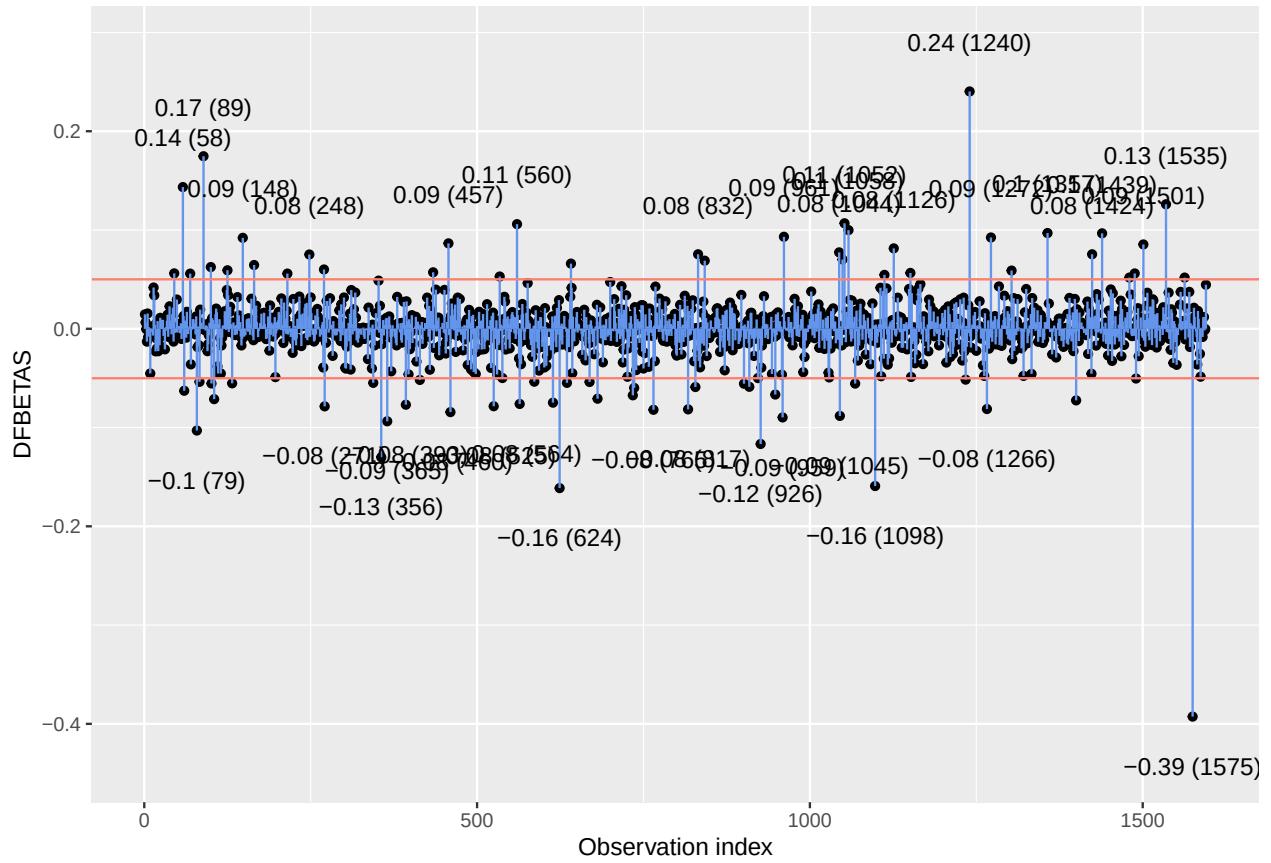
Thresholds are at $\pm 2.(p/n)$



[[1]]

DFBETAS values for coefficient of PctPopUnderPov

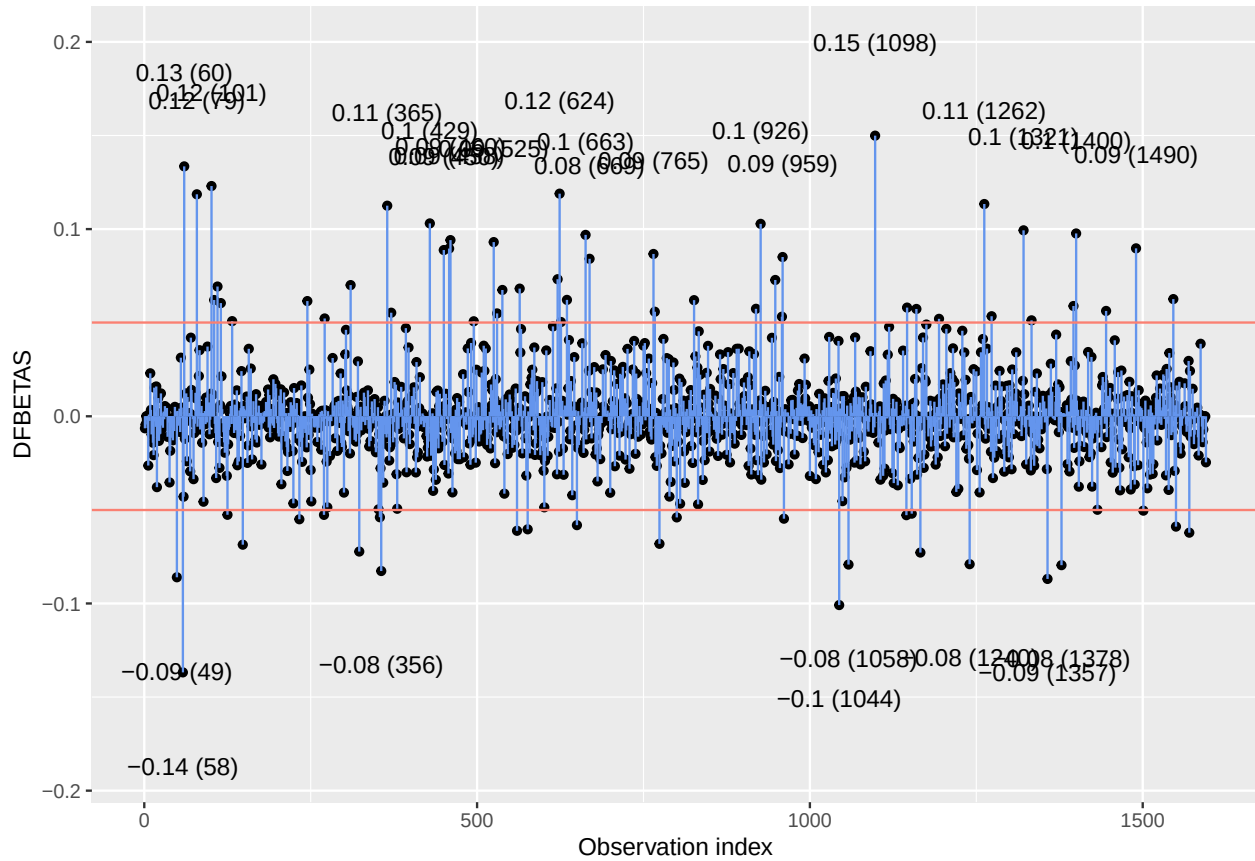
Thresholds are at $\pm(2\div.n)$



```
##
## [[2]]
```

DFBETAS values for coefficient of PctLess9thGrade

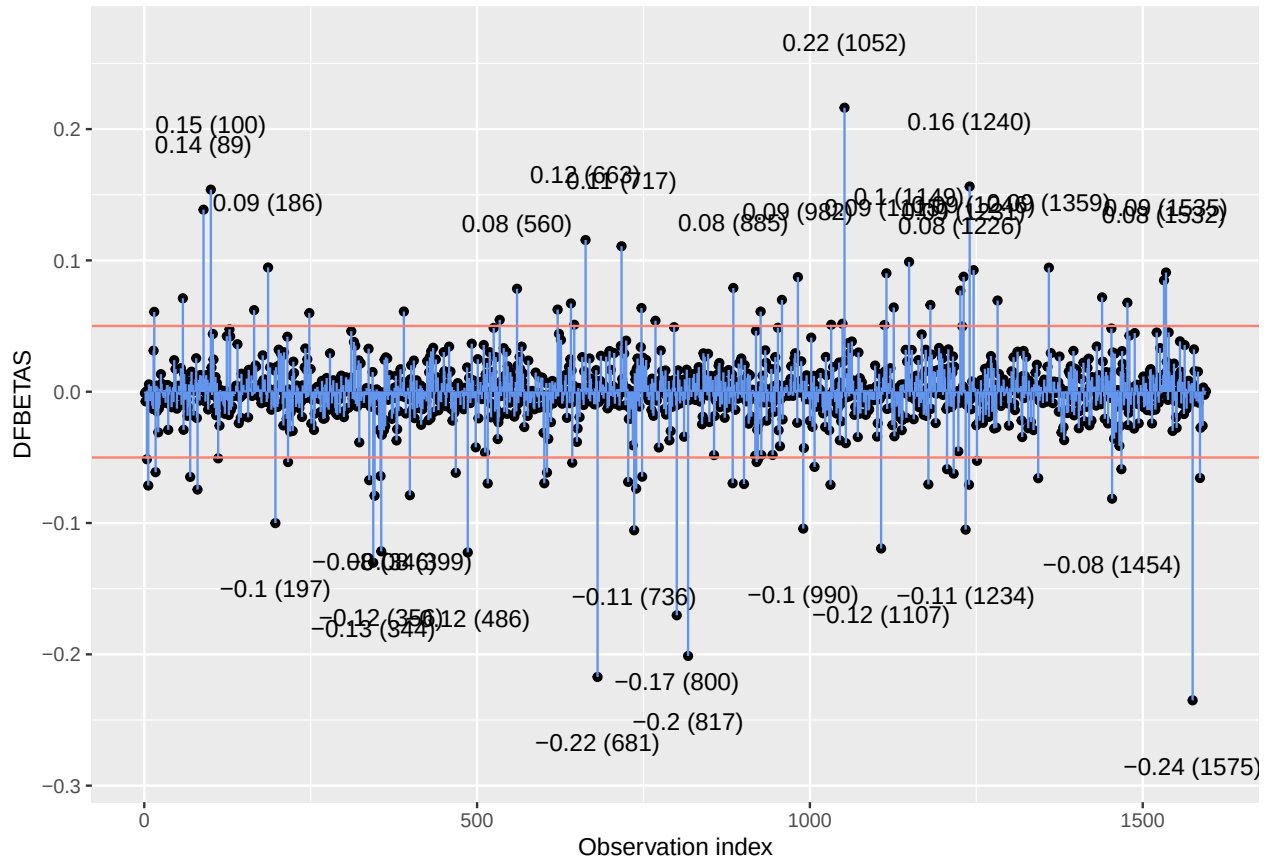
Thresholds are at $\pm(2\div.n)$



```
##
## [[3]]
```

DFBETAS values for coefficient of PctBSorMore

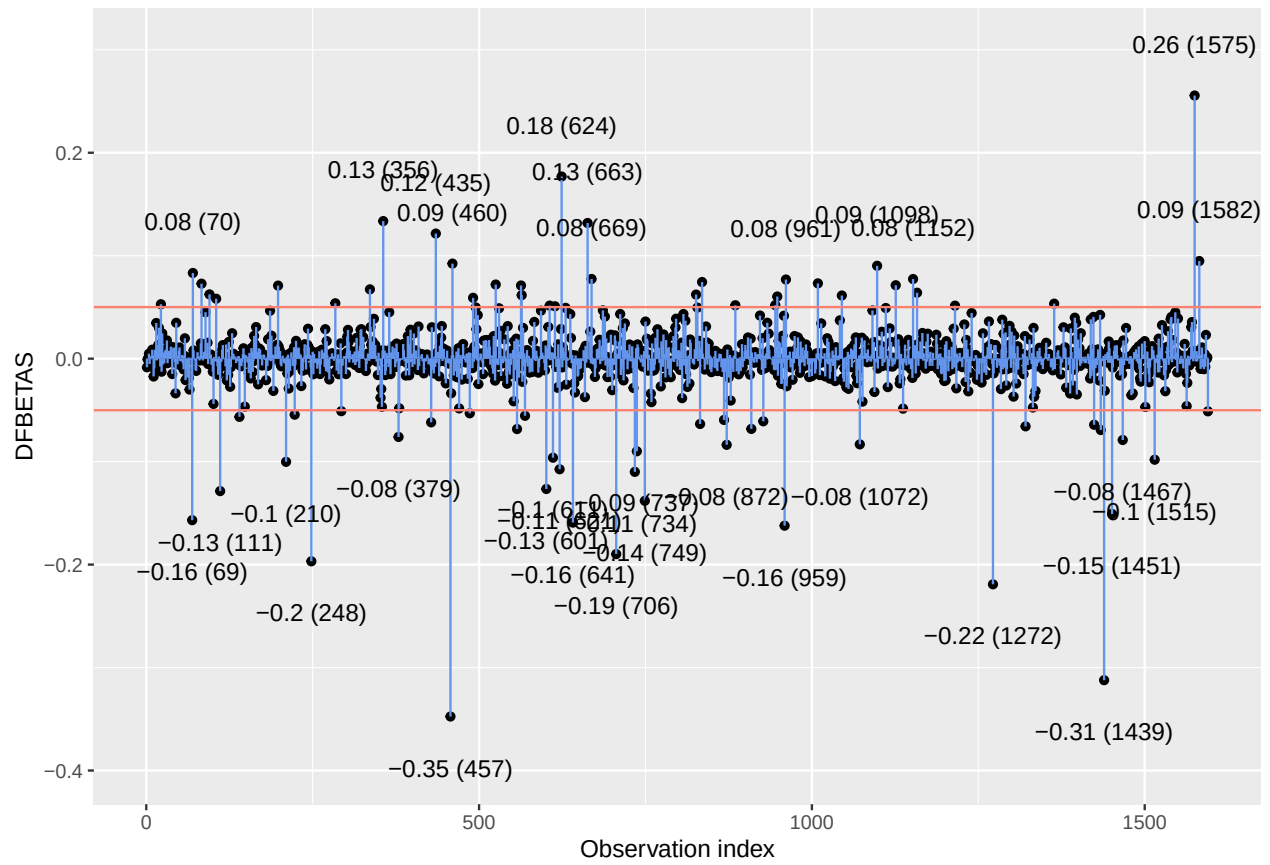
Thresholds are at $\pm(2\div.n)$



```
##
## [[4]]
```

DFBETAS values for coefficient of racepctblack

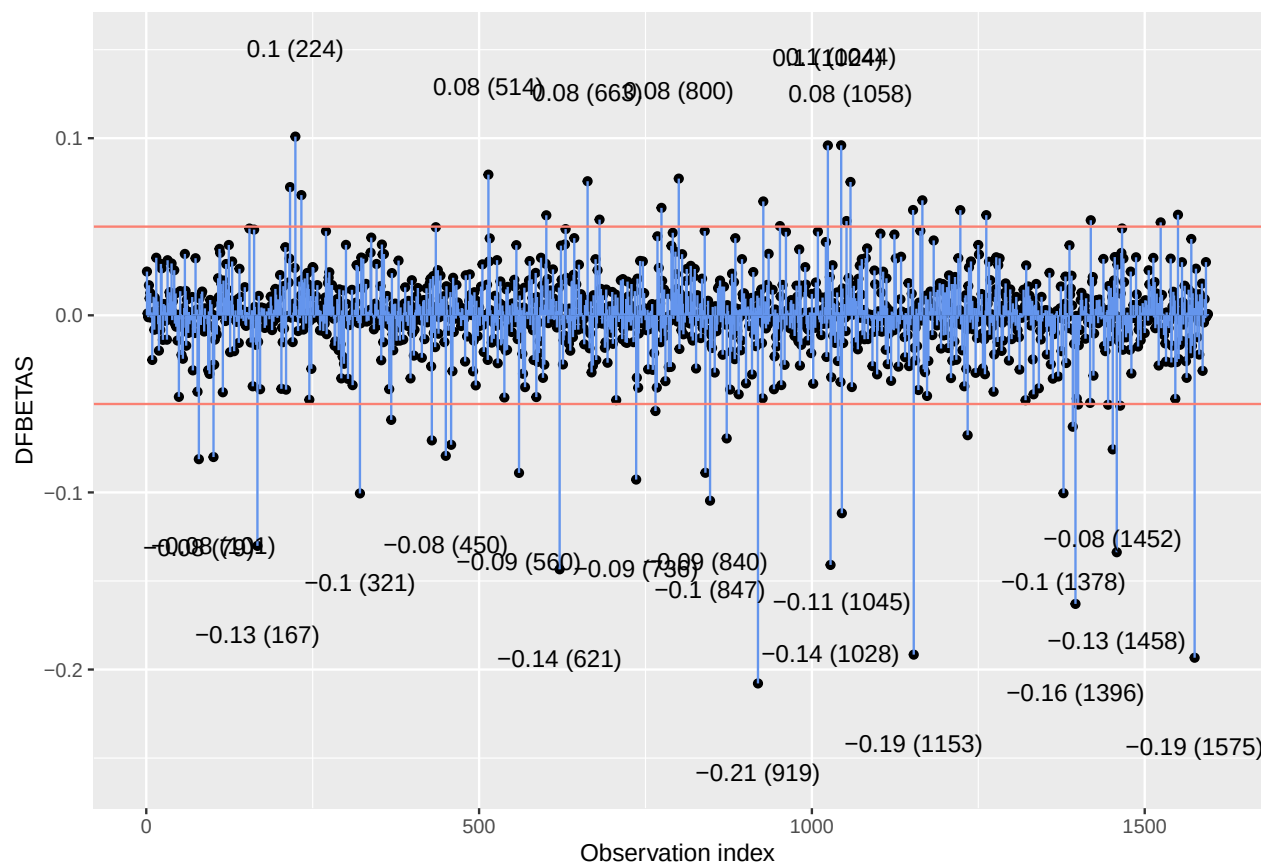
Thresholds are at $\pm(2\div n)$



```
##
## [[5]]
```

DFBETAS values for coefficient of PctImmig

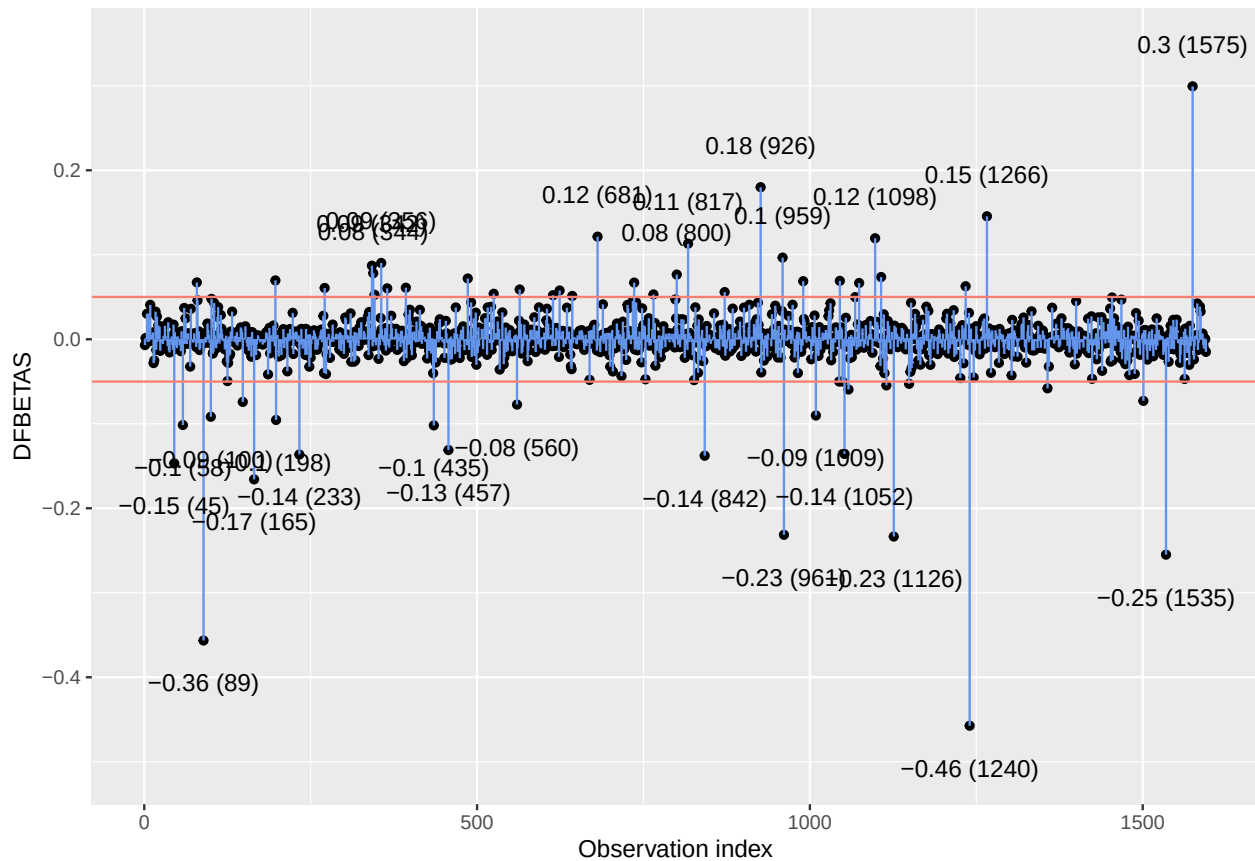
Thresholds are at $\pm(2\div.n)$



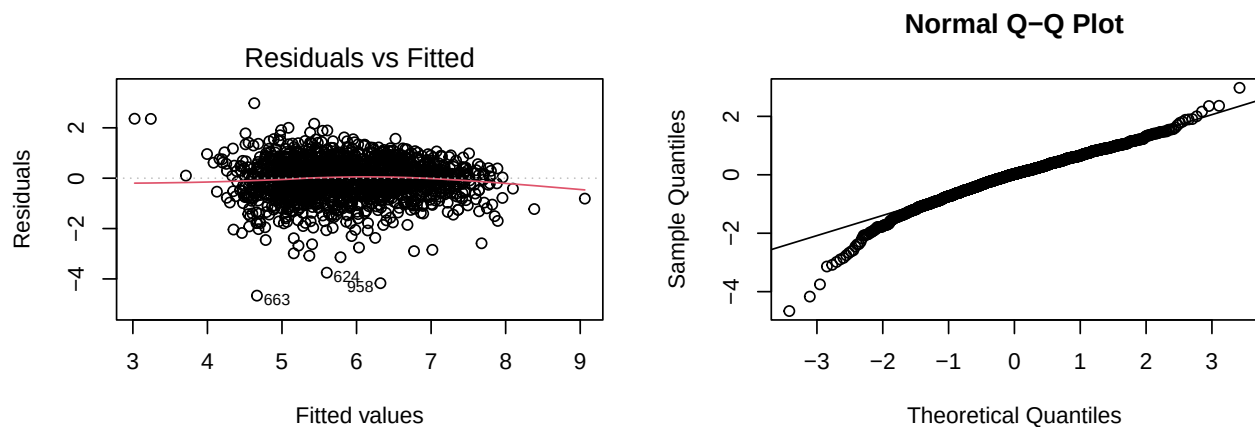
```
##
## [[6]]
```

DFBETAS values for coefficient of PctPopUnderPov.PctBSorMore

Thresholds are at $\pm(2\div n)$



Robust model plots:



Model validation

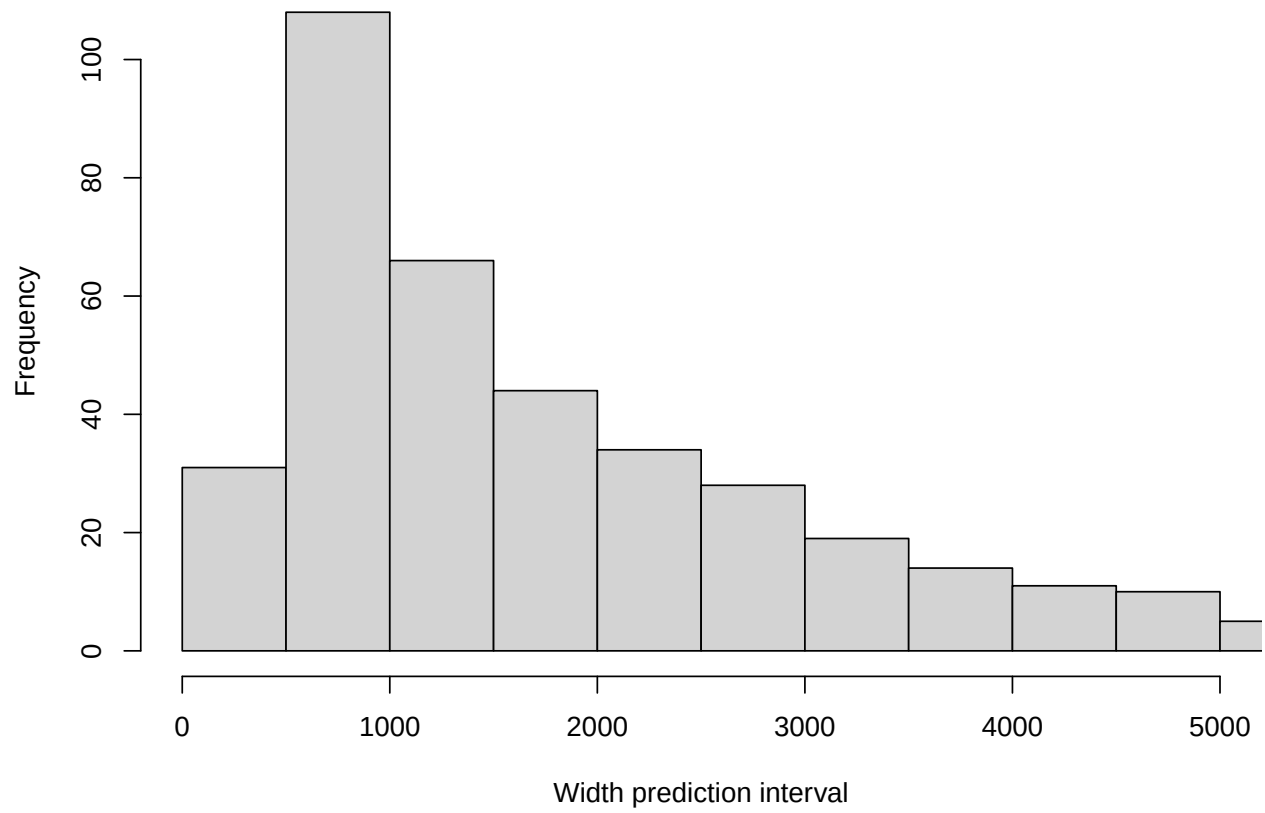
prediction interval plots validation:

Fraction of datapoints in prediction interval (logscale): 0.9724311

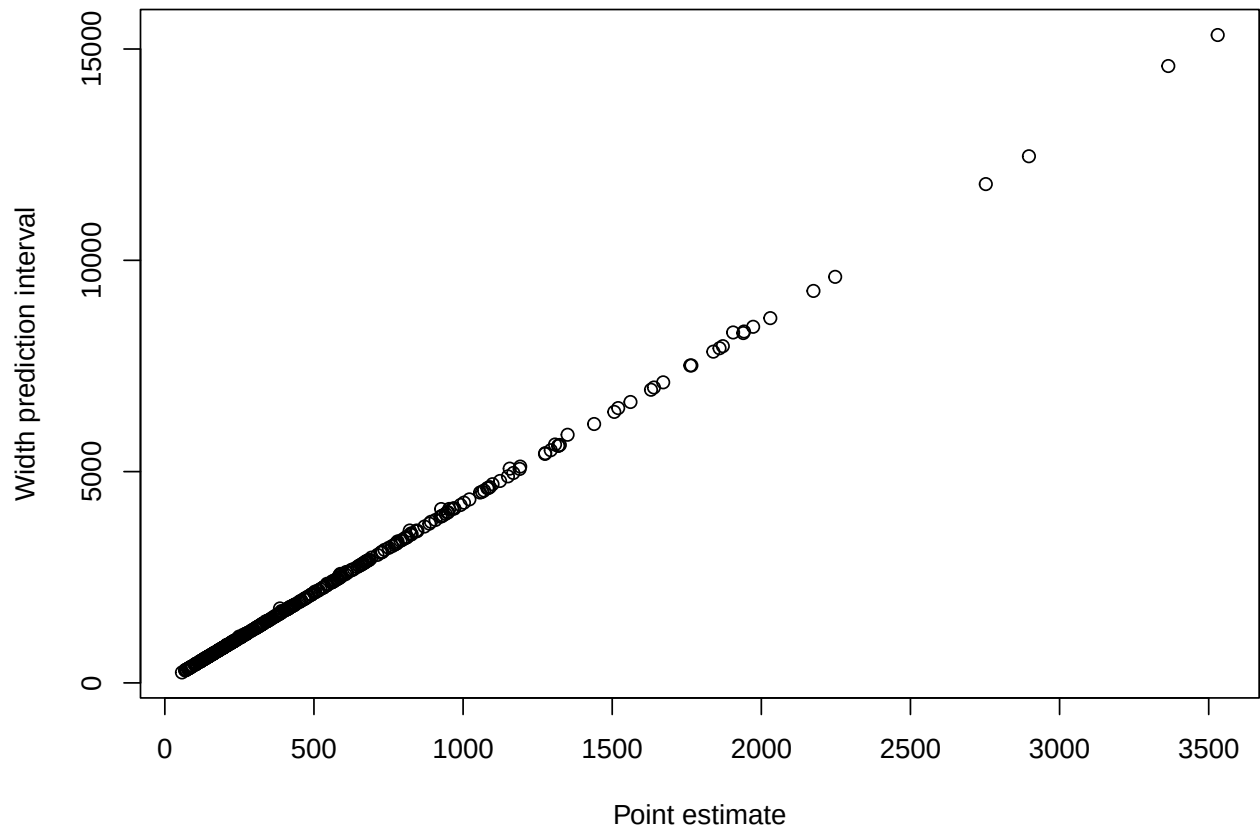
Fraction of datapoints in prediction interval: 0.9724311

Fraction of datapoints in prediction interval (univariate model): 0.9172932

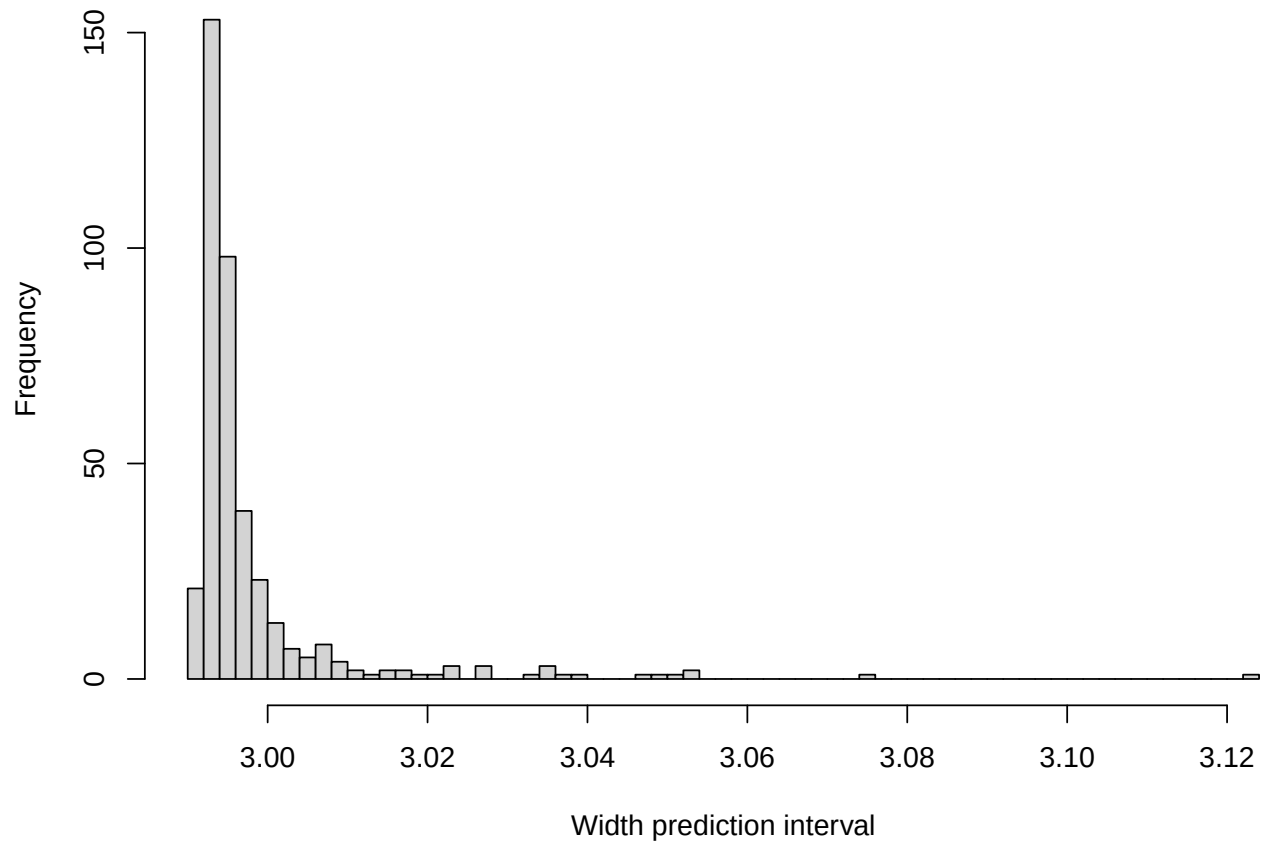
Width prediction interval



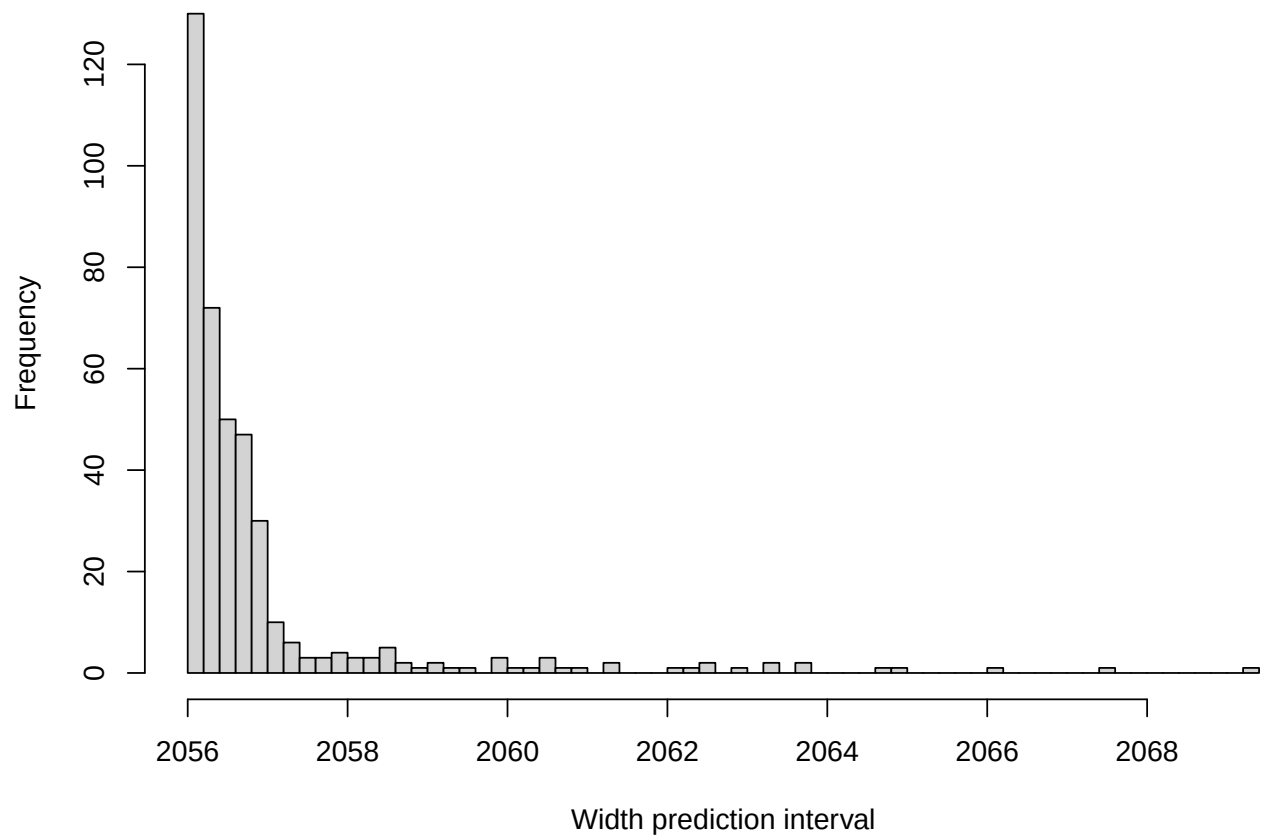
Width prediction interval



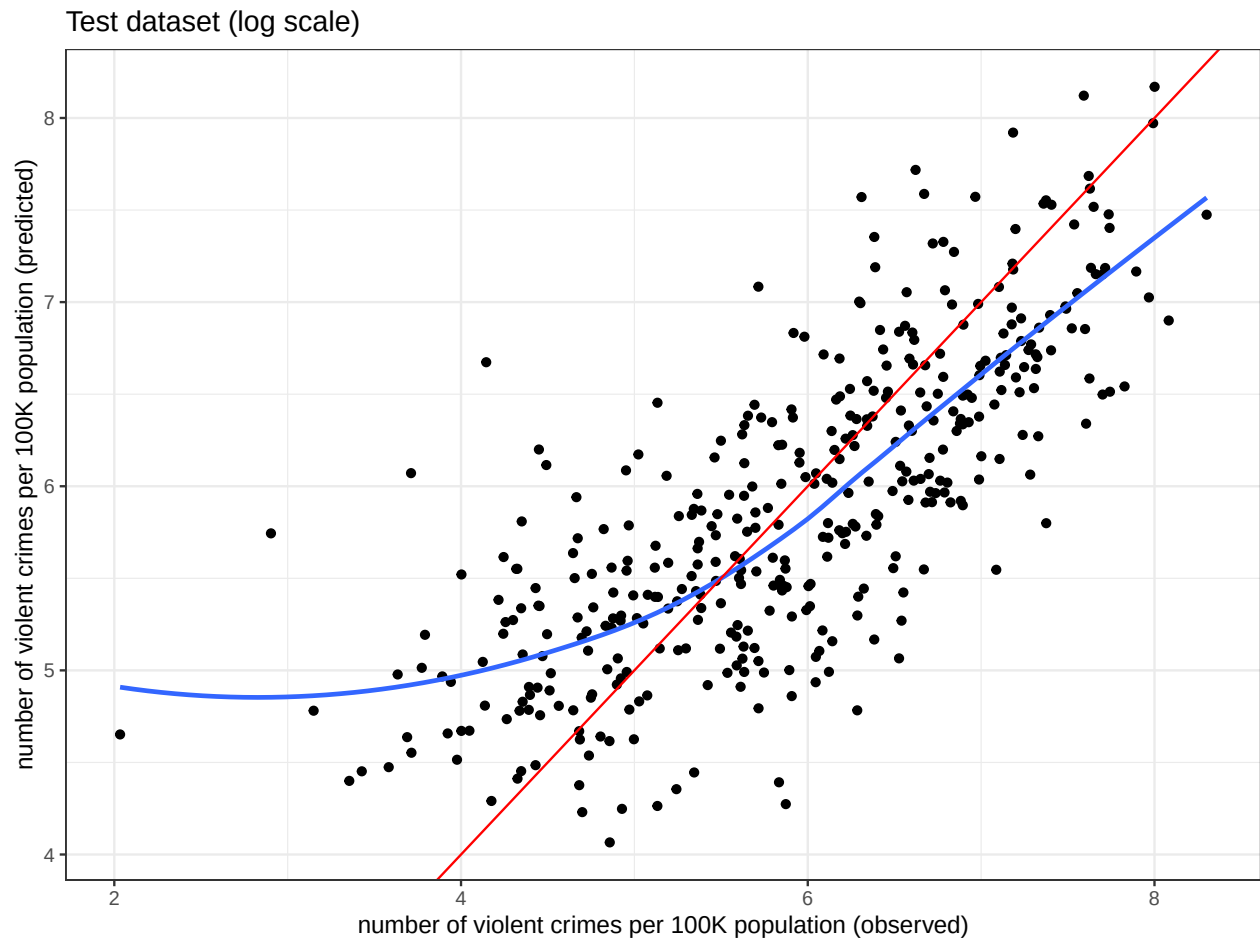
Width prediction interval (log)



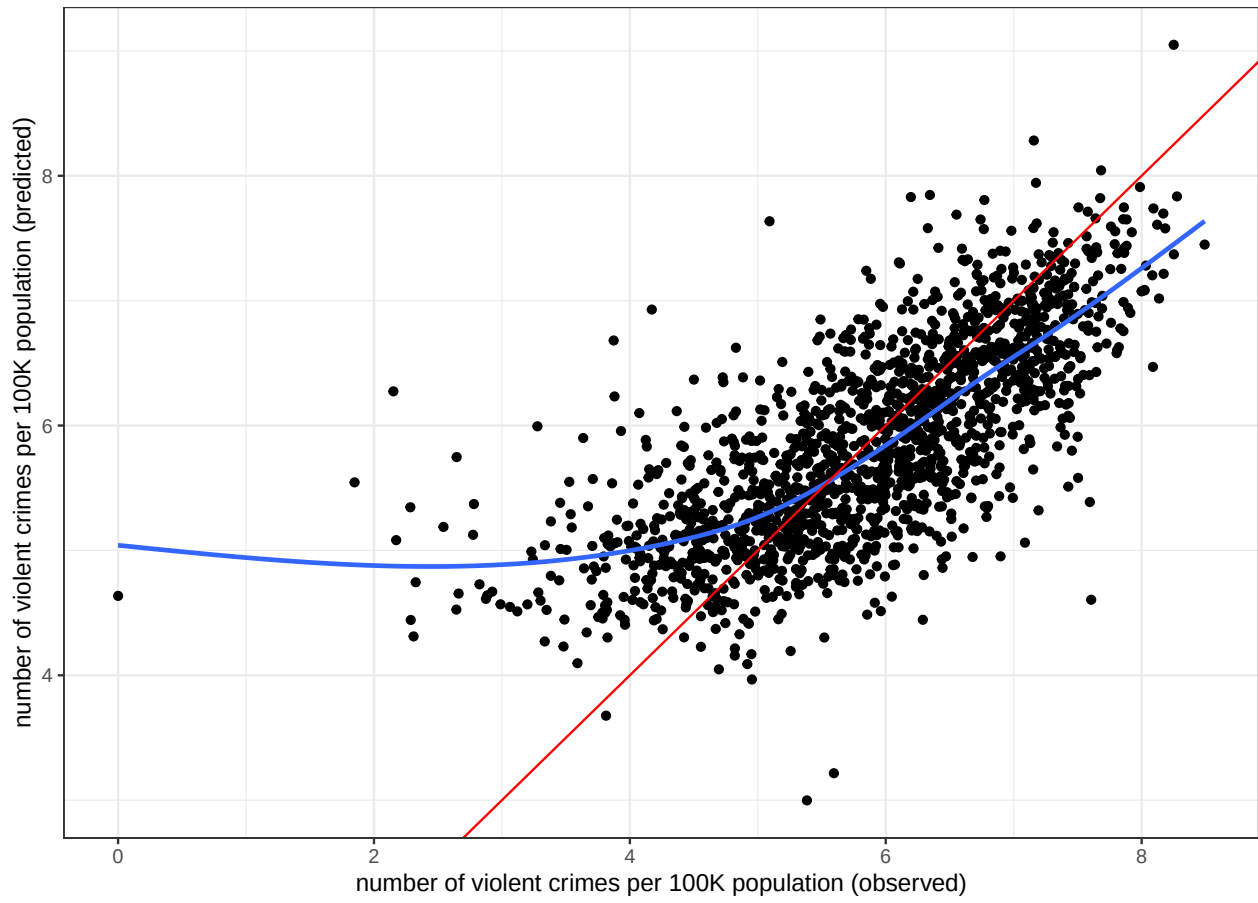
Width prediction interval (univariate model)



prediction tests:



Train dataset (log scale)



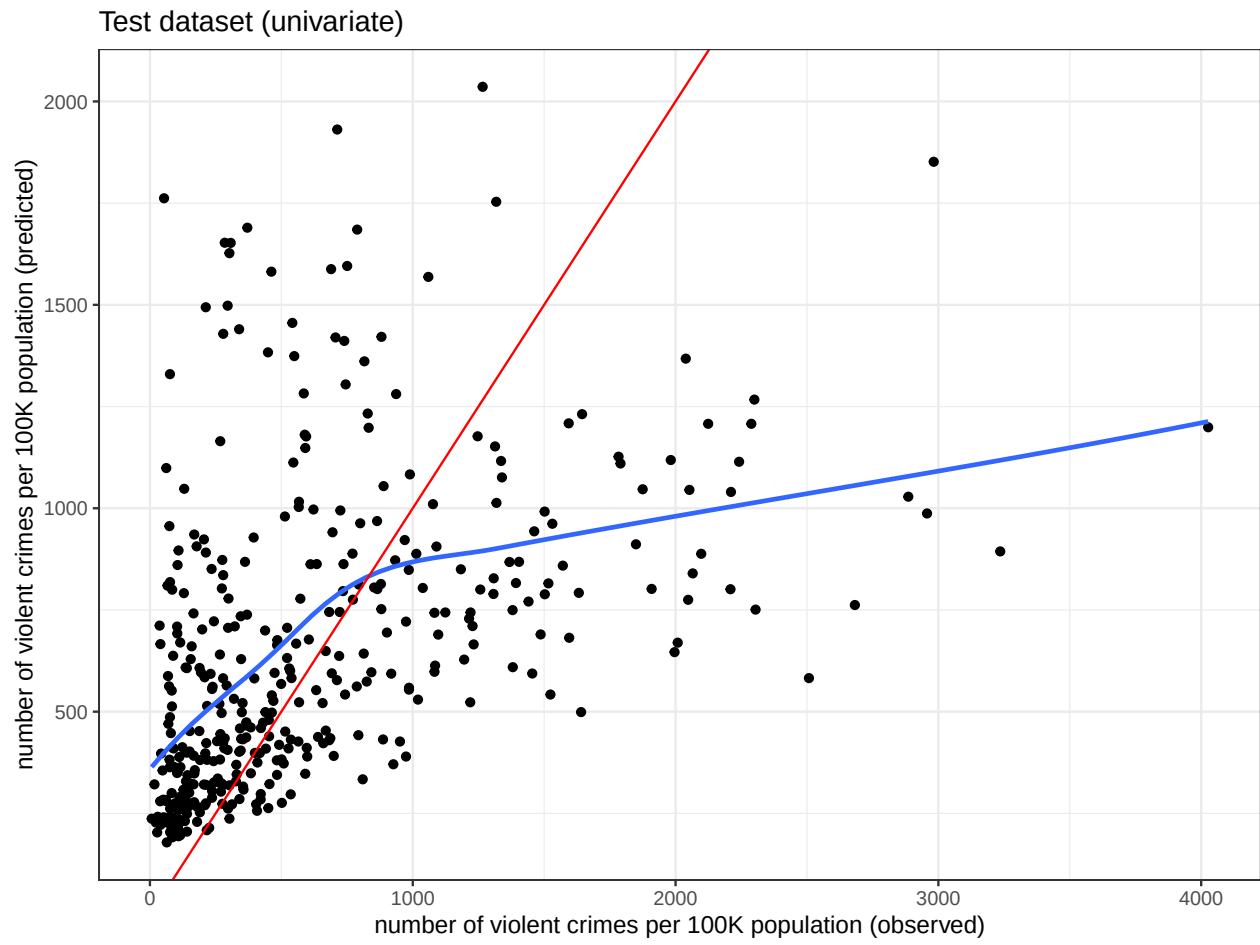
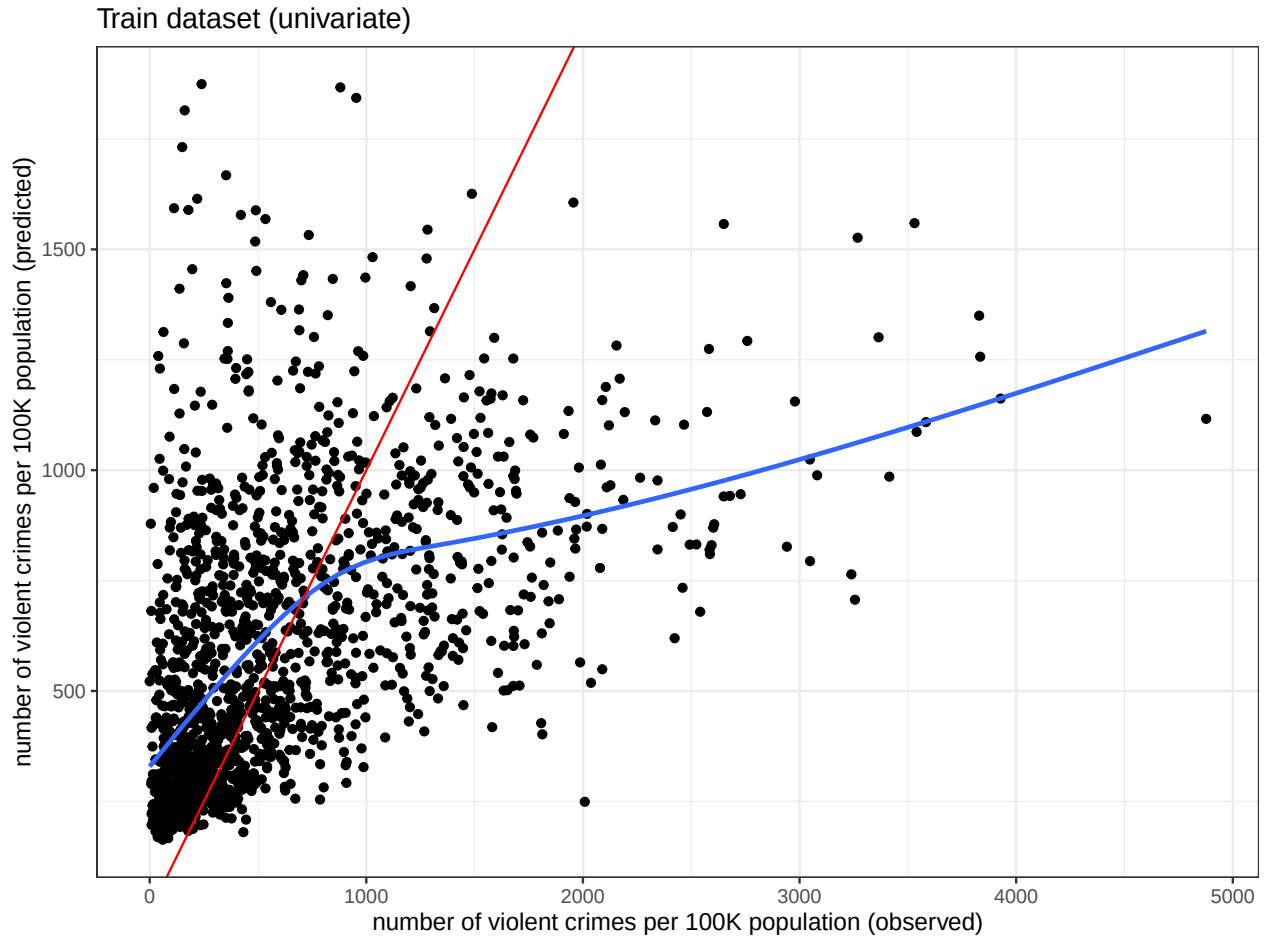


Table 1: Project contribution Overview

Subject	Code	Text
Data extraction	Thomas	Thomas
Data preparation	Ilja	Ilja
Descriptive analyses	Ilja	Ilja
Model building	Thomas	Thomas
Model diagnostics	Thomas&Ilja	Thomas
Model interpretation	Lieselot	Lieselot
Model validation	Ilja	Ilja
Statistical discussion linear model	Lieselot	Lieselot&Ilja
Final conclusion and discussion	Lieselot	Lieselot&Ilja



Contributions

AI:

During this project, AI has been used either to find out why certain errors occurred, Github related issues, and some practical issues such as for example referencing to figures in a later part of the code:

prompt:

how can i reference to figures in my appendix in r since this is after all text (did not work!!!)

Or to fix an error in getting the full formulas in a table during the model building:

```
prompt: model_ranking <- data.frame( + formula = sapply(formulas, deparse), + BIC = bic_values + )
Error in (function (..., row.names = NULL, check.rows = FALSE, check.names = TRUE, : arguments imply
differing number of rows: 1, 2, 3. Why do i get this error?
```

Code

```
knitr::opts_chunk$set(message = FALSE, warning = FALSE,
                        echo = FALSE, include = FALSE)

library(knitr)
library(glue)
library(dplyr)
library(data.table)
library(dplyr)
library(stringr)
library(rvest)
library(ggplot2)
library(gtsummary)
library(sjlabelled)
library(tidyr)
library(ggcorrplot)
library(patchwork)
library(MASS)
library(stdmod)

violent_crimes_table <- fread("curl https://archive.ics.uci.edu/static/public/211/communities+and+crime")

url <- "https://archive.ics.uci.edu/dataset/211/communities+and+crime+unnormalized"

# Read the HTML page
page <- read_html(url)

# Extract all text from the page
text <- page %>% html_text()

# Split into lines
lines <- str_split(text, "\n")[[1]]

start <- grep("Additional Variable Information", lines, ignore.case = TRUE)
end <- grep("Summary Statistics:", lines, ignore.case = TRUE)

# only retain the lines starting with --
var_lines <- lines[str_starts(str_trim(lines), "--")]
# remove the --
var_names <- sapply(strsplit(var_lines, "--"), function(x) str_trim(x[2]))
# only retain the variable names by cutting everything after the :
var_names <- str_extract(var_names, "^[^:]+")
colnames(violent_crimes_table) <- var_names
crimes_table_subset <- violent_crimes_table %>%
  dplyr::select(communitname, state, countyCode, communityCode, fold, population,
               PctPopUnderPov, perCapInc, PctEmploy, PctLess9thGrade, PctNotHSGrad, PctBSorMore,
               NumImmig, racepctblack, agePct12t29, ViolentCrimesPerPop
               )
# transformation of NumImmig variable
```



```

crimes_table_subset$ViolentCrimesPerPop <- as.numeric(crimes_table_subset$ViolentCrimesPerPop)
crimes_table_subset$PctImmig <- crimes_table_subset$NumImmig/crimes_table_subset$population*100
crimes_table_subset = crimes_table_subset[,-c('NumImmig', 'fold')]
sjlabelled::set_label(crimes_table_subset) <- c("communityname", "state", "countyCode", "communityCode"
or over, that have not graduated highschool (%)", "percentage of people 25 or over, with
at least a bachelor's degree (%)", "percentage of population that is african american (%)", "percentage
# counting the number of NA values per column
crimes_table_subset %>%
  pivot_longer(cols = where(is.numeric), names_to = "variable", values_to = "value") %>%
  group_by(variable) %>%
  summarise(
    NAs = sum(is.na(value))
  )
# filtered subset with only NA values
na_subset <- crimes_table_subset %>%
  filter(is.na(ViolentCrimesPerPop))
)
na_subset <- na_subset[,-'ViolentCrimesPerPop']
na_subset
# filtered subset without NA values
crimes_table_subset = na.omit(crimes_table_subset)
colSums(crimes_table_subset == "?", na.rm = TRUE)
# calculating table with summary statistics for non-missing data
crimes_table_subset %>%
  pivot_longer(cols = where(is.numeric), names_to = "variable", values_to = "value") %>%
  group_by(variable) %>%
  summarise(
    min = min(value, na.rm = TRUE),
    q25 = quantile(value, 0.25, na.rm = TRUE),
    mean = mean(value, na.rm = TRUE),
    sd = sd(value, na.rm = TRUE),
    q75 = quantile(value, 0.75, na.rm = TRUE),
    max = max(value, na.rm = TRUE),
    n = n(),
    NAs = sum(is.na(value))
  )
# calculating table with summary statistics for missing data
na_subset %>%
  pivot_longer(cols = where(is.numeric), names_to = "variable", values_to = "value") %>%
  group_by(variable) %>%
  summarise(
    min = min(value, na.rm = TRUE),
    q25 = quantile(value, 0.25, na.rm = TRUE),
    mean = mean(value, na.rm = TRUE),
    sd = sd(value, na.rm = TRUE),
    q75 = quantile(value, 0.75, na.rm = TRUE),
    max = max(value, na.rm = TRUE),
    n = n(),
    NAs = sum(is.na(value))
  )
# selecting numeric columns
numeric_cols_na <- sapply(na_subset, is.numeric)
# NA subset, only numeric columns

```

```

crimes_table_subset_num_na <- na_subset[, ..numeric_cols_na]
# extra column logpopulation
crimes_table_subset$logpopulation <- log10(crimes_table_subset$population)

# selecting numeric columns
numeric_cols <- sapply(crimes_table_subset, is.numeric)
# subset, only numeric columns
crimes_table_subset_num <- crimes_table_subset[, ..numeric_cols]
# setting the number of plots per pg
par(mfrow = c(4,2))

# plots for visualisation distributions
plots <- lapply(names(crimes_table_subset_num_na), function(columnname) {
  column <- crimes_table_subset_num[[columnname]]
  boxplot(column,
    main = columnname
  )
  hist(column,
    main = columnname,
    xlab = get_label(column)
  )
})
)
numeric_cols_na <- sapply(na_subset, is.numeric)
crimes_table_subset_num_na <- na_subset[, ..numeric_cols_na]

# setting number of plots per pg
par(mfrow = c(4,2))

# plotting histograms for comparison missing and non-missing data
plots <- lapply(names(crimes_table_subset_num_na), function(columnname) {
  column <- crimes_table_subset_num[[columnname]]
  column_na <- crimes_table_subset_num_na[[columnname]]
  hist(column,
    main = columnname,
    xlab = get_label(column)
  )
  hist(column_na,
    main = paste(columnname, "(missing data)"),
    xlab = get_label(column)
  )
})
)

# calculating correlation matrix
cor_matrix <- cor(crimes_table_subset_num[, -c('population', 'logpopulation')])
# transformation to dataframe
cor_values <- as.data.frame(as.table(cor_matrix))

# correlation plot with ggplot
cor_discriptives <- ggcorrplot(cor_matrix, lab = TRUE, type = "lower",
  lab_size = 3, colors = c("red", "white", "blue"))

# vector with variables

```

```

x_vars <- colnames(crimes_table_subset_num)

# creating named vector
dict_labels <- setNames(sapply(x_vars, function(x_var) get_label(crimes_table_subset_num[[x_var]])), x_vars)

# preparing dataframe for plots
df <- crimes_table_subset_num[, -c("population", "logpopulation")]
y_var <- "ViolentCrimesPerPop"
x_vars <- setdiff(colnames(df), y_var)

# creating plots with ViolentCrimesPerPop as Y variable
plots <- lapply(x_vars, function(x_var) {
  ggplot(df, aes_string(x_var, y_var)) +
    geom_point(alpha = 0.6, size = 0.7) +
    geom_smooth(method = "lm", color = "blue", se = FALSE) +
    geom_smooth(method = "loess", color = "red", se = FALSE) +
    theme_bw(base_size = 8) +
    labs(x = str_wrap(paste(x_var, " (", dict_labels[x_var], ")", sep = ""), width = 45))
})

# Print 9 plots per pg
print(wrap_plots(plots, ncol = 3))

# creating plots with PctPopUnderPov as Y variable
df <- crimes_table_subset_num[, -c("population", "ViolentCrimesPerPop", "logpopulation")]
y_var <- "PctPopUnderPov"
x_vars <- setdiff(colnames(df), y_var)
plots <- lapply(x_vars, function(x_var) {
  ggplot(df, aes_string(x_var, y_var)) +
    geom_point(alpha = 0.6, size = 0.7) +
    geom_smooth(method = "lm", color = "blue", se = FALSE) +
    geom_smooth(method = "loess", color = "red", se = FALSE) +
    theme_bw(base_size = 8) +
    labs(x = str_wrap(paste(x_var, " (", dict_labels[x_var], ")", sep = ""), width = 45))
})

# Print 9 plots per pg
print(wrap_plots(plots, ncol = 3))

# table of observations with low number of crimes
crimes_table_subset[order(crimes_table_subset_num$ViolentCrimesPerPop, decreasing=FALSE), , drop = FALSE]

# preparing dataframe for plots
df <- crimes_table_subset_num
x_var <- "logpopulation"
y_vars <- setdiff(colnames(df), x_var)

# plot variables in relation to log(population)
plots <- lapply(y_vars, function(y_var) {
  ggplot(df, aes_string(x_var, y_var)) +
    geom_point(alpha = 0.6, size = 0.7) +
    geom_smooth(method = "lm", color = "blue", se = FALSE) +
    geom_smooth(method = "loess", color = "red", se = FALSE) +
    theme_bw(base_size = 8) +

```

```

    labs(x = "log(population)", y = str_wrap(y_var, width = 45))
  }
)

# Print 9 plots per pg
print(wrap_plots(plots, ncol = 3))

# set.seed and splitting dataset at random in test and training dataset
set.seed(987654321)
n <- nrow(crimes_table_subset_num)
training <- sample(1:n, size = floor(0.8 * n))
train_data <- crimes_table_subset[training, ]
test_data <- crimes_table_subset[-training, ]
n_training <- nrow(train_data)

cat("Training set size:", n_training, "\n")
cat("Test set size:", nrow(test_data), "\n")
# fit of simple linear model
fit_simple <- lm(ViolentCrimesPerPop ~ PctPopUnderPov, data = train_data)
summary(fit_simple)

cat("Regression equation: ViolentCrimesPerPop =",
    round(coef(fit_simple)[1], 2), "+",
    round(coef(fit_simple)[2], 2), "* PctPopUnderPov\n\n")

# R-squared, R-squared_adjusted and BIC-value
cat("R-squared:", round(summary(fit_simple)$r.squared, 4), "\n")
cat("Adjusted R-squared:", round(summary(fit_simple)$adj.r.squared, 4), "\n")
cat("BIC:", BIC(fit_simple), "\n")

# MSE calculation
sse_simple <- sum(fit_simple$residuals^2)
mse_simple <- sse_simple/(n_training - 2)
cat("SSE:", round(sse_simple, 2), "\n")
# Calculation of BIC-waarde in R: -2 * as.numeric(logLik(fit)) + attr(logLik(fit), "df") * log(n)
# Not the one from the course slides n*log(sse_simple) - n*log(n) + p*log(n), but ok?
cat("MSE:", round(mse_simple, 2), "\n")

# Confidence intervals for coefficients
cat("\n95% Confidence Intervals:\n")
print(confint(fit_simple))

# ANOVA
anova(fit_simple)
par(mfrow = c(2, 2))

#Residuals vs Fitted
plot(fit_simple$fitted.values, fit_simple$residuals,
     xlab = "Fitted values", ylab = "Residuals",
     main = "Residuals vs Fitted")
abline(h = 0, col = "red", lty = 2)
lines(lowess(fit_simple$fitted.values, fit_simple$residuals), col = "blue")

```

```

# Squared residuals vs Fitted
plot(fit_simple$fitted.values, fit_simple$residuals^2,
     xlab = "Fitted values", ylab = "Squared Residuals",
     main = "Squared Residuals vs Fitted")
lines(lowess(fit_simple$fitted.values, fit_simple$residuals^2), col = "blue")

# QQ-plot of residuals (normality)
qqnorm(fit_simple$residuals, main = "Normal Q-Q Plot of Residuals")
qqline(fit_simple$residuals, col = "red")

# Studentized residuals
stud_res <- rstudent(fit_simple)
plot(fit_simple$fitted.values, stud_res,
     xlab = "Fitted values", ylab = "Studentized Residuals",
     main = "Studentized Residuals vs Fitted")
outliers_simple <- which(abs(stud_res) > 2)
par(mfrow = c(1, 2))
# Residuals vs log(population)
plot(train_data$logpopulation, fit_simple$residuals,
     xlab = "log(population)", ylab = "Residuals")
abline(h = 0, col = "red", lty = 2)
lines(lowess(train_data$logpopulation, fit_simple$residuals), col = "blue")

# Studentized residuals vs log(population)
plot(train_data$logpopulation, stud_res,
     xlab = "log(population)", ylab = "Studentized Residuals")
abline(h = 0, col = "red", lty = 2)
lines(lowess(train_data$logpopulation, stud_res), col = "blue")
par(mfrow = c(1, 1))
max_extra_predictors <- 4
# define predictor variables for model selection
predictors <- c("perCapInc", "PctEmploy",
               "PctLess9thGrade", "PctNotHSGrad", "PctBSorMore",
               "racepctblack", "agePct12t29", "PctImmig")

# education variables
educ <- c("PctLess9thGrade", "PctNotHSGrad", "PctBSorMore")

formulas <- list()
# create all possible formulas to a max of max_extra_predictors plus main pred
for (i in 1:max_extra_predictors) {
  tmp <- combn(predictors, i)
  tmp <- apply(tmp, 2, paste, collapse=" + ")
  tmp <- paste0("ViolentCrimesPerPop~PctPopUnderPov + ", tmp)
  formulas[[i]] <- tmp
}
# make the list compatible with the as formula function
formulas <- unlist(formulas)
# makes formulas into real formulas
formulas <- sapply(formulas, as.formula)
# make a model for each fomula option
models <- lapply(formulas, lm, data=train_data)

```

```

# calculate the BIC, R squared, adjusted R squared for each model
bics <- sapply(models, BIC)
r_square <- sapply(models, function(m) summary(m)$r.squared)
adj_r_square <- sapply(models, function(m) summary(m)$adj.r.squared)
formula_vector <- vapply(formulas, function(f) paste(deparse(f), collapse = ""))
                        , character(1))

# make dataframe for better comparison with these criteria
model_ranking <- data.frame(
  formula = formula_vector,
  r.square = r_square,
  adj.r.square = adj_r_square,
  BIC = bics
)

# sort models based on lowest BIC value
model_ranking <- model_ranking[order(model_ranking$BIC), ]
# Print best model based on lowest BIC
best <- which.min(model_ranking$BIC)
# extract formula
best_pred <- model_ranking$formula[best]
# print stats
cat("\nBest model:", best_pred)
cat("\nBIC:", round(model_ranking$BIC[best], 2), "\n")
cat("Adjusted R2:", round(model_ranking$adj.r.square[best], 4), "\n")
# show model stats
fit_multi <- lm(best_pred, data = train_data)
multi_var_summary <- summary(fit_multi)
multi_var_summary
par(mfrow = c(2, 2))

# Residuals vs Fitted
plot(fit_multi$fitted.values, fit_multi$residuals,
     xlab = "Fitted values", ylab = "Residuals",
     main = "Residuals vs Fitted")
abline(h = 0, col = "red", lty = 2)
lines(lowess(fit_multi$fitted.values, fit_multi$residuals), col = "blue")

# Squared residuals vs Fitted
plot(fit_multi$fitted.values, fit_multi$residuals^2,
     xlab = "Fitted values", ylab = "Squared Residuals",
     main = "Squared Residuals vs Fitted")
lines(lowess(fit_multi$fitted.values, fit_multi$residuals^2), col = "blue")

# QQ-plot
qqnorm(fit_multi$residuals, main = "Normal Q-Q Plot")
qqline(fit_multi$residuals, col = "red")

# Studentized residuals
stud_res_multi <- rstudent(fit_multi)
plot(fit_multi$fitted.values, stud_res_multi,
     xlab = "Fitted values", ylab = "Studentized Residuals",
     main = "Studentized Residuals vs Fitted")
abline(h = 0, col = "red", lty = 2)

```

```

lines(lowess(fit_multi$fitted.values, stud_res_multi), col = "blue")

par(mfrow = c(1, 1))
# log regression
fit_log <- lm(log(ViolentCrimesPerPop + 1) ~
              PctPopUnderPov + PctBSorMore + racepctblack +
              PctImmig + PctPopUnderPov:PctBSorMore,
              data = train_data)

multi_var_summary_logtest <- summary(fit_log)
plot(fit_log)
par(mfrow = c(1, 1))
# adding extra column log(ViolentCrimesPerPop + 1)
train_data$logViolent <- log(train_data$ViolentCrimesPerPop + 1)
test_data$logViolent <- log(test_data$ViolentCrimesPerPop + 1)
max_extra_predictors <- 4
# Define predictor variables for model selection
predictors <- c("perCapInc", "PctEmploy",
               "PctLess9thGrade", "PctNotHSGrad", "PctBSorMore",
               "racepctblack", "agePct12t29", "PctImmig")

# Educ variables
educ <- c("PctLess9thGrade", "PctNotHSGrad", "PctBSorMore")

# create all possible formulas to a max of max_extra_predictors plus main pred
formulas <- list()
for (i in 1:max_extra_predictors) {
  tmp <- combn(predictors, i)
  tmp <- apply(tmp, 2, paste, collapse=" + ")
  tmp <- paste0("logViolent~PctPopUnderPov + ", tmp)
  formulas[[i]] <- tmp
}
# make the list compatible with the as formula function
formulas <- unlist(formulas)
# makes formulas into real formulas
formulas <- sapply(formulas, as.formula)
# make a model for each formula option
models <- lapply(formulas, lm, data=train_data)

# calculate the BIC, R squared, adjusted R squared for each model
bics <- sapply(models, BIC)
r_square <- sapply(models, function(m) summary(m)$r.squared)
adj_r_square <- sapply(models, function(m) summary(m)$adj.r.squared)
formula_vector <- vapply(formulas, function(f) paste(deparse(f), collapse = "")
                        , character(1))

# make dataframe for better comparison with these criteria
model_ranking <- data.frame(
  formula = formula_vector,
  r.square = r_square,
  adj.r.square = adj_r_square,
  BIC = bics
)

```

```

# sort models based on lowest BIC value
model_ranking <- model_ranking[order(model_ranking$BIC), ]
# Print best model based on lowest BIC
best <- which.min(model_ranking$BIC)
# extract formula
best_pred <- model_ranking$formula[best]
# print stats
cat("\nBest model:", best_pred)
cat("\nBIC:", round(model_ranking$BIC[best], 2), "\n")
cat("Adjusted R2:", round(model_ranking$adj.r.square[best], 4), "\n")
# show model stats
# fit linear model with selected variables
fit_multi <- lm(best_pred, data = train_data)
multi_var_summary1 <- summary(fit_multi)
multi_var_summary1
# list with all predictors
predictor_list <- row.names(multi_var_summary1$coefficients[-1,])

# function to generate partial regression plots
partial_regression_plot <- function(data, outcome, predictor, predictor_list) {
  temp <- data
  controls <- setdiff(predictor_list, predictor)
  formula_outcome <- as.formula(paste(outcome, "~",
                                     paste(controls, collapse = " + ")))
  formula_pred <- as.formula(paste(predictor, "~",
                                   paste(controls, collapse = " + ")))

  # regression of outcome variable ~ controls
  lm_outcome <- lm(formula_outcome, data = temp)
  temp$resid_outcome <- resid(lm_outcome)
  # regression of predictor variable ~ controls
  lm_pred <- lm(formula_pred, data = temp)
  temp$resid_pred <- resid(lm_pred)
# plot
ggplot(temp, aes(x = resid_pred, y = resid_outcome)) +
  geom_point() +
  geom_smooth(method = "lm", se = FALSE) +
  geom_smooth(method = "loess", se = FALSE) +
  geom_hline(yintercept = 0, linetype = "dashed") +
  labs(x = paste("Residuals ", predictor, " ~ controls", sep = ""),
       y = paste("Residuals ", outcome, " ~ controls", sep = ""))
}

# generate plots for all predictor variables
plots <- lapply(predictor_list, function(predictor) {
  partial_regression_plot(train_data, "logViolent", predictor, predictor_list)
})

# Print 9 plots per pg
print(wrap_plots(plots, ncol = 3))
# function for logit transformation of racepctblack predictor variable
pct_to_logit <- function(percentage, addition = 1e-6) {

```



```

percentiles_in_range01 <- percentage / 100
# add small value to avoid 0 and 1
percentiles_in_range01 <- pmin(
  pmax(percentiles_in_range01, addition),
  1 - addition)
logit <- log(percentiles_in_range01 /
              (1 - percentiles_in_range01))

logit
}

# extra columns in dataframes with logit(racepctblack)
train_data$race_black_logit <- pct_to_logit(train_data$racepctblack)
test_data$race_black_logit <- pct_to_logit(test_data$racepctblack)
# fit logit model to test
fit_logit <- lm(logViolent ~
                PctPopUnderPov +
                PctLess9thGrade +
                PctBSorMore +
                race_black_logit +
                PctImmig,
                data = train_data)

# summary and assumptions plots
multi_var_logit_summary <- summary(fit_logit)
#multi_var_logit_summary
plot(fit_logit)
par(mfrow = c(1, 1))
# generate partial regression plots for all predictor variables
predictor_list <- row.names(multi_var_logit_summary$coefficients[-1,])
plots <- lapply(predictor_list, function(predictor) {
  partial_regression_plot(train_data, "logViolent", predictor, predictor_list)
})

# Print 9 plots per pg
print(wrap_plots(plots, ncol = 3))
# extract all coefficients that are not the intercept or PctPopUnderPov
selected_vars <- setdiff(
  row.names(multi_var_logit_summary$coefficients),
  c("(Intercept)", "PctPopUnderPov")
)

# create interaction terms with PctPopUnderPov and the remaining variables
interaction_terms <- paste("PctPopUnderPov", selected_vars, sep = ":")

# make formulas containing the best model and each interaction term
formulas_interaction <- sapply(interaction_terms, function(i) {
  paste("logViolent ~", paste(predictor_list, collapse = " + "), "+",
        i)
})

# make them formulas
formulas_interaction <- lapply(formulas_interaction, as.formula)
# create a model for each formula
models <- lapply(formulas_interaction, lm, data=train_data)

```

```

# reconstruct formula in a way to add in table later (see AI help)
interaction_models_form <- vapply(formulas_interaction, function(f)
  paste(deparse(f), collapse = ""), character(1))

# extract summary values function
summary_val_extraction <- function(x, item) {
  tmp <- summary(x)$coefficients
  tmp[nrow(tmp), item]
}

# extract p and t values
p_vals_interaction <- sapply(models, function(m)
  summary_val_extraction(m, "Pr(>|t|)"))
t_vals_interaction <- sapply(models, function(m)
  summary_val_extraction(m, "t value"))

# calculate the BIC, R squared, adjusted R squared for each model
bics <- sapply(models, BIC)
r_square <- sapply(models, function(m) summary(m)$r.squared)
adj_r_square <- sapply(models, function(m) summary(m)$adj.r.squared)
delta_adj_r_square <- sapply(models, function(m)
  summary(m)$adj.r.squared - summary(fit_multi)$adj.r.squared)

# add values to table
interaction_table <- data.frame(
  p.vals = p_vals_interaction,
  t.vals = t_vals_interaction,
  r.square = r_square,
  adj.r.square = adj_r_square,
  delta.adj = delta_adj_r_square,
  BIC = bics
)

# sort based on p-values
interaction_table <- interaction_table[order(interaction_table$p.vals), ]
interaction_table

best_interaction <- row.names(interaction_table[1,])
# Estimate model with interaction
formula_final <- as.formula(paste("logViolent ~",
  paste(predictor_list, collapse = " + "), "+",
  "PctPopUnderPov:PctLess9thGrade"))
fit_less_9thgrade <- lm(formula_final, data = train_data)
summary(fit_less_9thgrade)
library(car)
# calculating VIF values
vif <- vif(fit_less_9thgrade)
print(vif)
cat("BIC: ", BIC(fit_less_9thgrade))
# filtering formulas with more than 1 education variable
filtered_formulas <- Filter(function(f) {
  variables <- all.vars(f)
  number_education_variables <- sum(variables %in% educ)
  number_education_variables <= 1

```

```

}, formulas)

#change formulas to contain the logit based race variable
filtered_formulas <- lapply(filtered_formulas, function(f) {
  f_str <- deparse(f) # make the formula a vector
  f_str <- paste(f_str, collapse = " ") # make the vectors into strings
  f_str <- gsub("racepctblack", "race_black_logit", f_str) # replace racepctblack to logit
  as.formula(f_str) # revert it to a formula
})

# test each model with a logit transformed variable and only 1 education variable
models <- lapply(filtered_formulas, lm, data=train_data)

# calculate the BIC, R squared, adjusted R squared for each model
bics <- sapply(models, BIC)
r_square <- sapply(models, function(m) summary(m)$r.squared)
adj_r_square <- sapply(models, function(m) summary(m)$adj.r.squared)
formula_vector <- vapply(filtered_formulas, function(f) paste(deparse(f), collapse = "")
  , character(1))

# make it into a dataframe
model_ranking <- data.frame(
  formula = formula_vector,
  r.square = r_square,
  adj.r.square = adj_r_square,
  BIC = bics
)

# rank on BIC
model_ranking <- model_ranking[order(model_ranking$BIC), ]
# Print best model
best <- which.min(model_ranking$BIC)
best_pred <- model_ranking$formula[best]
cat("\nBest model:", best_pred)
cat("\nBIC:", round(model_ranking$BIC[best], 2), "\n")
cat("Adjusted R2:", round(model_ranking$adj.r.square[best], 4), "\n")

fit_multi <- lm(best_pred, data = train_data)
multi_var_summary2 <- summary(fit_multi)
multi_var_summary2
# extract all predictors in the best model
predictor_list <- row.names(multi_var_summary2$coefficients[-1,])
# get out all the variables not intercept or poverty
selected_vars <- setdiff(
  row.names(multi_var_summary2$coefficients),
  c("(Intercept)", "PctPopUnderPov")
)

# make all interaction terms
interaction_terms <- paste("PctPopUnderPov", selected_vars, sep = ":")
# make formulas for all
formulas_interaction <- sapply(interaction_terms, function(i) {
  paste("logViolent ~", paste(predictor_list, collapse = " + "), "+",
    i)

```

```

})
formulas_interaction <- lapply(formulas_interaction, as.formula)
# model formulas
models <- lapply(formulas_interaction, lm, data=train_data)
# get formula
interaction_models_form <- vapply(formulas_interaction, function(f)
  paste(deparse(f), collapse = ""), character(1))

# extract p and t values
p_vals_interaction <- sapply(models, function(m)
  summary_val_extraction(m, "Pr(>|t|)"))
t_vals_interaction <- sapply(models, function(m)
  summary_val_extraction(m, "t value"))

# calculate the BIC, R squared, adjusted R squared for each model
bics <- sapply(models, BIC)
r_square <- sapply(models, function(m) summary(m)$r.squared)
adj_r_square <- sapply(models, function(m) summary(m)$adj.r.squared)
delta_adj_r_square <- sapply(models, function(m)
  summary(m)$adj.r.squared - summary(fit_multi)$adj.r.squared)

# add values to table
interaction_table <- data.frame(
  p.vals = p_vals_interaction,
  t.vals = t_vals_interaction,
  r.square = r_square,
  adj.r.square = adj_r_square,
  delta.adj = delta_adj_r_square,
  BIC = bics
)

# sort based on p-values
interaction_table <- interaction_table[order(interaction_table$p.vals), ]
interaction_table

best_interaction <- row.names(interaction_table[1,])
# fitting selected linear model with 1 education variable
formula_final <- as.formula(paste("logViolent ~",
  paste(predictor_list, collapse = " + "), "+",
  best_interaction))
fit_final <- lm(formula_final, data = train_data)
summary(fit_final)
plot(fit_final)
par(mfrow = c(1, 1))
vif <- vif(fit_final)
print(vif)
cat("BIC: ", BIC(fit_final))
# fitting final selected linear model with 1 education variable
formula_final <- as.formula(paste("logViolent ~",
  paste(predictor_list, collapse = " + "), "+",
  "PctPopUnderPov:PctBSorMore"))
fit_final <- lm(formula_final, data = train_data)
final_sums <- summary(fit_final)

```

```

#final_sums
plot(fit_final)
par(mfrow = c(1, 1))
# calculating VIF
vif <- vif(fit_final)
print(vif)
cat("BIC: ", BIC(fit_final))
# partial regression plots for predictor variables
predictor_list <- row.names(multi_var_summary2$coefficients[-1,])
plots <- lapply(predictor_list, function(predictor) {
  partial_regression_plot(train_data, "logViolent", predictor, predictor_list)
})

# Print 9 plots per pg
print(wrap_plots(plots, ncol = 3))
# Compare models
cat("Comparison of models:\n")
cat("Univariate linear model adjusted R²: ", round(summary(fit_simple)$adj.r.squared, 4), "\n")
cat("interaction term model adjusted R²: ", round(summary(fit_final)$adj.r.squared, 4), "\n")

par(mfrow = c(2, 2))

# Residuals vs Fitted
plot(fit_final$fitted.values, fit_final$residuals,
     xlab = "Fitted values", ylab = "Residuals",
     main = "Residuals vs Fitted (Final Model)")
abline(h = 0, col = "red", lty = 2)
lines(lowess(fit_final$fitted.values, fit_final$residuals), col = "blue")

# Squared residuals vs Fitted
plot(fit_final$fitted.values, fit_final$residuals^2,
     xlab = "Fitted values", ylab = "Squared Residuals",
     main = "Squared Residuals vs Fitted")
lines(lowess(fit_final$fitted.values, fit_final$residuals^2), col = "blue")

# QQ-plot
qqnorm(fit_final$residuals, main = "Normal Q-Q Plot (Final Model)")
qqline(fit_final$residuals, col = "red")

# Studentized residuals
stud_res_final <- rstudent(fit_final)
plot(fit_final$fitted.values, stud_res_final,
     xlab = "Fitted values", ylab = "Studentized Residuals",
     main = "Studentized Residuals vs Fitted")
abline(h = 0, col = "red", lty = 2)
lines(lowess(fit_final$fitted.values, stud_res_final), col = "blue")

par(mfrow = c(1, 2))
# Residuals vs log(population)
plot(train_data$logpopulation, fit_final$residuals,
     xlab = "log(population)", ylab = "Residuals")
abline(h = 0, col = "red", lty = 2)

```

```

lines(lowess(train_data$logpopulation, fit_final$residuals), col = "blue")

# Studentized residuals vs log(population)
plot(train_data$logpopulation, stud_res_final,
      xlab = "log(population)", ylab = "Studentized Residuals")
abline(h = 0, col = "red", lty = 2)
lines(lowess(train_data$logpopulation, stud_res_final), col = "blue")

par(mfrow = c(1, 1))
p <- length(coef(fit_final))
n_train <- nrow(train_data)
n_test <- nrow(test_data)

library(olsrr)
olsrr::ols_plot_cooks_bar(fit_final)
olsrr::ols_plot_resid_lev(fit_final, threshold = NULL, print_plot = TRUE)
olsrr::ols_plot_resid_stud_fit(fit_final, threshold = NULL, print_plot = TRUE)
# IMPORTANT remark: the labelled observations are the points with 2 times the threshold (the reason is
dffits_vals <- data.frame(dffits(fit_final))

# DFFITS Plot
plot <- ggplot(dffits_vals) +
  geom_point(aes(x = 1:nrow(dffits_vals), y = dffits_vals[[1]])) +
  geom_segment(aes(x = 1:nrow(dffits_vals), xend = 1:nrow(dffits_vals), y = 0, yend = dffits_vals[[1]]),
              color = 'cornflowerblue') +
  geom_hline(yintercept = c(2, -2) / sqrt(nrow(dffits_vals)), color = 'salmon') +
  labs(x = 'Observation index', y = 'DFFITS', title = 'DFFITS',
       subtitle = 'Thresholds are at  $\pm 2\sqrt{p/n}$ ') +
  geom_text(aes(x = 1:nrow(dffits_vals), y = ifelse(dffits_vals[[1]] > 0, dffits_vals[[1]] + .05, dffits_vals[[1]] - .05),
               label = ifelse(abs(dffits_vals[[1]]) > 2*sqrt(p/nrow(dffits_vals)),
                             paste0(round(dffits_vals[[1]], digits = 2), ' (', 1:nrow(dffits_vals), ')')),
               size = 8))

# Print plot
print(plot)
# IMPORTANT remark: the labelled observations are the points with 1,5 times the threshold (the reason is
# DFBETAS values (placing them in a data frame ensures easy compatibility with ggplot2)
dfbetas_vals <- data.frame(dfbetas(fit_final))

# DFBETAS plot
colnames_dfbetas <- colnames(dfbetas_vals)
plots <- lapply(colnames_dfbetas[-1], function(predictor) {
  ggplot(dfbetas_vals) +
    geom_point(aes(x = 1:nrow(dfbetas_vals), y = dfbetas_vals[[predictor]])) +
    geom_segment(aes(x = 1:nrow(dfbetas_vals), xend = 1:nrow(dfbetas_vals), y = 0, yend = dfbetas_vals[[predictor]]),
                color = 'cornflowerblue') +
    geom_hline(yintercept = c(2, -2) / sqrt(nrow(dfbetas_vals)), color = 'salmon') +
    labs(x = 'Observation index', y = 'DFBETAS',
         title = paste0('DFBETAS values for coefficient of ', predictor),
         subtitle = 'Thresholds are at  $\pm 2\sqrt{p/n}$ ') +
    geom_text(aes(x = 1:nrow(dfbetas_vals), y = ifelse(dfbetas_vals[[predictor]] > 0, dfbetas_vals[[predictor]] + 0.05, dfbetas_vals[[predictor]] - 0.05),
                 label = ifelse(abs(dfbetas_vals[[predictor]]) > 1.5*(2/sqrt(nrow(dfbetas_vals))),
                               paste0(round(dfbetas_vals[[predictor]], digits = 2), ' (', 1:nrow(dfbetas_vals), ')')),
                 size = 8))
})
# Print plots

```

```

print(plots)
# Calculate diagnostics and thresholds
stud_res_final <- rstudent(fit_final)
leverage <- hatvalues(fit_final)
cooks_dist <- cooks.distance(fit_final)
dffits_values <- dffits(fit_final)
dfbetas_valuues <- dfbetas(fit_final)

leverage_threshold <- 2 * p / n_train
dffits_threshold <- 2 * sqrt(p / n_train)
dfbetas_threshold <- 2 / sqrt(n_train)

# new dataframe with all diagnostics
diagnostics <- data.frame(
  obs = 1:n_train,
  # population = train_data$population,
  stud_residual = stud_res_final,
  leverage = leverage,
  cooks_dist = cooks_dist,
  dffits = dffits_values
)

# Flag observations crossing the thresholds
diagnostics$outlier_residual <- abs(diagnostics$stud_residual) > 2
diagnostics$high_leverage <- diagnostics$leverage > leverage_threshold
diagnostics$high_cooks <- diagnostics$cooks_dist > 4 / n_train
diagnostics$high_dffits <- abs(diagnostics$dffits) > dffits_threshold

high_dfbetas <- apply(abs(dfbetas_valuues), 1, function(x) any(x > dfbetas_threshold))

diagnostics$high_dfbetas <- high_dfbetas

# Select flag columns
flag_cols <- c("outlier_residual", "high_leverage", "high_cooks", "high_dffits", "high_dfbetas")

# Count TRUE flags
diagnostics$flag_count <- rowSums(diagnostics[flag_cols])

# summary
cat("Outliers by studentized residuals ( $|r^*| > 2$ ):", sum(diagnostics$outlier_residual), "\n")
cat("High leverage observations ( $h >$ ", round(leverage_threshold, 4), "):",
    sum(diagnostics$high_leverage), "\n")
cat("High Cook's distance ( $D >$ ", round(4/n_train, 4), "):",
    sum(diagnostics$high_cooks), "\n")
cat("High DFFITS ( $|DFFITS| >$ ", round(dffits_threshold, 4), "):",
    sum(diagnostics$high_dffits), "\n")
# Extract flagged observations
problematic <- diagnostics[
  diagnostics$outlier_residual |
  diagnostics$high_leverage |
  diagnostics$high_cooks |
  diagnostics$high_dffits |

```

```

diagnostics$high_dfbetas , ]

problematic_sorted <- problematic[order(-problematic$flag_count), ]

problematic_with_data <- merge(problematic_sorted, train_data, by.x = "obs", by.y = "row.names", all.x = TRUE)

problematic_with_data <- problematic_with_data[order(-problematic_with_data$flag_count), ]

problematic_with_data[, c("flag_count", "communityname", "population", "PctPopUnderPov", "PctBSorMore", "logViolentCrimesPerPop")]

library(MASS)

# robust linear regression
robust <- rlm(formula(fit_final), data = train_data, psi = psi.bisquare)
summary(robust, method = "XtX")
#par(mfrow=c(2,2))
#plot(fit_final)
par(mfrow=c(2,2))
plot(robust, which = 1)
qqnorm(resid(robust))
qqline(resid(robust))

# Compare OLS vs Robust coefficients
comparison <- data.frame(
  OLS = coef(fit_final),
  Robust = coef(robust),
  Difference = coef(fit_final) - coef(robust)
)

print(round(comparison, 4))
summary(fit_final)
print(confint(fit_final))
b <- coef(fit_final)

mean_bsormore <- mean(train_data$PctBSorMore)
mean_poverty <- mean(train_data$PctPopUnderPov)

coef_mean <- b["PctPopUnderPov"] +
  b["PctPopUnderPov:PctBSorMore"] * mean_bsormore

confidence_intervals <- confint(fit_final)
# calculating confidence intervals using bootstrap
ci_pov <- cond_effect_boot(fit_final,
  x = "PctPopUnderPov", w = "PctBSorMore", nboot = 2000)
ci_lower <- ci_pov$"CI Lower"[ci_pov$Level == "Medium"]
ci_upper <- ci_pov$"CI Upper"[ci_pov$Level == "Medium"]
# calculating confidence intervals using bootstrap
ci_pov2 <- cond_effect_boot(fit_final,
  x = "PctBSorMore", w = "PctPopUnderPov", nboot = 2000)
ci_lower2 <- ci_pov2$"CI Lower"[ci_pov2$Level == "Medium"]
ci_upper2 <- ci_pov2$"CI Upper"[ci_pov2$Level == "Medium"]
# adding new columns to total dataframe
crimes_table_subset$logViolent <- log(crimes_table_subset$ViolentCrimesPerPop + 1)

```



```

crimes_table_subset$race_black_logit <- pct_to_logit(crimes_table_subset$racepctblack)

# fitting final multivariate model to total dataframe and to test dataframe
fit_final_test <- lm(formula_final, data = test_data)
fit_final_total <- lm(formula_final, data = crimes_table_subset)
summary(fit_final_test)

# summary of fitting results: fit on training data vs fit on test data vs fit on total data
df1 <- data.frame(
  fit_on_training_data = coef(fit_final),
  fit_on_test_data = coef(fit_final_test),
  fit_on_total_data = coef(fit_final_total),
  p_value_training = summary(fit_final)$coefficients[, 4],
  p_value_test = summary(fit_final_test)$coefficients[, 4],
  p_value_total = summary(fit_final_total)$coefficients[, 4]
)
df1

# summary of fitting results: fit on training data vs fit on test data vs fit on total data
df2 <- data.frame(
  R_squared = c(summary(fit_final)$r.squared, summary(fit_final_test)$r.squared, summary(fit_final_total)$r.squared),
  Adjusted_R_squared = c(summary(fit_final)$adj.r.squared, summary(fit_final_test)$adj.r.squared, summary(fit_final_total)$adj.r.squared),
  BIC_model = c(BIC(fit_final), BIC(fit_final_test), BIC(fit_final_total))
)
rownames(df2) <- c("fit on training data", "fit on test data", "fit on all data")
df2

### Predictions on test set
# backtransformation of the data to the normal scale
backtransformation <- function(value){
  exp(value) - 1
}

# prediction intervals on logscale (final model)
pred_interval_test_log <- predict(fit_final, newdata = test_data, interval = "prediction", level = 0.95)
pred_interval_train_log <- predict(fit_final, newdata = train_data, interval = "prediction", level = 0.95)

# prediction intervals on normal scale (final model)
pred_interval_test <- apply(pred_interval_test_log, 1:2, backtransformation)
pred_interval_train <- apply(pred_interval_train_log, 1:2, backtransformation)

# prediction intervals on normal scale (simple model)
pred_interval_test_simple <- predict(fit_simple, newdata = test_data, interval = "prediction", level = 0.95)
pred_interval_train_simple <- predict(fit_simple, newdata = train_data, interval = "prediction", level = 0.95)
# number of observations in test set inside prediction interval (logscale, final model)
inside_log <- test_data$logViolent >= pred_interval_test_log[, "lwr"] & test_data$logViolent <= pred_interval_test_log[, "upr"]

# number of observations in test set inside prediction interval (normal scale, final model)
inside <- test_data$ViolentCrimesPerPop >= pred_interval_test[, "lwr"] & test_data$ViolentCrimesPerPop <= pred_interval_test[, "upr"]

# number of observations in test set inside prediction interval (normal scale, simple model)
inside_simple <- test_data$ViolentCrimesPerPop >= pred_interval_test_simple[, "lwr"] & test_data$ViolentCrimesPerPop <= pred_interval_test_simple[, "upr"]
cat("Fraction of datapoints in prediction interval (logscale):", mean(inside_log), "\n")
cat("Fraction of datapoints in prediction interval:", mean(inside), "\n")
cat("Fraction of datapoints in prediction interval (univariate model):", mean(inside_simple), "\n")

```

```

# histogram of width of prediction interval (normal scale, final model)
hist(pred_interval_test[, "upr"] - pred_interval_test[, "lwr"],
     main = "Width prediction interval",
     xlab = "Width prediction interval",
     xlim = c(0, 5000),
     breaks = 50)
plot(pred_interval_test[, "fit"], pred_interval_test[, "upr"] - pred_interval_test[, "lwr"], main = "Width prediction interval",
     xlab = "Point estimate", ylab = "Width prediction interval")

# histogram of width of prediction interval (logscale, final model)
hist(pred_interval_test_log[, "upr"] - pred_interval_test_log[, "lwr"],
     main = "Width prediction interval (log)",
     xlab = "Width prediction interval",
     #xlim = c(0, 15000),
     breaks = 50)

# histogram of width of prediction interval (normal scale, simple model)
hist(pred_interval_test_simple[, "upr"] - pred_interval_test_simple[, "lwr"],
     main = "Width prediction interval (univariate model)",
     xlab = "Width prediction interval",
     #xlim = c(0, 15000),
     breaks = 50)

# prediction intervals test data (logscale, final model)
df_prediction_test_logscale <- data.frame(observed = test_data$logViolent, predicted = pred_interval_test_log[, "fit"])

ggplot(df_prediction_test_logscale, aes(observed, predicted))+
  geom_point()+
  geom_smooth(se = FALSE)+
  geom_abline(intercept = 0, slope = 1, color = "red")+
  theme_bw() +
  labs(x = "number of violent crimes per 100K population (observed)",
       y = "number of violent crimes per 100K population (predicted)",
       title = "Test dataset (log scale)")

# prediction intervals training data (logscale, final model)
df_prediction_train_logscale <- data.frame(observed = train_data$logViolent, predicted = pred_interval_train_log[, "fit"])

ggplot(df_prediction_train_logscale, aes(observed, predicted))+
  geom_point()+
  geom_smooth(se = FALSE)+
  geom_abline(intercept = 0, slope = 1, color = "red")+
  theme_bw() +
  labs(x = "number of violent crimes per 100K population (observed)",
       y = "number of violent crimes per 100K population (predicted)",
       title = "Train dataset (log scale)")

# prediction intervals test data (normal scale, simple model)
df_prediction_test_simple <- data.frame(observed = test_data$ViolentCrimesPerPop, predicted = pred_interval_test_simple[, "fit"])

ggplot(df_prediction_test_simple, aes(observed, predicted))+
  geom_point()+
  geom_smooth(se = FALSE)+
  geom_abline(intercept = 0, slope = 1, color = "red")+

```

```

theme_bw() +
labs(x = "number of violent crimes per 100K population (observed)",
     y = "number of violent crimes per 100K population (predicted)",
     title = "Test dataset (univariate)")

# prediction intervals training data (normal scale, simple model)
df_prediction_train_simple <- data.frame(observed = train_data$ViolentCrimesPerPop, predicted = pred_in

ggplot(df_prediction_train_simple, aes(observed, predicted))+
  geom_point()+
  geom_smooth(se = FALSE)+
  geom_abline(intercept = 0, slope = 1, color = "red")+
  theme_bw() +
  labs(x = "number of violent crimes per 100K population (observed)",
       y = "number of violent crimes per 100K population (predicted)",
       title = "Train dataset (univariate)")

# MSPR for test data
mspr_test_log <- mean((test_data$logViolent - pred_interval_test_log[, "fit"])^2)

# MSE for training data
mse_train_log <- sum((train_data$logViolent - pred_interval_train_log[, "fit"])^2)/(n_train - p)

cat("MSPR:", round(mspr_test_log, 2), "\n")
cat("MSE:", round(mse_train_log, 2), "\n")
cat("Ratio MSPR/MSE:", round(mspr_test_log/mse_train_log, 4), "\n")

# R-squared on test data
ss_res_test <- sum((test_data$logViolent - pred_interval_test_log[, "fit"])^2)
ss_tot_test <- sum((test_data$logViolent - mean(test_data$logViolent))^2)
r2_test_log <- 1 - ss_res_test/ss_tot_test
cat("R^2 on test set:", round(r2_test_log, 4), "\n")

# R-squared on training data
ss_res_train <- sum((train_data$logViolent - pred_interval_train_log[, "fit"])^2)
ss_tot_train <- sum((train_data$logViolent - mean(train_data$logViolent))^2)
r2_train_log <- 1 - ss_res_train/ss_tot_train
cat("R^2 on training set:", round(r2_train_log, 4), "\n")

# Results for multivariate model logscale
validation_logscale <- c("log(ViolentCrimesPerPop)", round(mspr_test_log, 2), round(mse_train_log, 2),

# MSPR for test data
mspr_test_simple <- mean((test_data$ViolentCrimesPerPop - pred_interval_test_simple[, "fit"])^2)

# MSPR for training data
mse_simple <- sum((train_data$ViolentCrimesPerPop - pred_interval_train_simple[, "fit"])^2)/(n_train - p)

cat("MSE:", round(mse_simple, 2), "\n")
cat("MSPR:", round(mspr_test_simple, 2), "\n")
cat("Ratio MSPR/MSE:", round(mspr_test_simple/mse_simple, 4), "\n")

# R-squared on test data
ss_res_test_simple <- sum((test_data$ViolentCrimesPerPop - pred_interval_test_simple[, "fit"])^2)
ss_tot_test <- sum((test_data$ViolentCrimesPerPop - mean(test_data$ViolentCrimesPerPop))^2)

```

```

r2_test_simple <- 1 - ss_res_test_simple/ss_tot_test
cat("R2 on test set:", round(r2_test_simple, 4), "\n")

# R-squared on training data
ss_res_train_simple <- sum((train_data$ViolentCrimesPerPop - pred_interval_train_simple[, "fit"])^2)
ss_tot_train <- sum((train_data$ViolentCrimesPerPop - mean(train_data$ViolentCrimesPerPop))^2)
r2_train_simple <- 1 - ss_res_train_simple/ss_tot_train
cat("R2 on training set:", round(r2_train_simple, 4), "\n")

# Results for univariate model (normal scale)
validation_simple <- c("ViolentCrimesPerPop", round(mspr_test_simple, 2), round(mse_simple, 2), round(mse_simple/mspr_test_simple, 2))

df_validation <- data.frame(multivar_logscale = validation_logscale, univar = validation_simple)
rownames(df_validation) <- c("outcome variable", "MSPR", "MSE", "Ratio MSPR/MSE", "R2 (test)", "R2 (training)")
df_validation

```

```

# Create the table in R
schedule <- data.frame(
  Subject = c("Data extraction",
              "Data preparation",
              "Descriptive analyses",
              "Model building",
              "Model diagnostics",
              "Model interpretation",
              "Model validation",
              "Statistical discussion linear model",
              "Final conclusion and discussion"),
  Code = c("Thomas",
           "Ilja",

```

```

      "Ilja",
      "Thomas",
      "Thomas&Ilja",
      "Lieselot",
      "Ilja",
      "Lieselot",
      "Lieselot"),
Text = c("Thomas",
      "Ilja",
      "Ilja",
      "Thomas",
      "Thomas",
      "Lieselot",
      "Ilja",
      "Lieselot&Ilja",
      "Lieselot&Ilja")
)

# Print the table
kable(schedule, caption = "Project contribution Overview", align = c('l', 'c', 'c'))

```